

Department of Engineering and Computer



Hybrid Text Steganography System Using Bit Mapping & Invisible Characters

Prepared by Cyber Security Students

Names	Student ID
<i>Abdullah Adel Mohammed Nasser</i>	202210200930
<i>Firas Salah</i>	202210200898
<i>Mohammed salah</i>	202210201684
<i>Ali Abdualh</i>	202210200920
<i>Mohammed Khaled</i>	202210201694

Under supervision

Eng. Mohammed Ali

Abstract

The challenge of concealing information within digital texts continues to be one of the main problems of cybersecurity caused by the lack of inherent redundant storage capacity, which often requires balancing the trade-off between payload capacity, visual invisibility, and cryptographic safety. This study proposes and implements an innovative hybrid structural text steganography technique. The proposed approach uses a decentralized and client-side-based web architecture, including Brotli stream compression algorithm (WebAssembly) and AES-256-CTR encryption (PBKDF2 key derivation). The result is the embedding of the encrypted payload into the cover text through the use of dynamic bit-mapping with the use of invisible Unicode Variation Selectors (VS). As the result of evaluation tests, outstanding quantitative and qualitative performance has been obtained: up to 100% capacity ratio along with absolute visual invisibility without any distortions was achieved. Besides, the produced stego-texts showed their perfect robustness being capable of passing through text-sanitization and copy-pasting through major social media Platforms. The main contribution of this research is the working steganographic web application without any need of a server, which ensures zero-trust boundary protection by processing all data locally in the browser sandbox.

Keywords: Structural Text Steganography, Information Hiding, Invisible Characters, Variation Selectors, Bit-Mapping, Social Media Platforms.

1. Contents

1.	Introduction.....	5
1.1	Background and Context	6
1.2	Objectives	7
1.3	Scope & Limitations	8
1.4	Problem Statement.....	8
2.	Literature review	10
2.1	Domain Overview	11
2.1.1	Definition of Terms	11
2.2	Existing Approaches and Tools.....	13
2.2.1	Approach (emoji-encoder).....	13
2.2.2	Approach (Zero Width Characters)	14
2.2.3	Approach (New Text Steganography Technique based on Multilayer Encoding with Format-Preserving Encryption and Huffman Coding)	15
2.2.4	Approach (AITSteg: An Innovative Text Steganography Technique for Hidden Transmission of Text Message via Social Media).....	16
2.2.5	Approach (SNOW Tool)	17
2.2.6	Approach (A High Capacity and Imperceptible Text Steganography).....	18
2.2.7	Approach (A Novel Approach of Bulk Data Hiding using Text Steganography)	19
2.2.8	Approach (ASCII Art Steganography)	20
2.2.9	Approach (A New 3-Bit Hiding Covert Channel Algorithm for Public Data and Medical Data Security Using Format Based Text Steganography)	21
2.2.10	Approach (Text Steganography Using Character Spacing after Normalization).....	22
2.2.11	Approach (A new method for pdf steganography in justified texts).....	24
2.2.12	Approach (A high-capacity text steganography scheme based on LZW compression and color coding).....	25
2.2.13	Approach (A Space based reversible high-capacity text steganography scheme using Font type and style)	26
2.2.14	Approach (A Novel Approach to Format Based Text Steganography)	27
2.2.15	Approach (Emoticon-based Steganography for Securing Sensitive Data)	28
2.2.16	Approach (Hiding Information in Email Texts Using Repeated Characters)	30
2.3	Comparative analysis.....	31
2.4	Identified gap	35
3.	Methodology	36
3.1	Research Design	37
3.2	Environment and Tools	38
3.3	Procedures and Metrics	40
3.3.1	Procedures	40
3.3.2	Evaluation Metrics.....	42
3.4	Standards applied	43
4.	System Design & Implementation	45

4.1	Architecture	46
4.1.1	System Diagrams	46
	Logical diagrams:	46
	Sequence Diagram of the Chat Scanner Extraction Pipeline	63
4.1.2	Design Principles	68
4.2	Components	70
4.2.1	System Modules	70
4.2.2	System Interfaces	76
4.3	Security Controls	76
4.3.1	Preventive controls	76
4.4	Implementation details	76
4.4.1	Languages and frameworks	76
4.4.2	Testing	77
4.5	Threat modeling	80
4.6	System Documentation	83
4.6.1	Embedding WorkFlow	83
4.6.2	Extraction WorkFlow	84
4.6.3	Technical Report for Technical People	86
5.	Results	88
5.1	Experimental setup recap	89
5.1.1	Baseline	89
5.1.2	Test cases	90
5.2	Findings	93
5.2.1	Quantitative results	93
5.2.2	Qualitative insights	95
6.	Analysis & Discussion	97
6.1	Analysis	98
6.1.1	Effectiveness	98
6.2	Validation	101
7.	Conclusion	103
8.	References	105
9.	Appendices	108
9.1	Source code excerpts	109
	Unit & Integration Tests	109
	ESLint Security Static Analysis	120
10.	Terms and Acronyms	122

FIGURE 2.1 ATTACHING THE DATA-REPRESENTING VARIATION SELECTORS TO THE CARRIER SYMBOL.....	13
FIGURE 2.2 ZERO WIDTH CHARACTERS.....	15
FIGURE 2.3 AN ILLUSTRATION OF EMBEDDING AN SM IN FRONT OF CM.....	16
FIGURE 2.4 EXAMPLE OF WHITE SPACE	17
FIGURE 2.5 BIT MAPPING	18
FIGURE 2.6 STEGOKEY.....	18
FIGURE 2.7 CHAOTIC SEQUENCE MATRIX STORED VALUES	19
FIGURE 2.8 ASCII ART STEGANOGRAPHY	20
FIGURE 2.9 ILLUSTRATED EXAMPLE OF PROPOSED ALGORITHM USING TWITTER DATA	21
FIGURE 2.10 EMBEDDING OF VALUES 0 TO 15 IN A HOST LINE	24
FIGURE 2.11 COLOR MAPPING.....	25
FIGURE 2.12 BIT STREAM PARTITIONING	25
FIGURE 2.13 TEXT COLOR ENCODING PROCESS.....	25
FIGURE 2.14 A SPACE BASED REVERSIBLE HIGH-CAPACITY TEXT STEGANOGRAPHY SCHEME USING FONT TYPE AND STYLE	27
FIGURE 2.15 EXAMPLE OF LINE SHIFTING METHOD	28
FIGURE 2.16 EXAMPLE OF WORD SHIFING METHOD	28
FIGURE 218.THE BIT-STREAM OBTAINED FROM XYZ.....	30
FIGURE 2.17 SIMPLE SECRET MESSAGE	30
FIGURE 3.1 RESEARCH WORKFLOW	38
FIGURE 3.2 PROCEDURES	42
FIGURE 4.1 SENDER DATA FLOW DIAGRAM	47
FIGURE 4.2 RECEIVER DATA FLOW DIAGRAM	48
FIGURE 4.3 SENDER WORK FLOW DIAGRAM.....	50
FIGURE 4.4 RECEIVER WORKFLOW DIAGRAM.....	53
FIGURE 4.5 RECEIVER WORKFLOW (SCANNER) DIAGRAM	56
FIGURE 4.6 SEQUENCE DIAGRAM FOR FULL SYSTEM PIPELINE (SENDER)	58
FIGURE 4.7 SEQUENCE DIAGRAM FOR FULL SYSTEM PIPELINE (RECEIVER)	61
FIGURE 4.8 SEQUENCE DIAGRAM FOR FULL SCANNER PIPELINE (RECEIVER)	63
FIGURE 4.9 PHYSICAL DIAGRAM OF THE SYSTEM.....	66
FIGURE 4.10 TRUST BOUNDARY DIAGRAM.....	67
FIGURE 4.11 PSEUDO CODE FOR UI MODULE (SENDER SIDE)	70
FIGURE 4.12 PSEUDO CODE FOR UI MODULE (RECEIVER SIDE)	71
FIGURE 4.13 PSEUDO CODE FOR SCANNER ENGINE	72
FIGURE 4.14 PSEUDO CODE FOR CO/DECOMPRESSION ENGINE	73
FIGURE 4.15 PSEUDO CODE FOR EN/DECRYPTION ENGINE.....	74
FIGURE 4.16 PSEUDO CODE FOR STEGANOGRAPHIC ENGINE.....	75
FIGURE 4.17 SUITES & TESTS TESTING	77
FIGURE 4.18 EMBEDDING PAGE	83
FIGURE 4.19 EMBEDDING RESULT	84
FIGURE 4.20 SENDING THE STEGO-TEXT.....	84
FIGURE 4.21 RECEIVING THE STEGO-TEXT	84
FIGURE 4.22 EXTRACTING PAGE	85
FIGURE 4.23 EXTRACTION RESULT.....	85
FIGURE 4.24 STEGANOGRAPHY HEX STREAM.....	86
FIGURE 4.25 PRNG RANDOM BASE POSITIONS	86
FIGURE 4.26 BINARY XOR KEY DIFFERENCE MASK.....	87
FIGURE 5.1 WHEN COVER TEXT SIZE IS BIGGER THAN SECRET TEXT SIZE	89

FIGURE 5.2 WHEN COVER TEXT SIZE IS SLIGHTLY BIGGER THAN SECRET TEXT SIZE	89
FIGURE 5.3 WHEN BOTH SIZES ARE EQUALS	90
FIGURE 5.4 WHEN SECRET TEXT SIZE IS BIGGER THAN COVER TEXT SIZE	90
FIGURE 9.1 REPO	خطأ! الإشارة المرجعية غير معروفة.
FIGURE 9.2 XOR MASKING TESTS	خطأ! الإشارة المرجعية غير معروفة.
FIGURE 9.3 PRNG POSITION GENERATOR TESTS.....	خطأ! الإشارة المرجعية غير معروفة.
FIGURE 9.4 SHA-256 HASHING TESTS.....	خطأ! الإشارة المرجعية غير معروفة.
FIGURE 9.5 VS CODEC (VARIATION SELECTOR ENCODING) TESTS.....	خطأ! الإشارة المرجعية غير معروفة.
FIGURE 9.6 TEXT CODEC (BINARY ↔ TEXT CONVERSION) TESTS	خطأ! الإشارة المرجعية غير معروفة.
FIGURE 9.7 PAYLOAD CODEC (BUILD & PARSE) TESTS	خطأ! الإشارة المرجعية غير معروفة.
FIGURE 9.8 BROTLI COMPRESSION TESTS	خطأ! الإشارة المرجعية غير معروفة.
FIGURE 9.9 AES-CTR ENCRYPTION TESTS.....	خطأ! الإشارة المرجعية غير معروفة.
FIGURE 9.10 FULL STEGO PIPELINE INTEGRATION TESTS.....	خطأ! الإشارة المرجعية غير معروفة.
FIGURE 9.11 ESLINT SECURITY STATIC CODE ANALYSIS	خطأ! الإشارة المرجعية غير معروفة.
TABLE 2.1 MAPPING OF 2-BIT BINARY CLASSIFICATIONS TO UNICODE CONTROL CHARACTERS	14
2.2 UNICODE MAP FOR HIDING 3 BITS IN EACH CHARACTER	15
TABLE 2.3 MAPPING MATRIX OF ZERO-WIDTH CHARACTERS TO 2-BIT BINARY SEGMENTS IN AITSTEG	16
TABLE 2.4 ENCODING 3 BITS USING SPACES	17
TABLE 25. 3-BIT EMBEDDING RULES	22
TABLE 2.6 CUMULATIVE FREQUENCY	23
TABLE 2.7 CHARACTER SPACING	23
TABLE 2.8 APPROACHES (A SPACE BASED REVERSIBLE HIGH-CAPACITY TEXT STEGANOGRAPHY SCHEME USING FONT TYPE AND STYLE)	26
TABLE 2.9 INITIALIZED DICTIONARY	28
TABLE 2.10 GENERATED DICTIONARY	29
TABLE 211. COMPARATIVE ANALYSIS OF LITERATE STUDIES	34
TABLE 4.1 THREAT MODELING	82
TABLE 5.1 CAPACITY TEST RESULTS	93
TABLE 5.2 COMPRESSION TEST RESULTS TO EVALUATE THE EFFICIENCY OF THE BROTLI ALGORITHM	93
TABLE 5.3 STEGO-TEXT EXPANSION RATIO TEST RESULTS	94
TABLE 5.4 SECURITY TEST PARAMETERS	94
TABLE 5.5 ESTIMATED CRACKING TIME BREAKDOWN	94
TABLE 5.6 ESTIMATED CRACKING TIME BREAKDOWN	95
TABLE 5.7 ESTIMATED CRACKING TIME BREAKDOWN	96
TABLE 10.1 TERMS	124

1. Introduction

1.1 Background and Context

Considering the contemporary digital landscape, the feeling of surveillance over messages or content is no longer merely personal obsessions for users but has become a reality imposed by frequent security breaches and the misuse of data by companies. The danger of this informational exposure is manifested in documented incidents; in September 2018, Facebook revealed a technical vulnerability that led to the leakage of Access Tokens for about 50 million users, making users' data and correspondence exposed to hackers [1], followed by the exploitation of the critical vulnerability (CVE(Common Vulnerabilities and Exposures)-2019-3568) in the WhatsApp system by the NSO Group to pass the spyware [2] 'Pegasus'. The matter was not limited to external breaches but extended to include the internal misuse of data by employees, as appeared in the 'SnapLion' tool belonging to Snapchat [3].

These events led to a decline in user trust, and here the need emerged to find unconventional communication methods that reduce the likelihood of detection or suspicion. Although encryption techniques have evolved, they remain subject to suspicion, which highlights the science of Steganography as an alternative for exchanging and concealing sensitive messages in a manner that renders the secret text unnoticeable. While concealment media (stego-objects) vary, such as images and video clips, Text Steganography stands out as a more practical and harmonious option, specifically in cases of frequent and continuous use. In an instant messaging environment, exchanging short texts is the natural and prevailing pattern of communication. In contrast, the frequent reliance on sending media files (such as images) to pass short messages is considered an impractical approach and often arouses doubt and suspicion due to its deviation from the typical behavior of users. Therefore, text steganography provides a seamless and familiar stealth environment that naturally integrates into the context of daily conversations without drawing attention.

However, text steganography is considered one of the most difficult fields of data concealment due to the lack of Redundant Bits compared to other file types such as images, video, and audio, and due to the severe sensitivity of the human eye to any difference between the original text and the modified text (stego-text), which makes detecting changes relatively easy [4].

Therefore, the need arises to develop a text steganography method that preserves its natural appearance and contextual semantics, while considering the fundamental efficiency criteria for steganography systems, and at the same time ensures that the hidden message remains extractable.

1.2 Objectives

This research primarily aims to design and develop an innovative method for hiding data within texts, relying on the employment of invisible symbols to represent the secret text. To achieve this end, the research seeks to accomplish the following objectives:

1.2.1 **Developing the concealment mechanism:** Inventing an algorithm to embed secret messages within texts using invisible symbols, while completely maintaining the overall appearance of the carrier text without causing any visual distortions.

1.2.2 **Improving concealment capacity:** Employing powerful compression mechanisms to reduce the size of the secret text, which doubles the available storage space (concealment capacity) within the cover text and allows sending a larger amount of information with high efficiency.

1.2.3 **Achieving Imperceptibility:** Enhancing the system's stealth capability so that the average user or traditional monitoring systems are unable to distinguish any differences between the original text and the modified text.

1.2.4 **Enhancing system Robustness:** Ensuring that the hidden data remains intact and accurately extractable, even after the carrier text is subjected to transmission processes, such as copying and pasting across various communication platforms.

1.2.5 **Adding an encryption layer:** Providing an additional level of protection using a strong encryption algorithm to secure the secret text, provided that this algorithm prevents any extra increase in data size (Overhead) to preserve the limited space.

1.2.6 **Developing an applied tool:** Programming and building an integrated and user-friendly tool that smoothly executes the encryption, compression, and concealment processes.

1.3 Scope & Limitations

The scope of this research is limited to Structural Text-Steganography, which deals with altering the format, layout, or structural element of a medium. And using only text as a medium for hiding secret data. Bitmapping and Variation Selectors (VS) will be used as embedding techniques within the cover text. Finally, the resulting stego-text will be tested across multiple social media platforms to ensure the capability of sending and receiving without any problems.

The research excludes the hiding of data within any digital media other than text, such as images, videos, audios, and any other media that is not text. In addition to the other approaches of Steganography will be excluded. With only focusing on Structural Text-Steganography approach.

The research is constrained to limited social media platforms, the tests will only be on the following platforms: WhatsApp, Facebook, Instagram, X / Twitter, Telegram, TikTok, YouTube (Comments), LinkedIn, Snapchat. In addition to the size of a single VS character, that lies between two and four bytes for a single VS, which may lead to high expansion ratio in the size of the cover text (CT).

1.4 Problem Statement

Concealing data within texts is considered one of the most challenging branches of steganography, due to its fundamental limitations compared to other concealment media such as images, audio, and video. Inherently, this field suffers from a clear limitation in capacity due to the absence of redundant bits in texts, which restricts the size of the secret texts that can be embedded. In addition, most of the proposed textual methods demonstrate difficulty in achieving a balance between the fundamental requirements for the success of any steganography system: concealment capacity, unnoticeability, robustness, and Security.

Despite the efforts presented by previous studies to develop text steganography methods, they still suffer from fundamental research gaps in fulfilling those requirements. In terms of capacity, previous techniques provided very minimal concealment spaces; either due to a shortcoming in the concealment mechanisms themselves, or due to the lack of employing strong compression algorithms prior to the embedding process. From the security perspective, some studies relied on traditional concealment methods that are easily detected, while others resorted to the use of encryption algorithms to add an additional layer of protection, except that the algorithms were old or weak.

As for robustness, the most prominent weakness lies in the restriction of many methods to operate within specific platforms or environments, which deviates from the natural pattern of communication, alongside the hidden messages being severely affected by the filtering and processing factors imposed by social media platforms, which often leads to the complete loss of the message. Finally, many techniques suffer from minor distortion due to their reliance on inducing direct changes in the structure or formatting of the text.

Based on the above, the research problem is summarized in the absence of an integrated text steganography system that achieves an efficient balance between storage capacity and security, and possesses sufficient robustness to withstand filtering, copying, and pasting operations without leaving any visual or stylistic trace in the carrier text when transmitted across various social media platforms.

2. Literature review

2.1 Domain Overview

Steganography is considered one of the branches of Information Hiding, and it refers to the art of hiding information inside media that appear natural and unsuspected, such as images, videos, audio files, protocols, and texts, in ways that prevent the discovery of hidden messages [5], this research focuses on developing a hybrid Text Steganography approach that combines more than one hiding mechanism, taking into account the Stegosystem requirements represented in: providing a mechanism in which Imperceptibility is high as it does not visually affect the Cover Media, and with the use of a strong compression algorithm that ensures excellent Embedding Capacity, and providing Robustness and using strong encryption to ensure Security.

2.1.1 Definition of Terms

A **stegosystem** is composed of a message embedded in the cover medium along with a stegokey. Taken together, these three items create a stegomedium. The recipient of this medium will use the stegokey and the stegomedium to derive the (embedded) message. [6]

Steganalysis is the process of identifying suspected objects and determining whether they contain an encoded payload (stego object) or not (cover object). [7]

Security refers to the ability of the hiding mechanism to withstand attempts by unauthorized parties to detect or access the hidden data, whether through attacks or advanced statistical analysis software (Steganalysis). [8]

The cover media/carries are used by the user to hide a message within another message, such as video, images, and sounds. [9]

Embedding/hiding It is the mechanism that is used to hide a secret text inside the cover. [10]

The secret text/payload is the information that is embedded in the cover message using invisible symbols. [11]

Stego-media/object are the outputs of steganography technique and the process of embedding secret data. [12]

Extraction/recovery algorithm is the process that the receiver can simply use to recover the secret text from the stego-text, stego-images, etc. [13]

Stego-key: -It is key that embeds the data into a cover and for extraction of data from the stego medium. [14]

Imperceptibility refers to how many perceptual changes are made in the cover text after embedding a watermark. The trace of embedding hidden data is completely unnoticeable. [15]

Robustness The capability of the hidden data to withstand and survive any modification, corruption, or destruction by third parties. [9]

Imperceptibility (Invisibility) is the ability to hide secret data so perfectly that no changes are noticeable to the human eye or statistical tools. [12]

Payload Capacity is the maximum amount of secret data (number of bits) that can be embedded inside the cover text. [11]

Payload capacity: refers to the amount of data that can be embedded in steganographic images, typically measured in bits. It is influenced by the size of the cover image and the method used .[16]

2.2 Existing Approaches and Tools

2.2.1 Approach (emoji-encoder)

In 2025 [17], Paul Butler introduced the "Emoji Encoder" tool as a technique that employs a part of the Unicode standard. The tool relies on appending Variation Selectors (VS) symbols located within the ranges (U+FE00–U+FE0F) and (U+E0100–U+E01EF) to the preceding character, which are non-printing symbols originally used to modify the appearance of letters and emojis. The algorithm converts the secret text into bytes, then maps each byte to one of 256 available variation selectors, to be subsequently appended as an invisible symbol behind a visible carrier symbol, such as emojis or regular alphabetical letters. This approach features high spatial embedding efficiency compared to traditional methods that use Zero Width Characters, as each selector represents a full byte. Furthermore, most platforms are programmed to preserve them and not ignore them during copy and paste operations, which ensures that the hidden data remains intact without leaving any visual trace or corrupting the formatting of the original text. Despite these advantages, the technique faces several limitations; most notably the possibility of the text cover (emoji) not displaying correctly across all platforms, and problems of exceeding the allowed length limits in input interfaces because of the size of the selectors, and its lack of a data compression algorithm. In addition to that, this mechanism is considered easy to detect upon technical inspection, as the anomalous strings of variation selectors can be easily discovered using Regular Expressions (Regex), and this tool also did not provide any encryption algorithm as an additional layer of protection.

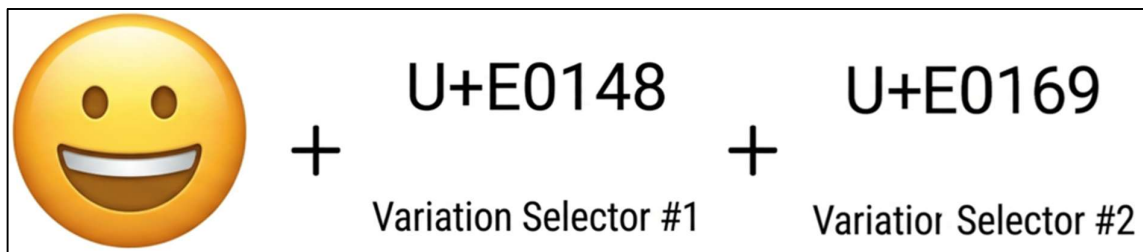


Figure 2.1 Attaching the data-representing Variation Selectors to the carrier symbol

2.2.2 Approach (Zero Width Characters)

In 2023, Priyansh Sharma [18] introduced an open-source information hiding tool that supports multiple media types, including images, video, audio, and text steganography. The text-based approach relies on concealing a secret text within a normal cover text by inserting invisible Unicode symbols, known as Zero Width Characters (ZWC), between the words of the text. The secret text is first converted into a binary representation and then subjected to a simple XOR operation using a fixed constant to introduce a basic level of obfuscation prior to embedding. Subsequently, the resulting binary representation is segmented and mapped to invisible symbols according to a predefined encoding table. These symbols are then embedded into a pre-prepared cover text without causing any visible modification to its content. The hidden message is later recovered by extracting these invisible symbols and converting them back to their original form.

Although this method achieves a basic level of Imperceptibility to human readers, its overall security and robustness remain limited. The reliance on a fixed XOR key provides weak cryptographic protection, making the hidden data vulnerable to extraction once the embedding mechanism is known. In addition, the embedding capacity is relatively low, as hiding each secret character requires multiple words from the cover text, which limits scalability when handling large messages. Moreover, the strictly sequential embedding pattern introduces regularities that can be detected using statistical steganalysis techniques. Motivated by these limitations, the present research aims to address the security issue by integrating a strong encryption algorithm prior to the embedding process, while also proposing a method that offers higher embedding capacity compared to this approach, thereby enhancing both the confidentiality and practical efficiency of the proposed system.

Unicode Character Type	Hex Code	2-bit Classification
Zero Width Non-Joiner (ZWNJ)	0x200C	00
Pop Directional Formatting (PDF)	0x202C	01
Left-to-Right Mark (LRM)	0x200E	10
Left-to-Right Override (LRO)	0x202D	11

Table 2.1 Mapping of 2-bit Binary Classifications to Unicode Control Characters

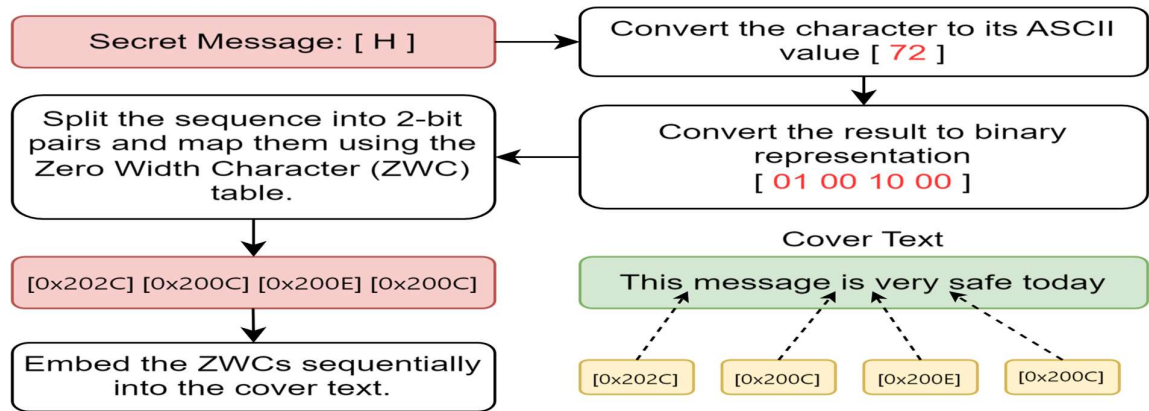


Figure 2.2 Zero Width Characters

2.2.3 Approach (New Text Steganography Technique based on Multilayer Encoding with Format-Preserving Encryption and Huffman Coding)

Majid et al. (2022) [19] presented a development of the data hiding mechanism in texts using invisible Unicode characters (ZWC), where he increased the size of each binary block from two bits to three bits. They added Format-Preserving Encryption (FPE) as an additional protection layer and used the (Huffman) algorithm to reduce the size of the hidden message. Methodologically, the tool first encrypts the text, which protects the data without increasing its size, then compresses the data using Huffman to further reduce its size, then divides the message into several blocks, each block consisting of three bits. After that, it converts the block into an invisible character and sequentially inserts these symbols in the spaces between each word, which preserves its visual appearance derived from the original text. The results showed high efficiency with a hiding capacity ratio of 18.4% of the cover text and excellent visual Imperceptibility with a Jaro-Winkler average of 0.984. Despite these featuresA, it does not support all languages as a text cover, such as the Arabic language, whose appearance is affected after using a hidden character that impacts the text direction.

Unicode characters	Abbreviation	Representation
Zero width character	ZWC	000
Zero width joiner	ZWJ	001
Zero width no- joiner	ZWNJ	010
Invisible plus	IP	011
Invisible separator	IS	100
Inhabit Symmetric Swapping	ISS	101
Empty string	“”	110
Left-To-Right Embedding	LRE	111

2.2 Unicode MAP FOR HIDING 3 BITS IN EACH CHARACTER

2.2.4 Approach (AITSteg: An Innovative Text Steganography Technique for Hidden Transmission of Text Message via Social Media)

Ahvanooey et al. (2018) [20] proposed the AITSteg technique, a text steganography technique that relies on invisible ZWC symbols. The methodology begins by encoding the secret text at the character level using a mathematical encoding approach based on ASCII (Gödel numbering), where each character is represented as a unique pair (α, β) and then converted into a binary stream. To enhance security, an XOR operation was performed between the message and a key derived from the time the message was sent, resulting in an ambiguous binary sequence. During the embedding process, the binary sequence is divided into 2-bit segments and then converted into (ZWC) based on a predefined table, creating an invisible hidden string that is inserted at the beginning of the cover message to ensure maximum robustness. Results indicate that the technique achieves a high degree of Imperceptibility, given that the payload consists of ZWC, which is invisible during reading. The authors also demonstrated practical feasibility through transmission testing across various messaging applications and social media platforms. Despite the strong advantages it provides, not all platforms and communication applications allow ZWC symbols. Furthermore, text transformations, such as copying and pasting, can destroy the hidden message. Although invisible to human readers, ZWC characters remain detectable by statistical analysis through raw text examination. Moreover, reliance on time-derived keys can be fragile in scenarios involving transmission delays, which increases decoding errors, as well as the noticeable consumption of cover capacity, as representing a single character requires six ZWC symbols.

Unicode Character	Description (Short Name & Full Name)	Binary Value
\u200C	ZWNJ - Zero Width Non-Joiner	00
\u202C	PDF - Pop Directional Formatting	01
\u202D	LRO - Left-to-Right Override	10
\u200E	LRM - Left-to-Right Mark	11

Table 2.3 Mapping Matrix of Zero-Width Characters to 2-bit Binary Segments in AITSteg

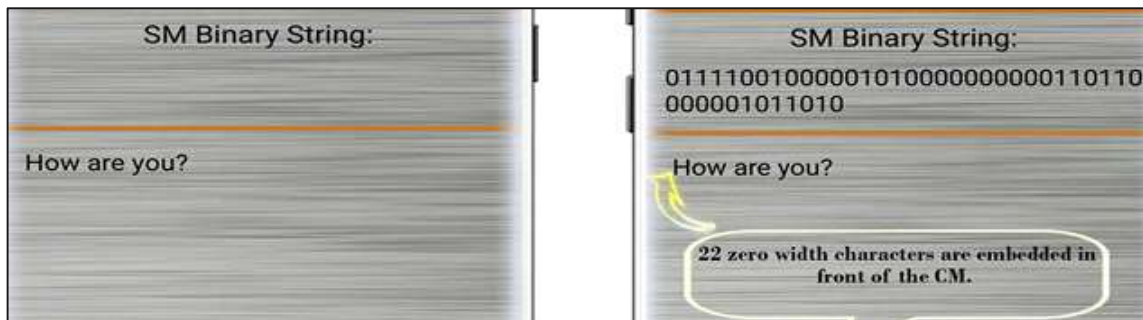


Figure 2.3 An illustration of embedding an SM in front of CM.

2.2.5 Approach (SNOW Tool)

Matthew Kwan (2013) [21] developed the (SNOW) tool, as the tool relied on exploiting spaces (Spaces) and tabs (Tabs) as a separator between each group of spaces without changing the content of the visible cover. Before the concealment mechanism begins, the secret text is converted into a series of bits, with the application of optional compression using the Huffman algorithm. After that, symmetric encryption based on the ICE algorithm in 1-bit Cipher Feedback (CFB) mode is used to secure the data before the concealment process. The encoding method relies on dividing the bits into groups of three bits, where each group is represented by a specific number of spaces, with the use of the tab (TAB) as a separator that defines the start of the hidden data in the line. Despite the advantages the tool provides, such as compression and encryption, it has several drawbacks, as space patterns may be noticed, making them susceptible to statistical analysis, and its complete reliance on space and tab patterns imposes constraints, as any text processing operations, such as copying and pasting, or reformatting lines, may lead to the removal or modification of these patterns, resulting in the loss of the hidden message.

Bits	Value	Encoding
000	0	Tab
001	1	Tab + 1 space
010	2	Tab + 2 spaces
011	3	Tab + 3 spaces
100	4	Tab + 4 spaces
101	5	Tab + 5 spaces
110	6	Tab + 6 spaces
111	7	Tab + 7 spaces

Table 2.4 Encoding 3 Bits Using Spaces

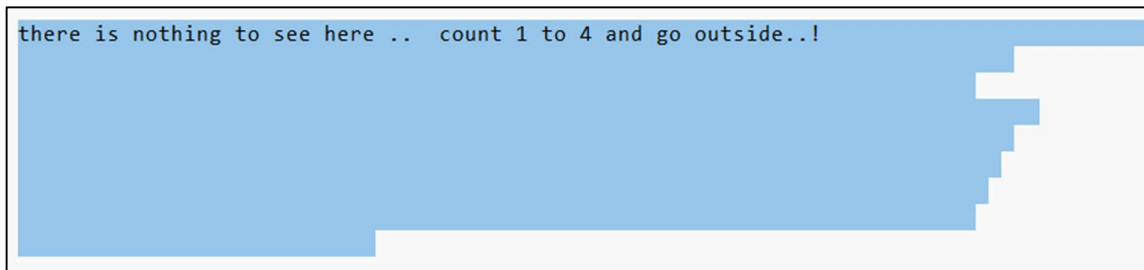


Figure 2.4 example of white space

2.2.6 Approach (A High Capacity and Imperceptible Text Steganography)

Naharuddin et al. (2018) [22] proposed a text steganography method based on binary bit mapping of ASCII representation between the secret text and the cover text, where the secret text is first encrypted using a One-Time Pad (OTP) and then converted into a 7-bit binary stream per character, while the cover text is directly converted into binary ASCII representation. The hiding process is performed by matching each bit of the secret text with a corresponding bit within one of the seven-bit positions of a character from the cover text, without making any visible modifications to the text, while recording the bit positions used in a stego-key to be used later in the extraction process. Results showed that the resulting text is perfectly identical to the cover text in terms of appearance, as the similarity value using the Jaro-Winkler Distance metric reached the maximum value (1); the method also achieved a hiding capacity of approximately 14.2%, meaning that each cover character can carry one bit of the secret text. Despite the high Imperceptibility and the good security level provided by this method resulting from the combination of encryption and steganography, it suffers from several practical limitations as it relies entirely on the stability of the cover text's character sequence, making it extremely fragile against any addition or deletion operations, which leads to a total failure in the extraction process. Additionally, the length of the stego-key increases proportionally with the length of the secret text, which imposes an additional burden on key management and reduces practical applicability in real-world environments.

		1	1	1	3	2	2	4	→ Stego-Key
		1	2	3	4	5	6	7	Sequence number
Z		1	0	1	1	0	1	0	→ Secret Text
		7	6	5	4	3	2	1	Sequence number
A		1	0	0	0	0	0	1	} Cover Text
B		1	0	0	0	0	1	0	
C		1	0	0	0	0	1	1	
D		1	0	0	0	1	0	0	
E		1	0	0	0	1	0	1	
F		1	0	0	0	1	1	0	
G		1	0	0	0	1	1	1	
ASCII		Binary digits (bit) value							

Figure 2.5 Bit Mapping

121112511211131222622112651111122411221213213241112214151426121121
111141152426224212112111412261111221112232324411421112121261111143
121311115311511251131534213212113132612312311121126253214352342151
33314111113111111121312111121153223222325355431231111113265232113
12211226151325112212151342142121323165132221243312111112511122122
121113412421212313212131211641521516423113121231261113111222213514
412312121113161231232641112121411213216213311121121355331241122231
11111353311112261131111114235143113222121112632112221331211221213
211212132115341231122231316111222121223111212262114121421131554432
411211311112231151112412223126213111211332155443141112231111113111
11331211111251211512411311341242212323263522133122112261131211123
52355442121111122111112121211463211261312452511162112131165111126
41111311111643513211111226211211323421411222323161321412123226252
121122422155341131122212111221124321131261521211112253154412312111
111211131231211121615251223221231544114212121231151322431111322621
15211111253554411411121121161211122133121321125211112211115543213
11222323162132231115123136423113121111625222231151115544113221121
1311111132642112311312123121251222311531113412322212111213535412
121121211121152225311341241111212126312112223121211262123211412213
5533213112211111212211112243222112615212423212131534123222121112
12131352512331113121112222112225315443231112212121111332241122122
6212421132211155431141222232111351114121211115122111211515133

Figure 2.6 Stegokey

2.2.7 Approach (A Novel Approach of Bulk Data Hiding using Text Steganography)

Shivani et al. (2015) [23] proposed a novel approach for hiding bulk text data to ensure high hiding capacity without causing any visual or structural distortion to the cover text, utilizing the Zero Distortion Technique. The algorithm begins by compressing the secret text using the abbreviation method to reduce its actual size. Next, the system converts the texts into binary format, matches the bits of the reduced secret text with the cover text bits, and saves the "matched locations" in a specific matrix (Matrix of locations) rather than modifying the cover text itself. To secure this matrix, it is encrypted using the Indexed Based Chaotic Sequence algorithm to generate an encrypted sequence that is sent to the recipient.

The results demonstrated the immense superiority of this method; it successfully hid 1000 bytes entirely within a 3564-byte cover text. By contrast, other contemporary structural embedding methods (such as those based on curves or spaces) could only hide between 76 and 232 bytes within the same cover text. However, despite this high level of capacity and security that leaves the original text unaltered, the algorithm suffers from a practical limitation: it relies entirely on the probability of bit matching, which requires the cover text to be significantly large to accommodate the generation of the location's matrix; otherwise, the secret data will not be fully embedded.

Chaotic_sequence <900x8 double>								
	1	2	3	4	5	6	7	8
1	3269	1098	494	1567	3290	1117	251	1499
2	1507	2300	1463	2011	1524	2320	2572	1967
3	838	358	866	2168	855	366	229	2130
4	1381	988	3410	459	1396	998	300	417
5	3223	2834	3169	1582	3245	2851	2822	1540
6	657	3554	413	1299	669	10	2847	1251
7	3550	2856	2308	2298	5	2868	2618	2250
8	3380	1728	2910	1232	3398	1745	2844	1174
9	2234	3290	1757	1481	2247	3307	580	1438
10	1410	654	2579	2417	1427	670	1421	2376
11	1874	2532	3308	1416	1896	2545	2610	1372
12	1508	1969	3219	731	1525	1986	306	673
13	2386	564	1066	2024	2403	582	879	1977
14	1383	3235	2118	3323	1404	3253	2110	3277
15	3329	1162	2982	3032	3343	1185	1660	2983
16	68	1319	3562	2552	81	1341	303	2492

Figure 2.7 Chaotic sequence matrix stored values

2.2.8 Approach (ASCII Art Steganography)

Vincent Chu [24] (2009) introduced a tool known as ASCII Art Steganography, which hides information within images and then converts the image into a visual textual representation using image-to-text conversion algorithms, resulting in output that appears as ASCII Art.

The tool relies on representing a set of characters and symbols that simulate the grayscale gradients of the original image. This approach is distinguished by embedding data within a text-based medium rather than commonly used media such as images or audio files

The results demonstrated the tool's ability to accommodate a relatively large amount of hidden data, with the embedded message size reaching approximately 40 kilobytes, while maintaining the clarity of the textual image representation. However, the study also highlighted several limitations, including the large size of the generated text, which restricts its circulation on modern communication platforms with strict message size limits. Moreover, the unusual appearance of the generated text may raise suspicion among observers. Additionally, the hidden message may be affected by copying, deletion, or modification operations, potentially leading to data loss or distortion.



Figure 2.8 ASCII Art Steganography

2.2.9 Approach (A New 3-Bit Hiding Covert Channel Algorithm for Public Data and Medical Data Security Using Format Based Text Steganography)

Gurunath and Samanta (2023) [25] propose a data hiding algorithm in environments that support formatting features such as MS Word. The mechanism relies on dividing the secret text into a set of blocks. Each block consists of 3 bits. The blocks are merged into the cover text using eight rules or (a pre-coding table). Each rule is assigned to a specific pattern of bits (from 000 to 111), and accordingly, slight structural modifications are introduced to the text, including changing spaces, reducing font size to 0.5, or adjusting the word position vertically. To avoid automated detection, the authors incorporated a random function that selects a random position within each block composed of four words to apply the modification, which prevents the appearance of any repetitive pattern. To increase security, the message is first encrypted using (AES) Advanced Encryption Standard before the hiding process.

However, there are some drawbacks. First, the system relies heavily on specific file formats such as .doc and .html. Since the hidden bits are concealed within metadata and formatting, the process of copying to plain text will erase the secret text. Second, excessive insertion of spaces and punctuation may be detected by sophisticated steganalysis programs if used excessively.

Original Cover text (Twitter)
Located about 10 minutes from Kundasang town, Sosodikon Hill offers an easy hike suitable for beginners. Make sure to go before 7AM so you can enjoy a perfect view of the majestic Akinabalu! Enjoy the scenery with the cool weather, misty air, and the perfect view of Mount Kinabalu from Sosodikon Hill, Kundasang. One of the most beautiful highlands in Malaysia. It's a must-visit when in Sabah.

Stego-Text with Embedded Message
Located about 10 minutes from Kundasang town, Sosodikon Hill offers an easy hike suitable for beginners. Make sure to go before 7AM so you can enjoy a perfect view of the majestic Akinabalu! Enjoy the scenery with the cool weather, misty air, and the perfect view of Mount Kinabalu from Sosodikon Hill, Kundasang. One of the most beautiful highlands in Malaysia. It's a must-visit when in Sabah.

Secret word: hidden (6 chars)

Binary values : (no prefix 0)

011 010 000 110 100 101 100 100 011 001 000 110 010 101 101 110 (6X8=48bits ; 48/3=16Blocks, 4words /iteration)

Figure 2.9 Illustrated example of proposed Algorithm using twitter data

3-bit Pattern	Structural Modification
000	Add 3 spaces.
001	Add 1 space.
010	Add 2 spaces.
011	Shrink font size by 0.5.
100	Shift word up.
101	Shift word down.
110	Add an apostrophe.
111	Add a hyphen.

Table 2.5 3-Bit Embedding Rules

2.2.10 Approach (Text Steganography Using Character Spacing after Normalization)

Syed Tahir Ali Shah et al. (2020) [26] produced a new way of text steganography for embedding and hiding secret text into cover medium, the method used frequency modulation techniques, that highly improved the capacity of the cover by using a frequency normalization system combined with character mapping, effectively solving the problem of non-distribution of English alphabets by creating 26 unique strings of 8 characters each with nearly constant cumulative frequency values. This approach allows the secret data to be embedded through the manipulation of font attributes, specifically by changing the space that comes after the letter by increasing or decreasing by values of 0.1, 0.2, 0.3, or 0.4 points depending on the position of the cover character that is mapped to the secret character. By using these eight possible ways for each character, the researchers succeed in decreasing the overhead required for data hiding, requiring an average of only three cover characters to hide one secret character compared to four characters required in other models. Furthermore, the method remains case-insensitive and ensures that the cumulative probability of character occurrence is close to uniform, which maximizes the hiding capacity and maintains high perceptual transparency. However, the study identifies a critical vulnerability regarding robustness, noticing that the hidden message will be destroyed if any modifications are made to the cover document, whether intentionally by an attacker or accidentally through re-formatting. the current model is limited to the English alphabet and does not support case-sensitive, special characters, or numerical data, and is primarily optimized for digital formats as these infinitesimal spacing adjustments are unlikely to survive the printing process.

Mapped character		Cumulative Frequency	Mapped character		Cumulative Frequency
a	o b w z r c h s	31.63176	n	s t f n z p j r	30.3538
b	q s o a x d v f	31.33373	o	j f k o h r n g	31.05369
c	n u g j b e l c	31.03377	p	l z x p n s d a	31.51365
d	t w d c k g o q	30.77392	q	k y l w s t f m	30.79384
e	z o q d v w e x	29.44015	r	u x y k i h q t	29.43402
f	c g i e j n k z	31.07361	s	e i j r m u x b	31.57354
g	a j z f l i u n	31.01373	t	g e b s c v y w	29.94009
h	i k s g o j m u	31.03365	u	v q m t y f z e	29.67409
i	y a t u w k b p	30.39403	v	f p a v t x g h	30.99381
j	x m h i p l t j	31.03173	w	w r c h a y p l	30.914
k	d l r b f m w i	29.51421	x	d b h p q e z r d	31.21382
l	m n v l d o c y	30.27412	y	p v u x g a r i o	30.99381
m	h d n m u q s k	31.07374	z	r c e y q b a v	31.91371

Table 2.6 Cumulative Frequency

Possible character hidden in stego character "t"			
character spacing	Position based on character spacing	String in CSM that contains "t" at special point	Actual embedded character
increase by 0.1 pt	0	t w d c k g o q	d
increase by 0.2 pt	1	s t f n z p j r	o
increase by 0.3 pt	2	y a t u w k b p	i
increase by 0.4 pt	3	v q m t y f z e	u
decrease by 0.1 pt	4	f p a v t x g h	v
decrease by 0.2 pt	5	k y l w s t f m	r
decrease by 0.3 pt	6	x m h l p l t j	j
decrease by 0.4 pt	7	u x y k l h q t	s

Table 2.7 Character spacing

2.2.11 Approach (A new method for pdf steganography in justified texts)

Khosravi et al. and others (2019) [27] presented a text steganography technique that exploits the "Justification" process in Pop Directional Formatting (PDF) documents. The mechanism relied on the Huffman algorithm for message compression, followed by searching for "host lines" containing a minimum of nine regular spaces and three additional spaces. The hiding process is carried out by manipulating the order of the first nine spaces in the line, so that each line accommodates 4 bits (representing values from 0 to 15); to embed the value (0), the nine spaces are left regular; to include values (1 to 9), one additional space is inserted in the position corresponding to the number; as for the values (10 to 15), two additional spaces are inserted so that the number of regular spaces between them equals the ones digit of the number. This method is characterized by maintaining the original file size and its printability, as it allows data extraction from printed papers via Optical Character Recognition (OCR) technology without the need for the electronic file, while enhancing security using an encryption key. In contrast, the study faces fundamental limitations; most notably the close correlation with OCR accuracy in distinguishing between spaces, where any error leads to the corruption of the message entirely, and the algorithm also requires texts with high space density and suffers from extreme fragility toward manual modifications. Furthermore, its storage capacity (Capacity ratio) is considered very low (1/246), which requires a massive cover text compared to the size of the hidden data.

<i>i</i>	Embedding of <i>i</i> in the HL
0	Last night of all, when yond same star that's westward from the pole had made his course to illume
<i>i</i>	Embedding of <i>i</i> in the HL
1	Last night of all, when yond same star that's westward from the pole had made his course to illume
2	Last night of all, when yond same star that's westward from the pole had made his course to illume
3	Last night of all, when yond same star that's westward from the pole had made his course to illume
4	Last night of all, when yond same star that's westward from the pole had made his course to illume
5	Last night of all, when yond same star that's westward from the pole had made his course to illume
6	Last night of all, when yond same star that's westward from the pole had made his course to illume
7	Last night of all, when yond same star that's westward from the pole had made his course to illume
8	Last night of all, when yond same star that's westward from the pole had made his course to illume
9	Last night of all, when yond same star that's westward from the pole had made his course to illume
<i>i</i>	Embedding of <i>i</i> in the HL
10	Last night of all, when yond same star that's westward from the pole had made his course to illume
11	Last night of all, when yond same star that's westward from the pole had made his course to illume
12	Last night of all, when yond same star that's westward from the pole had made his course to illume
13	Last night of all, when yond same star that's westward from the pole had made his course to illume
14	Last night of all, when yond same star that's westward from the pole had made his course to illume
15	Last night of all, when yond same star that's westward from the pole had made his course to illume

Figure 2.10 Embedding of values 0 to 15 in a Host Line

2.2.12 Approach (A high-capacity text steganography scheme based on LZW compression and color coding)

Malik et al. (2016) [28] proposed a text steganography model based on color coding using the Lempel–Ziv–Welch (LZW) compression algorithm to increase embedding capacity. The method utilizes email forwarding as a cover medium to hide secret data; the algorithm first compresses the message, then embeds it within the email header fields and the message body. Data bits are represented by coloring the text based on a specific color-coding table. Experimental results showed that the proposed technique achieves higher embedding capacity compared to several previous methods, along with lower computational complexity. Additionally, the security level was enhanced using stego keys.

Despite these advantages, the scope of this method's application remains restricted to the email platform only, as it relies primarily on rich formatting features such as colors and the structure of email headers, making it unsuitable for exchange via other messaging applications.

Table 5
Color mapping for five bit code representation.

S. No.	Boundary line colors		Fill area Color		Binary code	S. No.	Boundary line Color		Fill area Color		Binary code
	Name	Color	Name	Color			Name	Color	Name	Color	
1	Rackley		Violet		00000	17	Green		Blue		10000
2	Rackley		Blue		00001	18	Green		Rackley		10001
3	Rackley		Rose		00010	19	Green		Rose		10010
4	Rackley		Black		00011	20	Green		Black		10011
5	Rackley		Scarlet		00100	21	Black		Violet		10100
6	Violet		Blue		00101	22	Black		Blue		10101
7	Violet		Rackley		00110	23	Black		Rackley		10110
8	Violet		Rose		00111	24	Black		Rose		10111
9	Violet		Black		01000	25	Black		Scarlet		11000
10	Violet		Scarlet		01001	26	Blue		Violet		11001
11	Rose		Violet		01010	27	Blue		Rackley		11010
12	Rose		Blue		01011	28	Blue		Rose		11011
13	Rose		Rackley		01100	29	Blue		Black		11100
14	Rose		Black		01101	30	Blue		Scarlet		11101
15	Rose		Scarlet		01110	31	Scarlet		Violet		11110
16	Green		Violet		01111	32	Scarlet		Blue		11111

Figure 2.11 Color Mapping

0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0	“Only boats catch connotes of the islands sober wines Only ships wrap the slips on the cleats of twining lines Only flags flap in tags with color that assigns Only passage on vessels”
---	---

Figure 2.12 Bit Stream Partitioning

Figure 2.13 Text Color Encoding Process

2.2.13 Approach (A Space based reversible high-capacity text steganography scheme using Font type and style)

Kumar et al. (2016) [28] proposed a mechanism designed to embed secret data in the spaces between words within Microsoft Word documents by manipulating the font type and style of the space. The method relies on mapping 6 bits to a specific font from a predefined table comprising 64 fonts, with the use of the seventh bit to switch to the Italic or regular font style. The researcher also mentioned adding AES encryption to increase security, and thereby it achieves an excellent embedding capacity (7 bits per space) and high security, while maintaining visual Imperceptibility. However, and despite these advantages, the researchers did not add any compression algorithm. The method also suffers from limitations, most notably its strict reliance on the text environment in Microsoft Word; as the hidden data relies entirely on Formatting Metadata, which is lost when executing "copy and paste" operations to plain text editors, web browsers, or social media platforms. Furthermore, the authors acknowledged that despite the visual security of the method, it remains vulnerable to statistical analysis, where the abnormal distribution of font types within the white spaces can be detected by examining the file structure.

ID	Font Type	ID	Font Type	ID	Font Type	ID	Font Type
0	Agency FB	16	DilleniaUPC	32	Kalinga	48	PMingLiU-ExtB
1	Aharoni	17	DokChampa	33	KodchiangUPC	49	Poor Richard
2	Aparajita	18	Dotum	34	Kokila	50	Rockwell
3	Arabic Typesetting	19	Ebrima	35	LilyUPC	51	Sakkal Majalla
4	Arial	20	Eras Medium ITC	36	Lucida Handwriting	52	Segoe UI
5	Arial Narrow	21	Estrangelo Edessa	37	Marigold	53	Segoe UI Light
6	Arial Unicode MS	22	Footlight MT Light	38	Microsoft New Tai Lue	54	Segoe UI Semilight
7	Batang	23	Franklin Gothic Book	39	Microsoft PhagsPa	55	Segoe UI Symbol
8	Bell MT	24	FreesiaUPC	40	Microsoft Sans Serif	56	Simplified Arabic
9	Bodoni MT Condensed	25	Gabriola	41	MS Reference Sans Serif	57	Sylfaen
10	Bradley Hand ITC	26	Gisha	42	MS UI Gothic	58	Times New Roman
11	Calibri	27	Gulim	43	Nirmala UI	59	Traditional Arabic
12	Calisto MT	28	IrisUPC	44	Palatino Linotype	60	Tw Cen MT Condensed
13	Centaur	29	Iskoola Pota	45	Perpetua	61	Univers
14	Copperplate Gothic Light	30	JasmineUPC	46	Perpetua Titling MT	62	Univers Condensed
15	David	31	Juice ITC	47	PMingLiU	63	Utsaah

Table 2.8 Approaches (A Space based reversible high-capacity text steganography scheme using Font type and style)

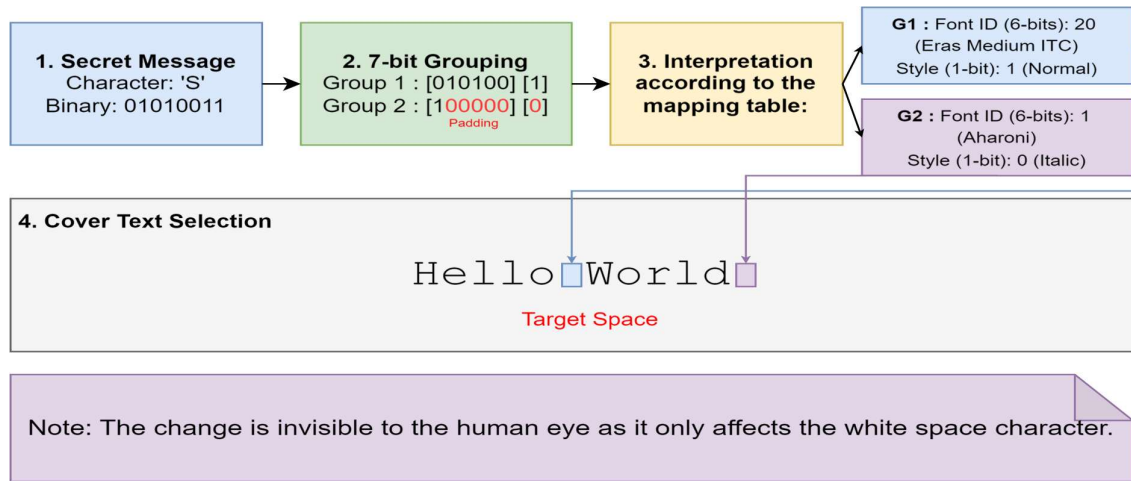


Figure 2.14 A Space based reversible high-capacity text steganography scheme using Font type and style

2.2.14 Approach (A Novel Approach to Format Based Text Steganography)

Sangita Roy and Manini Manasmita (2011) [29] proposed a hybrid approach to format-based text steganography. Their method combines line shifting, word shifting, and the use of special characters to conceal binary data within a cover text, aiming to achieve high embedding capacity. The hiding process begins by converting the secret text into its (ASCII) American Standard Code for Information Interchange values and then into a binary bit stream. This bit stream is subsequently divided into blocks of two bits each to prepare for the encoding phase.

The encoding method embeds the 2-bit blocks by altering the formatting of the cover text based on four specific conditions. A '00' block decreases the font size to shrink the line, while a '11' block increases the font size to expand the line. For a '01' block, extra space is inserted between two words, and for a '10' block, extra space is added immediately before a special character. The extraction process involves scanning the generated stego text using Optical Character Recognition (OCR) software. The hidden bits are then reconstructed by detecting the presence of double spaces, spaces preceding special characters, and variations in line sizes compared to a standard baseline.

While this hybrid model successfully increases the hiding capacity by encoding multiple bits per line without making visually apparent changes, it introduces significant limitations. The reliance on extra spaces makes the hidden information highly fragile, as common word processing software may automatically delete these spaces and destroy the embedded data. Furthermore, the method depends heavily on physical formatting for extraction and exhibits a higher computational complexity of $O(n^2)$ compared to existing $O(n)$ methods. These vulnerabilities highlight a research gap for developing text steganography techniques that minimize computational complexity and are more resilient to routine text-formatting operations.

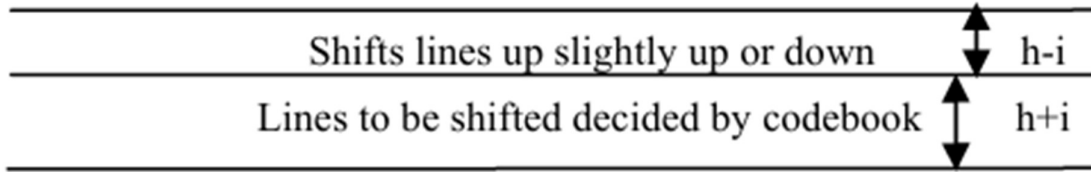


Figure 2.15 example of line shifting method

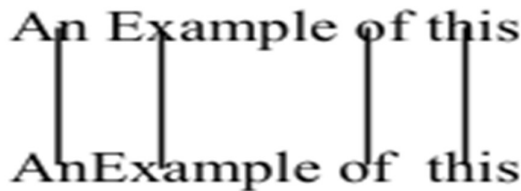


Figure 2.16 example of word shifting method

2.2.15 Approach (Emoticon-based Steganography for Securing Sensitive Data)

Ahmad et al. (2014) [30] introduced an emoticon-based text steganography method to secure sensitive data during data transmissions. The method used the Lempel-Ziv-Welch (LZW) algorithm, not to compress secret texts, but to work as embedding engine to map secret data bits into emoticons. The process included converting the secret text into 7-bit ASCII codes and using a shared emoticon dictionary as a stego key having at least 256 icons. The dictionary will be Initialized by each single symbol in the secret text.

Emoticon	Bit	Character
😄	1100001	'a'
😬	1100010	'b'
😬	1101110	'n'
😬	0100000	'_'
😬		<empty>
😬		<empty>
😬		<empty>
😬		<empty>
😬		<empty>
😬		<empty>
😬		<empty>
😬		<empty>
😬		<empty>

Table 2.9 INITIALIZED DICTIONARY

when embedding the secret data, substrings of the secret data are matched against the dictionary, if a match is found, it is extended by adding new char to the substring, and if not, a new entry containing the extended substring is created and the corresponding emoticon is added to the stego list to be embedded within a cover text.

Emoticon	Bit	Character
😊	1100001	‘a’
😄	1100010	‘b’
😆	1101110	‘n’
😏	0100000	‘_’
😮	11000101100001	‘ba’
😲	11000011101110	‘an’
😬	11011101100001	‘na’
😏	110000111011101100001	‘ana’
😏	11000010100000	‘a_’
😏	01000001100010	‘b_’
😏	110001011000011101110	‘ban’
😏	110111011000011101110	‘nan’

Table 2.10 GENERATED DICTIONARY

Its big advantage is the dynamic capacity, where each emoticon can represent a multiple number of bits, enhancing security through diffusion and making it more difficult for attackers to predict the hidden content. its disadvantage, the method offers good performance with high frequency text, but in low repetition texts, this will be negative as it requires frequent use of emoticon, which will make it difficult to create text that contains that large amount of emoticon, And the method requires the receiver to know the sequence of the emoticon in the initialized table ensure successful extraction.

2.2.16 Approach (Hiding Information in Email Texts Using Repeated Characters)

Fatih and Rezwani (2018) [31] proposed a novel method for hiding data in email-texts steganography, designed to achieve high data capacity and strong security without adding redundant or additional information or distorting the original text.

It starts to compress the secret text into multiple bit strings using the lossless LZW compression algorithm. To embed this compressed data, the system mathematically analyzes the structural elements of the email text, specifically the number of text repetitions, the number of sentences, and the remaining characters. These calculated parameters are after that linked to specific English letters and standard prefixes (such as @hotmail.com or @gmail.com) to generate meaningful email addresses for the recipients. By distributing the entire secret data across the generated email addresses in the recipient list, the original message text remains unchanged, allowing any plain text to be used as a cover.

The results showed that this method achieved a hiding power of 10.6 on plain text samples and was around 47% higher than other contemporary methods. Because the encrypted text is sent simultaneously to the generated addresses and the intended recipient without any structural or visual modifications, this method offers a high degree of message concealment. However, despite its effectiveness, the researchers pointed out a practical limitation: the algorithm still requires generating many email addresses to carry the data, which is a security vulnerability that necessitates future improvements to avoid arousing the suspicion of network monitors.

```
1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0,
0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1,
0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0
```

Figure 2.17 simple secret message



Figure 2.18The bit-stream obtained from xyz.

2.3 Comparative analysis

Author	Study Name	Year	Description of the Approach	Problem or Research Gap	Advantage
Paul Butler	Emoji Encoder	2025	Embedding bytes by appending Variation Selectors hidden behind Emojis.	Weak Security due to the lack of encryption and ease of Steganalysis, total lack of compression, with excellent Robustness, and the Cover Media may suffer Distortion across different platforms.	Excellent Embedding Capacity, and excellent Robustness that fully withstands copy-and-paste operations.
Priyansh Sharma	Zero Width Characters (ZWC)	2023	Encrypting the Payload with (XOR), then dividing it into (2-bits) blocks and embedding them as hidden characters (ZWC) between the words of the Cover-text.	Weak Security due to the use of weak encryption and ease of Steganalysis, total lack of compression, and good Imperceptibility due to the Cover Media being affected in some languages, with weak Robustness due to being affected by different platforms.	Total Imperceptibility maintains the original appearance of the Cover Media without any visual trace.
Majid et al.	New Text Steganography Technique based on Multilayer Encoding with Format-Preserving Encryption and Huffman Coding	2022	Encrypting the Payload with (FPE), compressing it with (Huffman), then dividing it into (3-bits) blocks and embedding them as hidden (ZWC) characters after each word.	Ease of detection by Steganalysis, excellent Imperceptibility, with weak Robustness due to being affected by different platforms.	Good Security due to using (FPE) encryption, good Embedding Capacity with (Huffman) compression, total Imperceptibility (0.984 J-W), and excellent Security with (FPE) encryption that does not increase the size.
Ahvanooey et al.	AITSteg: An Innovative Text Steganography Technique for Hidden Transmission of Text Message via social media	2018	Encoding the Secret text using the Gödel function, time-based encryption, then dividing it into (2-bits) blocks and embedding them as (ZWC) characters at the beginning of the Cover-text.	Ease of detection by Steganalysis, total lack of compression, and good Imperceptibility due to the Cover Media being affected in some languages, with weak Robustness due to being affected by different platforms.	Good Security due to using symmetric encryption (Security), and total Imperceptibility (Imperceptibility).
Matthew Kwan	SNOW Tool	2013	Compressing the Payload with (Huffman) and encrypting it with (ICE), then dividing it into (3-bits) blocks and embedding them as	Ease of detection by Steganalysis, and very weak Robustness as most platforms delete extra spaces, and weak Imperceptibility due to the possibility of noticing the extra spaces.	Embedding Capacity supported by (Huffman) compression, and total Imperceptibility of the visual content, with good Security thanks to encryption.

			spaces and Tabs at the end of the lines.		
Naharudin et al.	A High Capacity and Imperceptible Text Steganography Using Binary Digit Mapping on ASCII Characters	2018	Encrypting the Payload with (OTP), then embedding it by matching its bits with the bits of the Cover-text and saving the matching locations in a Stego-key.	Weak Robustness against any modification in the Cover Media, weak Embedding Capacity require a huge Cover Media (storing one character for every seven characters) and lack of compression, with the bloat of the key size.	Total Imperceptibility due to the Cover Media being unaffected.
Shivani et al	A Novel Approach of Bulk Data Hiding using Text Steganography	2015	Compressing the message using the abbreviation method then encrypting it with the Chaotic Sequence algorithm and embedding it by matching its bits with the bits of the cover text and saving the matching locations in a stego-key.	Weak robustness against any modification in the cover, and the use of a weak compression technique due to its reliance on word abbreviations, along with an inflation in the key size.	Complete visual Imperceptibility due to the cover being unaffected, in addition to excellent capacity.
Vincent Chu	ASCII Art Steganography	2009	Hiding data first inside an image, then converting the entire image into visual text art (ASCII Art).	Weak Security due to the lack of encryption, weak Imperceptibility due to the lack of a natural appearance, and weak Robustness due to platforms blocking the huge sizes resulting from the lack of compression.	Excellent Embedding Capacity, providing a huge Embedding space (~40 KB).
Gurunath & Samanta	A New 3-Bit Hiding Covert Channel Algorithm for Public Data and Medical Data Security Using Format Based Text Steganography	2023	Encrypting the Payload with (AES), then dividing it into (3-bits) blocks and embedding them as random structural modifications (spaces/font size) in Word and HTML files.	Weak Embedding Capacity due to the lack of compression, and weak Robustness to data loss during copying due to its reliance on specific platforms, and weak Imperceptibility for leaving a repetitive visual trace in the Cover Media.	Good Security thanks to strong (AES) encryption, and resistance to automated detection thanks to random Embedding.
Syed Tahir et al.	Text Steganography Using Character Spacing after Normalization	2020	Direct Embedding of data by altering the horizontal spaces between characters	Weak Robustness due to its fragility against modification, weak Embedding Capacity due to the lack of compression, and total	Good Imperceptibility as the Cover Media is affected in a way that is unnoticeable to the eye.

			imperceptibly inside Word files.	limitation of the Cover Media by restricting it to the English language and the (Word) environment.	
Khosravi et al.	A new method for pdf steganography in justified texts	2019	Starts with compressing the Payload with Huffman and encrypting it with Permutation Stego-key, then dividing it into (4-bits) blocks and embedding them as additional spaces in the lines of PDF files.	Weak Robustness due to its reliance on (OCR) Extraction, and very weak Embedding Capacity requiring huge texts full of spaces.	Excellent Security thanks to encryption, preservation of the Cover Media size, with the possibility of effective Extraction via (OCR).
Malik et al.	A high-capacity text steganography scheme based on LZW compression and color coding)	2016	Compressing the Payload with (LZW), then embedding it by coloring the texts of email messages based on a predefined color-coding table.	Weak Security due to the lack of encryption, and total limitation of the Cover Media due to its complete reliance on the email environment without supporting plain texts.	Good Embedding Capacity supported by (LZW) compression, low computational complexity. The idea can be applied on other platforms.
Kumar et al.	A Space based reversible high-capacity text steganography scheme using Font type and style	2016	Encrypting the Payload with (AES), then dividing it into (7-bits) blocks and embedding them by modifying the font type and style of spaces according to a table of 64 fonts.	Ease of detection by Steganalysis, lack of compression, weak Robustness to data loss during copying, and limitation to (Word).	Good Security due to using strong encryption, excellent Embedding Capacity (7 bits per space), and total Imperceptibility.
Sangita Roy & Manini	A Novel Approach to Format Based Text Steganography	2011	Dividing the Payload into (2-bits) blocks, then embedding them by making modifications to the formatting of the Cover-text.	Weak Security due to the lack of encryption, weak Embedding Capacity, lack of compression, and weak Robustness to data loss during copying, restricted to environments that support text formatting.	The Payload can be extracted using OCR.
Ahmad et al.	Emoticon-based Steganography for Securing Sensitive Data	2014	Using (LZW) to map the Secret text strings to a dictionary of Emojis.	Weak Security due to the lack of encryption, susceptibility to Steganalysis, lack of compression, and weak Imperceptibility due to the visual Distortion of the Cover	Excellent Embedding Capacity.

				Media when Embedding (capacity) increases.	
--	--	--	--	---	--

Table 2.11 Comparative analysis of Literate Studies

2.4 Identified gap

Despite the remarkable progress in the field of text steganography, most of the previous literature still suffers from shortcomings in robustness and weakness in security; as many of them lack effective encryption mechanisms, while others leave visible traces and structural distortions that facilitate the steganalysis process. Furthermore, some mechanisms are tied to a specific language or a specific platform to benefit from their formatting in hiding the message, which leads to the loss of the payload upon copying across different platforms or minor modification, along with low embedding capacity, and significant inflation in the size of the cover-text and stego-keys as a result of a weakness in the hiding mechanism or a shortcoming in compression algorithms.

In response to these challenges, this research seeks to bridge those gaps through the invention of a hybrid Steg-system that aims to achieve an effective balance between capacity, robustness, visual Imperceptibility, and security, as it seeks to achieve complete Imperceptibility and provides good robustness for data against copy and paste operations. It also addresses the issues of security and capacity by integrating strong compression and encryption algorithms that prevent the inflation of the cover and protect the message content.

3. Methodology

3.1 Research Design

This research adopts the Design Science Research Paradigm, as it focuses on the engineering and innovation of a hybrid algorithmic technique for text steganography. This methodology is practically translated through the construction of an integrated steganography environment (Stego-system) that combines compression and encryption algorithms, and this system is programmatically represented in an interactive client-side web platform functioning as a demonstration model, enabling users to conceal and retrieve data effectively and independently.

To verify the effectiveness of the proposed artifact, the research applies a Mixed-Methods Evaluation strategy that combines both quantitative and qualitative approaches. Quantitatively, the embedding capacity is measured numerically to demonstrate the ability of the proposed mechanism to accommodate a higher data payload. Qualitatively, the other fundamental properties are evaluated based on scenarios and functional tests; these properties include Imperceptibility, to ensure the preservation of the perceptual transparency of the text cover without introducing visual distortions, and robustness, to ensure the resilience of the data against copying and reformatting operations, in addition to the systematic evaluation of the security level.

This methodological approach seeks to demonstrate the capability of the proposed hybrid system to address the shortcomings identified in previous studies, such as limited capacity and weak robustness. This is achieved through a systematic scientific investigation into how the new algorithmic architecture overcomes traditional limitations and provides a solution that achieves the optimal balance between storage capacity and data protection.

The flowchart shown in Figure 3.1 shows the methodology adopted in this research, divided into two main stages. Stage 1 starts with reviewing previous text steganography systems to establish gaps in research, classified under four aspects: capacity, robustness, Imperceptibility, and security. As a result, these gaps will be addressed during Stage 2, including both design and evaluation processes. The design phase starts by designing a hybrid algorithm based on the integration of two core layers, where the compression layer is designed for increasing the capacity of cover text, while the encryption layer is built to ensure data security. Then comes the development phase to build the Stego-system as a proof of concept, as well as developing the model as an interactive web-based tool. At the same time, evaluation takes place through two tracks: quantitatively by measuring embedding capacity numerically, and qualitatively by testing the Imperceptibility, robustness, and security. Both tracks converge in the systematic analysis of results, concluding with the verification of gap bridging and achieving balance between capacity and security.

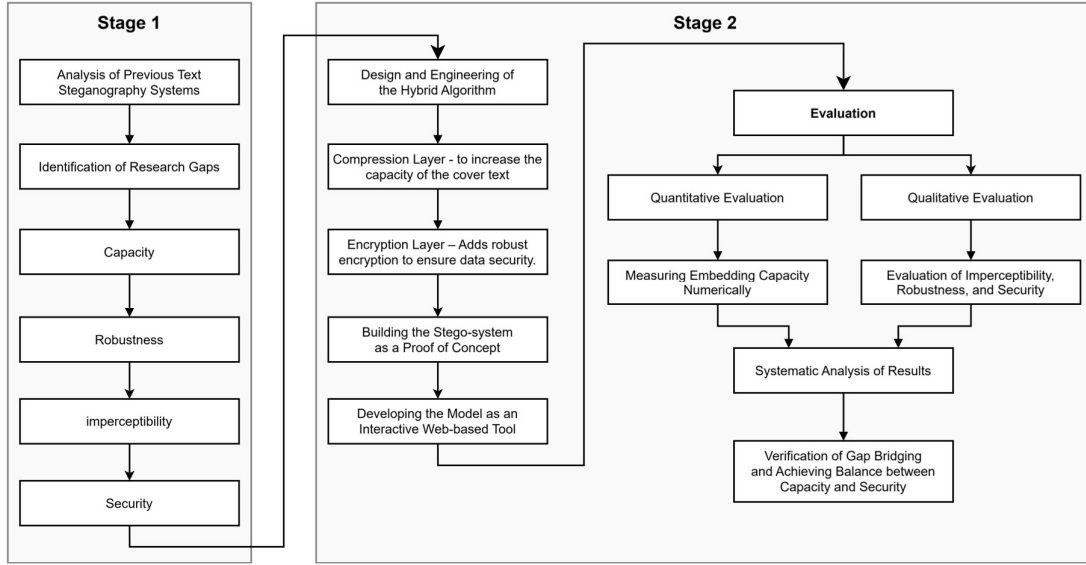


figure 3.1 Research Workflow

3.2 Environment and Tools

To implement the proposed stego-system and transform the theoretical framework into a practical application, an integrated development environment and a comprehensive set of tools and technologies were adopted to ensure the achievement of the research objectives. The system was designed as a fully independent client-side web application, enabling local data processing entirely within the user's browser without any server-side dependency, thereby preserving user privacy and data confidentiality.

3.2.1 Frontend Technologies: At the frontend level, the tool was built using standard web technologies — Hypertext Markup Language (HTML5), Cascading Style Sheets (CSS3), and JavaScript — to provide a flexible and responsive design supporting mobile devices and various screen sizes. A modular CSS architecture was adopted with a design token system for consistent theming, along with support for both dark and light display modes through a dedicated theme manager. The interface was also equipped with an input validation mechanism to ensure system stability and prevent errors during the entry of secret texts or cover texts.

3.2.2 Algorithmic Core: The core algorithmic pipeline was implemented in pure JavaScript and organized into three modular layers:

- **Compression Layer:** The Brotli compression algorithm (RFC 7932) was adopted to reduce the payload size prior to encryption. The Brotli engine, originally written in Rust, was compiled into WebAssembly (WASM) and directly embedded within the application through Base64 encoding, enabling high-performance data compression locally within the

user's browser. Additionally, an AVIF WebAssembly engine was integrated for image compression capabilities, allowing the system to handle image-based steganographic operations.

- **Cryptographic Layer:** The encryption mechanism employs the AES-256-CTR (Advanced Encryption Standard — Counter Mode) algorithm for symmetric encryption of the compressed payload. The key derivation process utilizes PBKDF2 (Password-Based Key Derivation Function 2) with HMAC-SHA-256 through 100,000 iterations to derive a 48-byte cryptographic material (256-bit AES key + 128-bit counter block) from the user's password and a deterministic 16-byte salt. This salt is dynamically generated by computing the SHA-256 hash of the cover text and extracting its first 128 bits, ensuring the cryptographic parameters are structurally bound to the carrier content.
- **Steganographic Layer:** The embedding and extraction processes are managed by a central orchestrator module (stego-composer) that coordinates the entire pipeline. This layer comprises a payload codec responsible for encoding and parsing the secret text with optional hints, an XOR masking engine that performs bit-level mapping between the encrypted payload and the cover text, a Variation Selector codec that encodes the resulting key sequence into invisible Unicode Variation Selectors (U+FE00–U+FE0F and U+E0100–U+E01EF), and a pseudorandom number generator (Mulberry32 PRNG) for distributing the embedded data across the cover text.

3.2.3 Extended Features: Beyond the core embedding and extraction functions, the tool incorporates several extended features to enhance usability and analytical capabilities. These include a forensics dashboard for stego-analysis and detection of hidden data, a chat scanner for examining conversations and identifying concealed content, a capacity calculator that estimates the available embedding capacity for a given cover text, and a hint system that allows the sender to attach auxiliary information to guide the extraction process.

3.2.4 Development and Testing Environment: Visual Studio Code was adopted as the primary Integrated Development Environment (IDE). Git and GitHub were used for version control to ensure proper documentation of the project development stages and efficient management of the source code. For quality assurance.

3.3 Procedures and Metrics

3.3.1 Procedures

The proposed system operates entirely on the client side to ensure maximum privacy and data security. The system workflow is organized into three main procedural stages:

- **System Initialization:** The procedures begin once the user accesses the web application, where the system initializes the WebAssembly (WASM) engine. To achieve high data compression efficiency, the Brotli algorithm written in Rust was adopted, translated into the WASM environment, and directly integrated into the application through Base64 encoding. This procedure enables local data compression within the user's browser with high performance and rapid responsiveness.
- **Embedding Process – Sender Side:** The data hiding process passes through a sequence of consecutive algorithmic operations:
 - **Compression:** The secret text is first compressed using the configured Brotli module to reduce the payload size and maximize embedding capacity.
 - **Key Derivation:** A deterministic 16-byte cryptographic salt is extracted from the cover text by computing its Secure Hash Algorithm 256-bit (SHA-256) hash and isolating the first 128 bits. This dynamic salt is then combined with the user's supplied password through the Password-Based Key Derivation Function 2 (PBKDF2) function, utilizing HMAC-SHA-256 through 100,000 iterations to mitigate brute-force vulnerabilities. The key stretching process deterministically derives exactly 48 bytes (384 bits) of cryptographic material, which is split to establish a static 256-bit AES key and a 128-bit initial counter block, natively satisfying the strict parameter requirements of the AES-CTR encryption standard.
 - **Encryption:** The compressed message is encrypted using the AES algorithm in Counter Mode (CTR), where the Initialization Vector (IV) is programmatically generated independently from the key derivation process.
 - **Bit-Level Mapping and Encoding:** A bit-level mapping operation is performed between the encrypted message (M) and the cover text (C) using the XOR operation to generate a key sequence (K), such that $[K = C \oplus M]$. Subsequently, the resulting sequence (K) is encoded into hidden Unicode Variation Selectors symbols. Since these symbols are classified as combining characters, they are seamlessly integrated with the visible

characters of the cover text, these hidden symbols are embedded in the beginning of the cover text resulting in the final stego-text without any visual distortions.

3.3.2 Extraction Process – Receiver Side:

- **Parsing:** When the stego-text is entered into the application, the system scans the entire text to identify and extract the hidden Variation Selectors symbols and separate them from the visible cover text.
- **Key Re-Derivation:** During the extraction phase, the system first isolates and removes the invisible Variation Selector characters from the Stego-Text to retrieve the clean, original cover text. This clean cover text is passed through the SHA-256 algorithm to recover the exact 16-byte deterministic salt. The system then applies the PBKDF2 function (HMAC-SHA-256, 100,000 iterations) using this salt along with the receiver's password to accurately regenerate the identical 48 bytes of key material, restoring both the 256-bit AES key and the 128-bit counter block required for decryption.
- **Reverse Mapping, Decryption, and Decompression:** The extracted variation symbols are converted into their binary representation corresponding to the sequence (K). The system performs a reverse mapping operation $[M = C \oplus K]$ to recover the encrypted message (M). Finally, the recovered message is decrypted through AES-CTR and passed to the Brotli engine for decompression to reveal the original secret text.

The flowchart in Figure (3.2) illustrates the structural sequence of the algorithmic operations through which the system operates. It is designed to be divided into two main paths reflecting the data lifecycle: the sender path (embedding process) and the receiver path (extraction process).

The sender path begins with the input of the secret text and the cover text, after which the message passes through a sequence of processing mechanisms including compression using Brotli, followed by the programmatic derivation of the encryption parameters, utilizing a 16-byte salt extracted from the first 16 bytes of the cover text's SHA-256 hash, and processing the password through the PBKDF2 function, followed by encryption using AES-CTR. Subsequently, an XOR mapping operation is performed between the encrypted bits and the cover text, after which the resulting sequence is transformed into hidden Variation Selectors symbols that are embedded in the beginning of the cover text to produce the stego-text. The transmission of this stego-text through external communication platforms (such as WhatsApp and Instagram) is represented by a dashed arrow, indicating the transfer of data through an untrusted external network.

On the other side, the receiver path begins with receiving the stego-text and analyzing it to isolate the hidden symbols from the visible cover text. The encryption key is then re-derived using the same mechanism to ensure consistency, after which the reverse XOR, operation is executed to recover the encrypted sequence. The process concludes with decryption followed by decompression to successfully extract the original secret text.

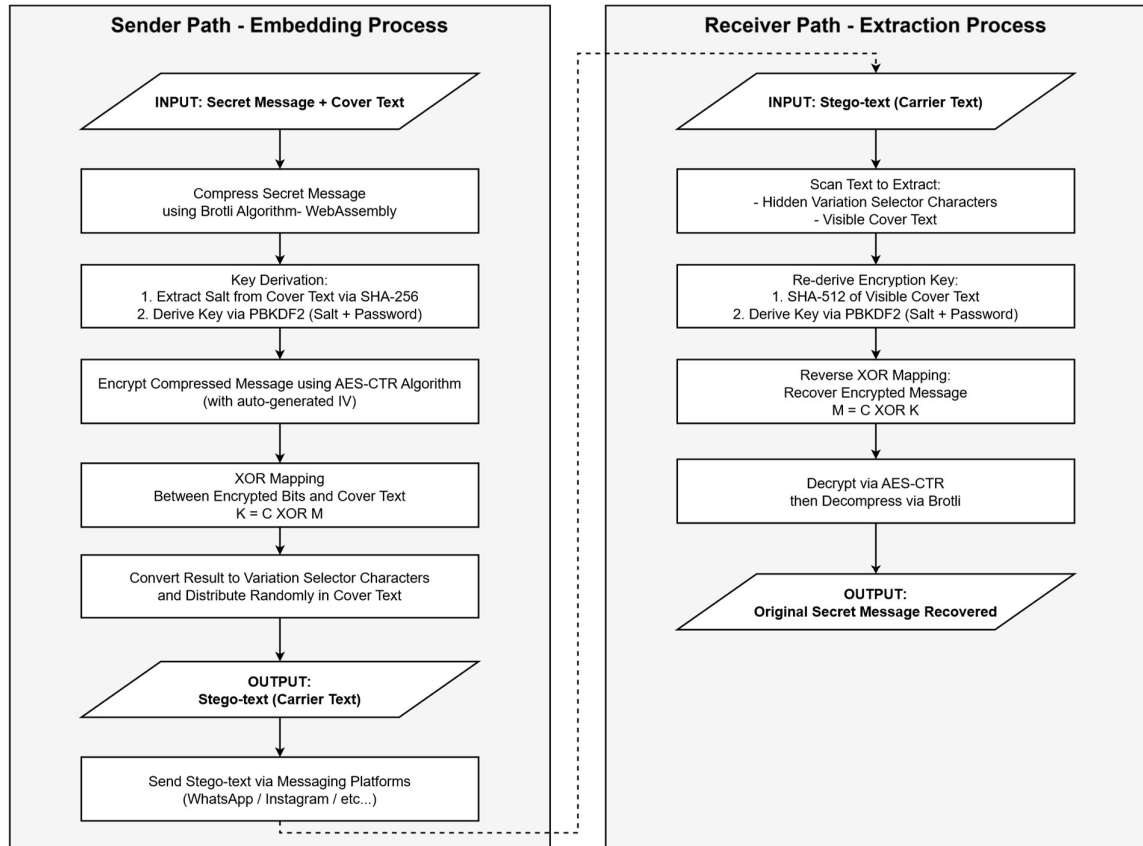


figure 3.2 procedures

3.3.3 Evaluation Metrics

To evaluate the efficiency and reliability of the proposed system, the following metrics were adopted to verify the performance of the hybrid algorithm:

- **Payload Capacity:** Measured as the ratio between the size of the secret data and the size of the cover text (in bytes). The system aims to achieve a high hiding capacity reaching a ratio of (1:1), meaning that a secret text equal in size to the cover text can be hidden without affecting its visual appearance.

- **Imperceptibility:** Evaluated through comparative visual analysis, where screenshots of the cover text before and after the embedding process are compared to ensure that the insertion of Variation Selectors symbols does not introduce any anomaly, distortion, or visible alteration to the text structure.
- **Robustness:** Tested by transmitting the stego-text through various social media platforms and messaging applications (such as WhatsApp and Telegram). This test aims to ensure the resilience of the hidden payload against normalization processes as well as the copy-and-paste operations performed by those platforms.
- **Performance Overhead:** Measured by calculating the execution latency of both the embedding and extraction phases independently to ensure rapid system responsiveness and its suitability for effective practical deployment within a web environment.

3.4 Standards applied

To ensure the security, interoperability, and high performance of the algorithms we employed, the proposed approach strictly compiles with several recognized industry standards and protocols

3.4.1 Encoding Standards:

- **Unicode Standard:** Designed for character representation, specifically utilizing Unicode Variation Selectors to seamlessly embed the binary payload within the text without visual detection.

3.4.2 Web and Execution Standards:

- **World Wide Web Consortium (W3C) Web Standards:** The frontend interface was developed using HTML5, CSS3, and JavaScript.
- **WebAssembly (WASM):** Employed as a portable compilation target for the Rust codebase. It is a W3C standard that allows high-performance applications, such as Brotli, to run securely within the client's web browser.
- **Cryptographic Standards:** *(based on NIST - National Institute of Standards and Technology guidelines)*
- **AES-CTR (Advanced Encryption Standard - Counter Mode):** A widely used symmetric-key encryption standard employed as a stream cipher. It provides multiple key length options, such as 128, 192, and 256 bits, to offer various levels of security.

- **SHA-256 (Secure Hash Algorithm 256-bit):** A cryptographic hash function designed by the National Security Agency (NSA) and published by NIST under the FIPS PUB 180-4 standard. It computes a fixed 256-bit 32-byte one-way conditional fingerprint from any variable-length input text. In this architecture, it is utilized as a deterministic extraction mechanism to generate a dynamic cryptographic salt from the visible cover text, ensuring that the randomness of the key derivation process is structurally bound to the carrier content without relying on server-side state generation.
- **PBKDF2 (Password-Based Key Derivation Function 2):** A cryptographically secure key stretching standard used to derive high-entropy cryptographic keys from user-supplied passwords. Rather than acting as a cipher itself, it functions as a security barrier against brute-force and dictionary attacks by executing 100,000 hashing iterations via HMAC-SHA-256. This process transforms a variable-length password and the cover text's generated salt into a fixed-length cryptographic material required to prime the AES-CTR stream cipher.

3.4.3 Data Compression Standards:

- **Brotli Compression (RFC 7932):** An open-source data compression algorithm that relies on an improved version of the LZ77 algorithm and Huffman coding. It is used to compress the text prior to initiating the encryption process.

4. System Design & Implementation

4.1 Architecture

The steganography system we are making is designed to run on web browsers with no need for external servers. Ensuring maximum user data confidentiality, and privacy. The full system pipeline process includes data compression using WASM Brotli algorithm, cryptographic key diversification using PBKFD2, Encryption and Decryption using AES-CTR, Embedding, and Extraction, they are all executed entirely within the user's local browser, by loading and initializing the WASM module for Brotli, with using Web Crypto API to execute AES-CTR. The architectural decision tries to eliminate the need for a backend server, and so reduce risks associated with data interception, server-side data breaches, or logs exposure.

The structure of this system contains four-tier sequential layers, works in sequence to process user data. The first one is the User Interface (UI) Layer, which is implemented using HTML5 and CSS3 to offer a responsive, interactive dashboard for the sender in the embedding phase and for the receiver in the extraction phase. In addition to handling user input data, and exceptions. The second one, powered by WebAssembly. By initializing the WASM module, that is specifically tasked with the Brotli, to produce stream compression and decompression. The third one is the Encryption Decryption layer, using native Web Crypto API for cryptographic operations. With the help of the PBKFD2 Key Diver, to generate the Crypto Key. The fourth and last layer is Embedding, and Extraction. This layer is responsible for the logic of converting the payloads into / from Variation Selectors using a customized bit-mapping technique, then embedding / extracting them within / out of the cover text.

4.1.1 System Diagrams

To present a clear picture of how the full system pipeline works, the user journey when using the system, and the system architecture, the system visualized into three diagrams:

- **Logical diagrams:**

Data Flow Diagrams:

Data Flow Diagram for the Embedding Process (Sender)

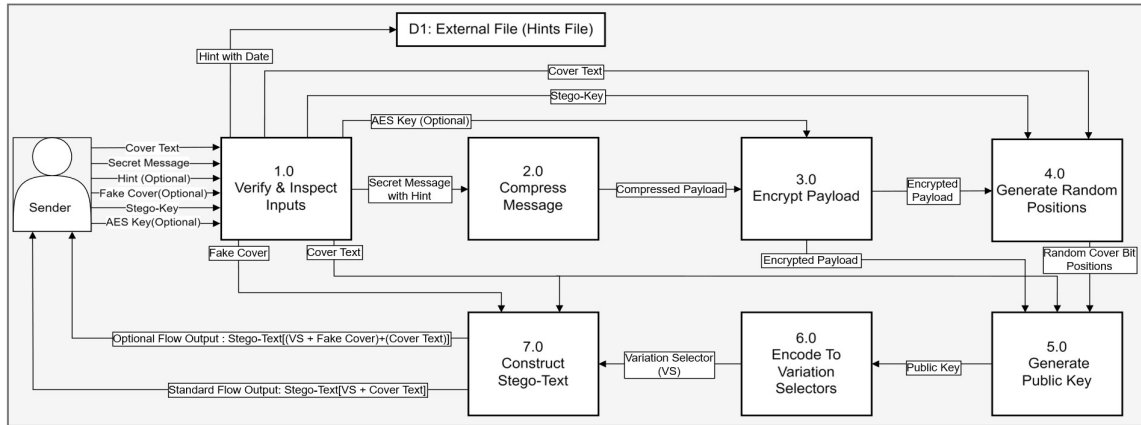


Figure 4.1 Sender Data Flow Diagram

This diagram shows how the secret text is hidden within the cover text.

Initially, the data flow begins from the Sender entity, which inputs (Secret text, Cover Text, Fake Cover, Hint, Stego key, AES Key) into Process 1.0 Verify & Inspect Inputs. The goal of this process is to verify the inputs to complete the hiding process properly. Among the operations are verifying the presence of required fields (Required Fields Validation) and verifying the size of the (Fake Cover) and the text capacity supported by the specified platform. It also stores the sent hint (Hint) in an external file D1.

After the verification process succeeds, the inputs are passed to their respective processes. The message along with the hint is sent to Process 2.0 Message Compress to compress it (using the Brotli algorithm). Afterwards, the compressed payload along with the AES Key is sent to Process 3.0 Encrypt Payload to encrypt it (encryption is a mandatory step, but adding the key is an optional step).

Simultaneously, Process 4.0 Generate Random Positions receives the encrypted payload from Process 3.0 to determine the payload length, along with the Stego-Key and the cover from Process 1.0 to generate scattered random bit positions along the length of the cover. Then, it sends these positions, along with the encrypted payload and the cover text from Process 1.0 to Process 5.0 Generate Public Key and performs the matching operation (XOR) to produce the Public Key, which is the public key that will be sent to the receiver.

This key is sent to Process 6.0 Encode to Variation Selectors, which aims to hide the key and make it invisible, as the bit string (Public Key) is encoded into invisible Unicode characters (Variation Selectors).

Finally, Process 7.0 Construct Stego-Text receives the (encoded Public Key, along with the cover text, and Fake Cover) and adds these invisible characters (VS) at the beginning of the cover text (Prepend). Its outputs to the sender either the standard path (Stego-Text) where the VS is added with the Cover Text, or the optional path (Stego-Text + Cover Text) where the VS is added with the fake cover.

Data Flow Diagram for the Extraction Process (Receiver)

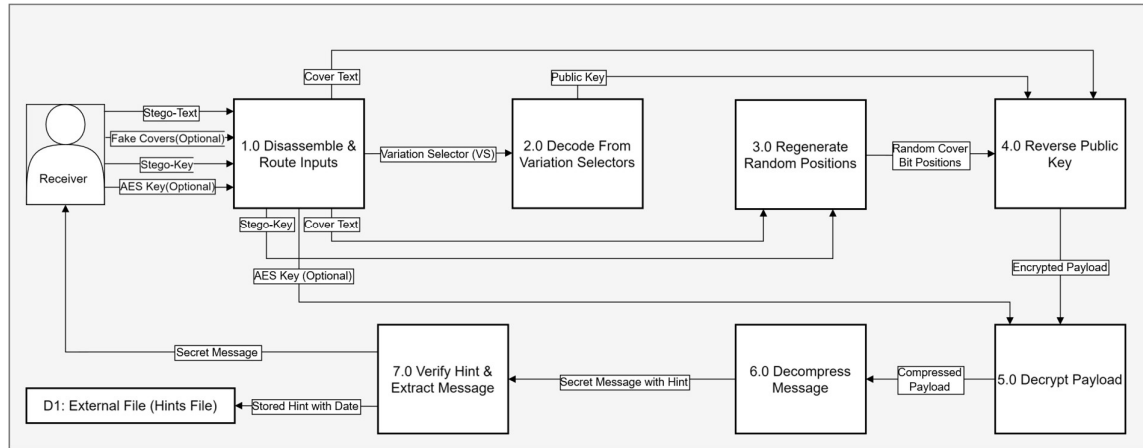


Figure 4.2 Receiver Data Flow Diagram

This diagram shows how the secret text is extracted from the cover text.

First, the data flow begins from the Receiver entity, which inputs (Stego-Text, Fake Covers, Stego-Key, AES Key) into Process 1.0, To disassemble & Route Inputs. This process disassembles the inputs, to complete the extraction process properly. Among the operations: separating the invisible characters from the text and routing the extracted inputs to their respective processes.

The invisible characters (Variation Selector) are sent to Process 2.0 Decode from Variation Selectors to decode them and revert them to the original bit string, which is the (Public Key).

Simultaneously, Process 3.0 Regenerate Random Positions receives the Stego-Key and Cover Text from Process 1.0 and the (public key) from Process 2.0 to determine its length to regenerate the same scattered random bit positions along the length of the cover. Then, it sends these positions (Random Cover Bit Positions) along with the Public Key and Cover Text to Process 4.0 Reverse Public Key and performs an (XOR) operation between the (Public Key) and (the cover text bits extracted from those random positions) to recover and produce the encrypted Payload (Encrypted Payload).

Afterwards, the encrypted payload along with the AES Key is sent to Process 5.0 Decrypt Payload to decrypt it. Then, the decrypted compressed payload (Compressed Payload) is sent to Process 6.0 Decompress Message to decompress it using the (Brotli) algorithm.

Finally, Process 7.0 Verify Hint & Extract Message receives the message with the hint (Secret text with Hint) and stores the incoming hint (Hint with Date) along with the reception date into the external file D1. It displays the secret text (Secret text) to the receiver (Receiver).

- **Workflow Diagram**

Workflow Diagram for the Embedding Process (Sender)

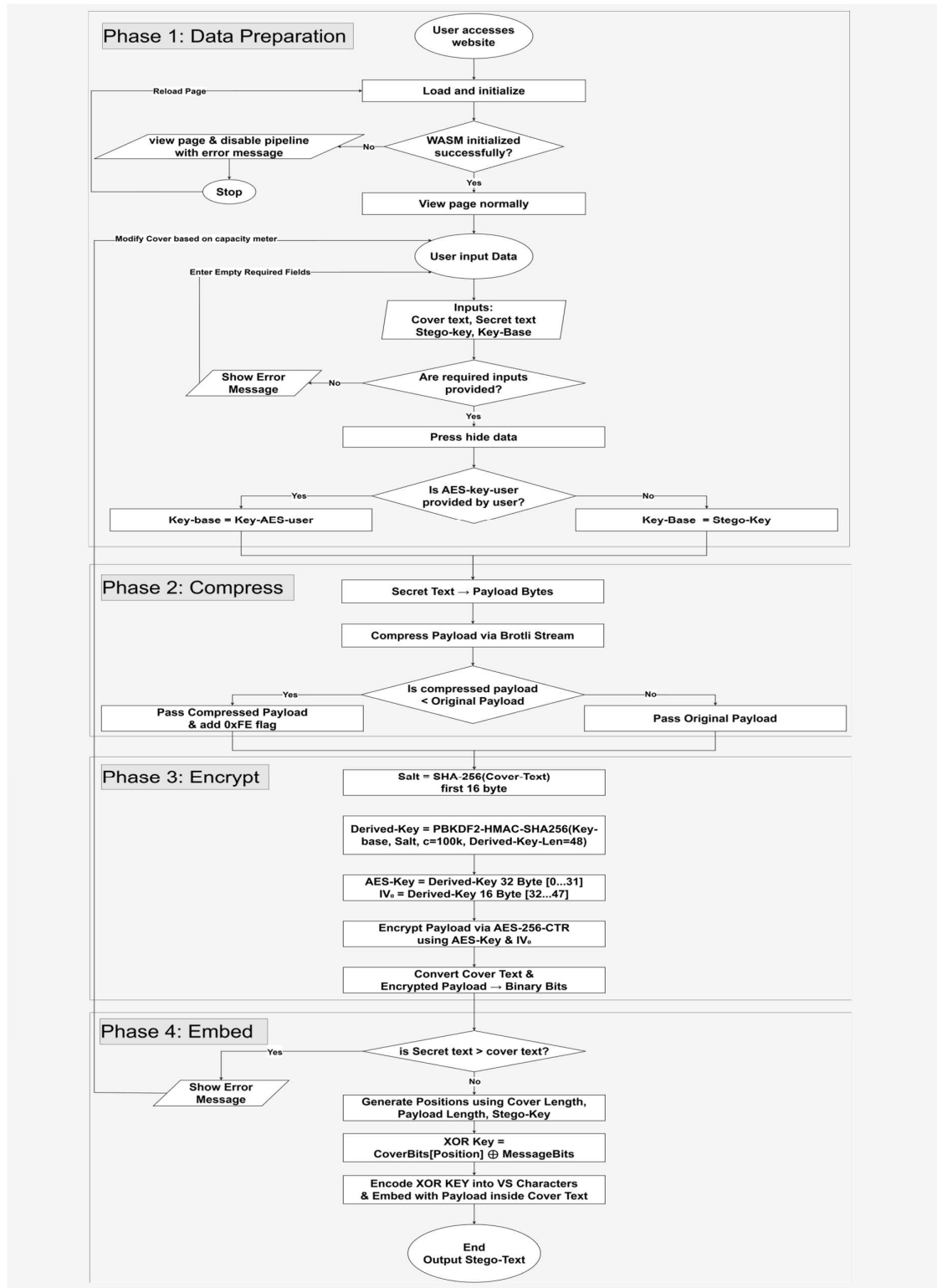


Figure 4.3 Sender Work Flow Diagram

The diagram shows the user journey, from the beginning of accessing the website, to the very end, printing the Stego-Text. The full process is divided into four phases, visualized to show how the system works, based on user inputs. where each phase is tied with a specific task related to the steganographic operation.

Phase 1 (Environment & Data Preparation): starts when the user accesses the website. the WASM module will be loaded and initialized into the Browser. Then the system does a check operation, if the WASM module was not initialized successfully, the page is displayed, but the process pipeline will be disabled, also showing error message to the user. Otherwise, the user will continue to the website normally, entering the necessary data, which are: the cover text, the secret text, a stego-key, as a main input, and as an optional input: AES key-base. After the data entered, the user presses the hide data button, the system then validates if the main inputs have been provided or not. If not, the system will show error message telling the user to fill the empty fields. In case the Key-Base is entered by the user, then its value will be assigned to be as AES-Key-base, but if the user doesn't enter a Key-Base, then stego-key value will be assigned to be as AES-Key-Base. After Checking that all fields provided correctly, the system continue its process to the second phase.

Phase 2 (Compression): In this phase, the secret text will be compressed, using a Brotli algorithm. The system converts the secret text into Bytes payload, to compress it as stream Bits. The compressed payload compared with the original. If the compressed payload size smaller, it will continue the process, with adding a flag byte (0xFE) in the beginning of the payload. If compression does not reduce the size, the original payload is passed forward as it is. The flag process helps to minimize the overhead on the cover-Text.

Phase 3 (Encryption): at this phase, generating a cryptographic Key is essential, to ensure the security of the payload. A salt value is taken by computing the SHA-256 hash of the cover text, then extracting the first 16 bytes from the hash. The salt, key-base, is then fed into the PBKDF2-HMAC-SHA256 key derivation function with 100,000 iterations to generate a 48-byte derived key. The first 32 bytes of the derived key, serve as the AES-256 encryption key, while the remaining 16 bytes, form the CTR counter (IV₀). The payload is then encrypted using AES-256-CTR with these parameters.

Phase 4 (Embedding): The last phase, it's responsible for the execution of the embedding process. Both the cover text, encrypted payload, are converted into their binary representations. The system first checks if the cover text has enough capacity to hold the secret data, if not, an error

message is displayed and the process stops. Only when the capacity is enough to hold the secret data, a random position of the cover-Text will be generated, based on the seed of Stego-Key, cover text length, and payload length. An XOR key is then produced, by performing an XOR operation between the selected bits of the cover text at these positions, and the message bits. The resulting XOR key is Converted into Variation Selector characters and embedded within the cover text. The process concludes by printing to the user the final stego-text as output.

Workflow Diagram for the Extraction Process (Receiver)

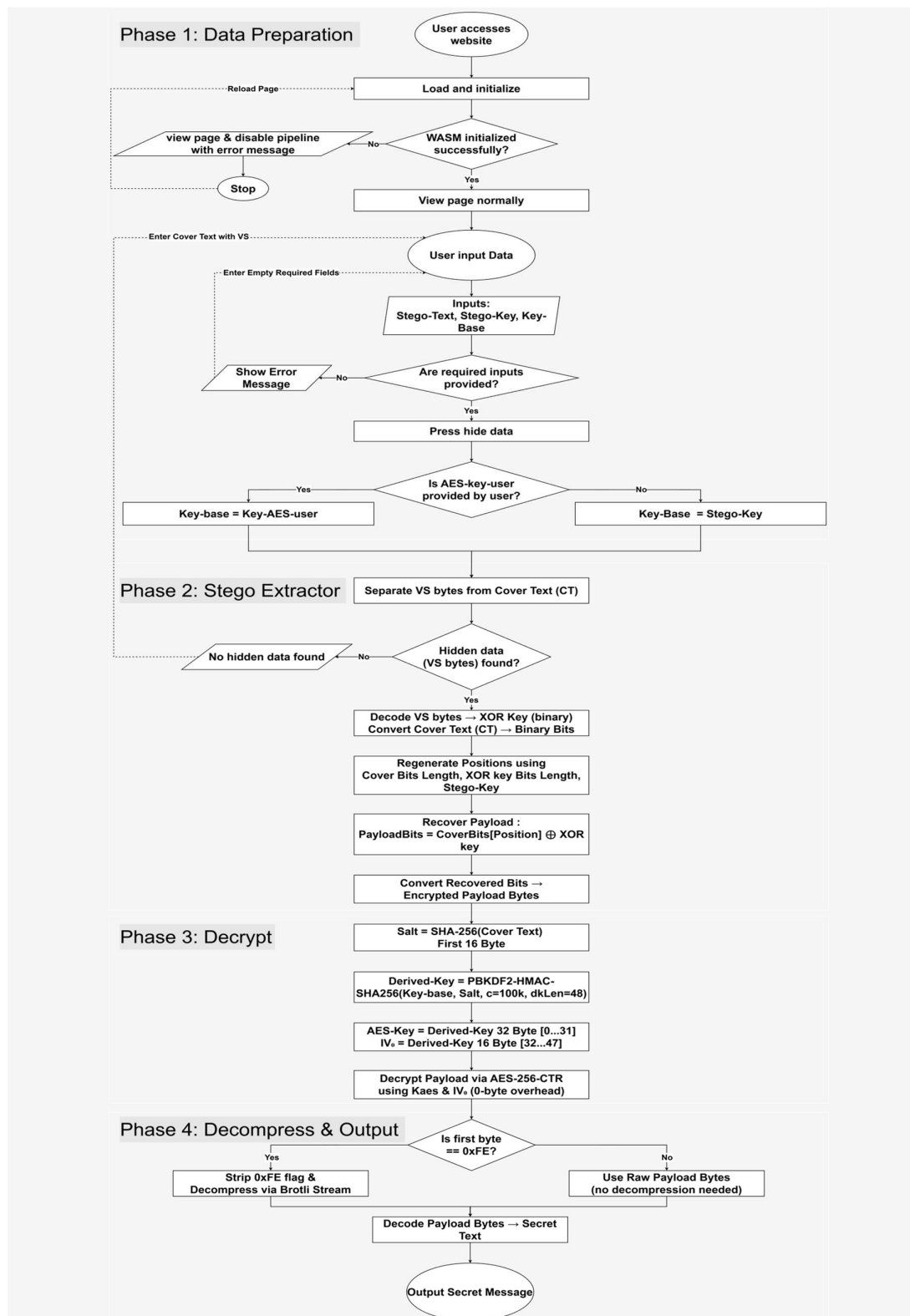


Figure 4.4 Receiver Workflow Diagram

The diagram shows the user journey, from the beginning of recovering a hidden message from the stego-text to the very end, from the perspective of the receiver. The full process is divided into four phases, visualized to show how the system works, based on user inputs. where each phase is tied with a specific task related to the stego extraction operation.

Phase 1 (Data Preparation): starts when the user accesses the website. the WASM module will be loaded and initialized into the browser. Then the system does a check operation, if the WASM module was not initialized successfully, the page is displayed, but the process pipeline will be disabled, also showing error message to the user. Otherwise, the user will continue to the website normally, entering the necessary data, which are: the stego-text, a stego-key, as a main input, and as an optional input: Key-base. After the data entered, the system then validates if the main inputs have been provided or not. If not, the system will show error message telling the user to fill the empty fields. In case the Key-Base is entered by the user, then its value will be assigned to be as Key-base directly, but if the user didn't enter a Key-Base, then stego-key value will be assigned to be as Key-Base. After Checking that all fields provided correctly, the system continue its process to the second phase.

Phase 2 (Stego Extraction): In this phase, the Variation Selector (VS) bytes will be separated from the visible cover text. The system determines if any hidden VS data is present. If no such content is found, an error message is displayed and the process stops. In case VS bytes have been identified, they are Decoded to extract the XOR key in binary format, also the cover text is converted into its binary bit representations. A random position will be reconstructed, based on the cover bits length, XOR key bits length, and the seed of Stego-Key. The payload bits are restored, by performing an XOR operation between the cover text bits at these positions, and the XOR key.

Phase 3 (Decryption): at this phase, deriving a cryptographic Key is essential, to ensure the decryption of the payload. A salt value is taken by computing the SHA-256 hash of the cover text, then extracting the first 16 bytes from the hash. The salt, key-base, is then fed into the PBKDF2-HMAC-SHA256 key derivation function with 100,000 iterations to generate a 48-byte derived key. The first 32 bytes of the derived key serve as the AES-256 encryption key, while the remaining 16 bytes form the CTR counter. The encrypted payload is then Decrypted using AES-256-CTR with these parameters.

Phase 4 (Decompression & Output): The last phase is responsible for the execution of the inspection process. The system first checks the first byte of the decrypted content, to determine if compression applied. If the 0xFE flag is present, the flag will be stripped and the remaining bytes

decompressed, using a Brotli stream decompression. If the flag is absent, the raw payload bytes are passed forward as it is. In both cases, the resulting bytes are Decoded to retrieve the original secret text. The process concludes by printing the user the final secret text as output.

Workflow Diagram for the Scanner Extraction Process (Receiver)

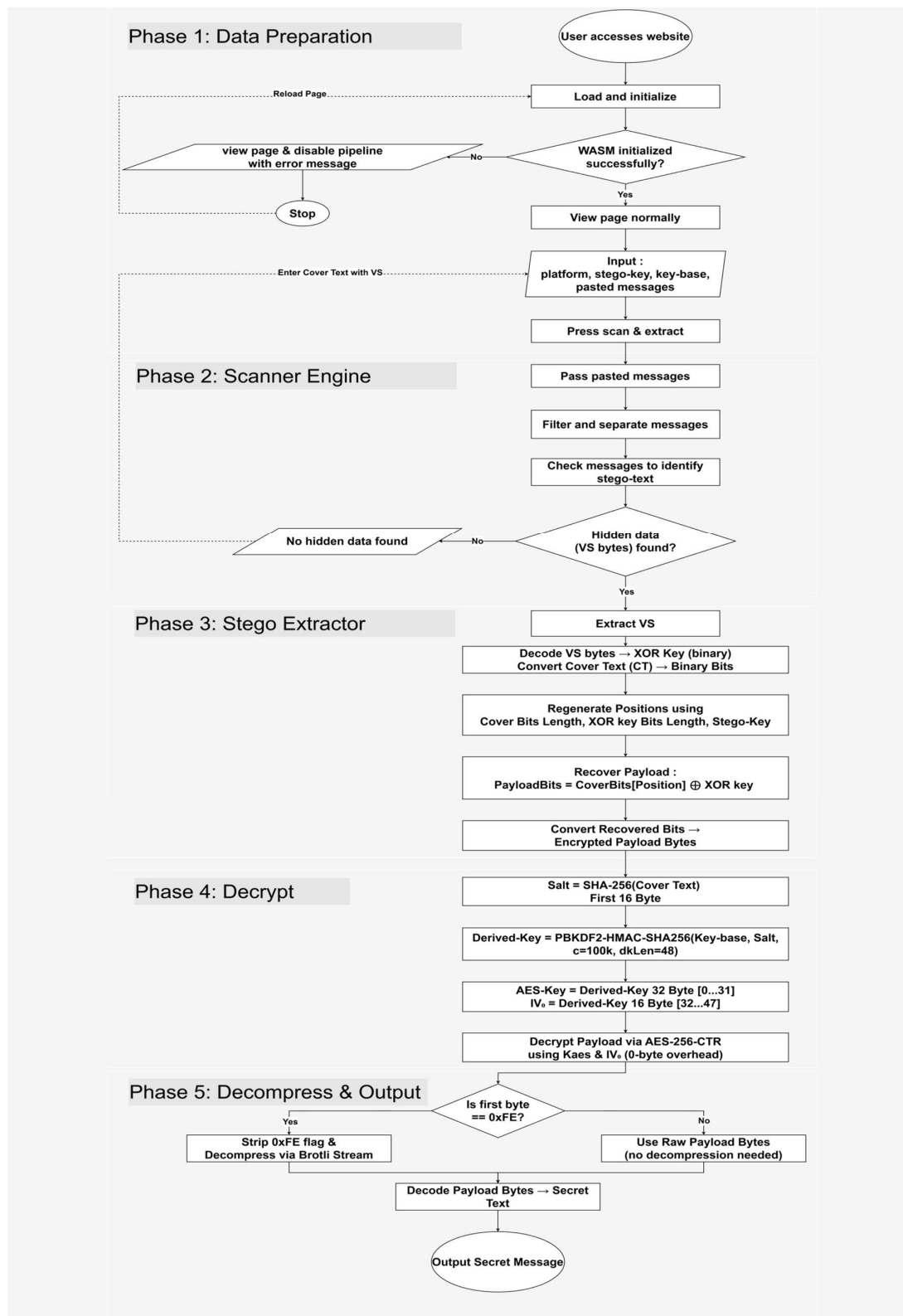


Figure 4.5 Receiver Workflow (Scanner) Diagram

The diagram shows the user journey, from the beginning of operating the scanning extraction mode, to the very end. The full process is designed for scenarios where the receiver has multiple pasted messages, visualized to show how the system identifies which ones contain steganographic data. Unlike the direct extraction, this mode introduces an additional scanning engine that analyses all messages in bulk, before proceeding with the extraction process.

Phase 1 (Initialization): starts when the user opens the application. the WASM module will be loaded and initialized into the browser. Then the system does a check operation, if the WASM module was not initialized successfully, the page is displayed, but the process pipeline will be disabled, also showing error message to the user. Otherwise, the user will continue to the website normally, entering the necessary data, which are: the target platform, the stego-key, the key-base, and a collection of pasted messages. After the data entered, the user presses the scan and extract button, to initiate the automated analysis. After Checking that all fields provided correctly, the system continue its process to the second phase.

Phase 2 (Scanner Engine): In this phase, it represents the main difference from direct extraction. The pasted messages will be fed into the scanner engine, to filter, and separate the messages. The system then checks each message to determine if it qualifies as stego-text, if it contains Variation Selector characters indicative of embedded data. In case stego-text candidates have been identified, the Variation Selector characters are Extracted from them, and the recover encrypted payload process is Executed, to obtain the encrypted message bits.

Phase 3 (Decryption): at this phase, the system receives the encrypted secret text, along with the key-base and the cover text. deriving a cryptographic Key is essential, using the identical procedure. A salt value is taken by computing the SHA-256 hash of the cover text, then extracting the first 16 bytes from the hash. The salt, key-base, are then fed into the PBKDF2 function with 100,000 iterations, and SHA-256 as the hash algorithm, to generate a 48-byte derived key. The first 32 bytes of the derived key serve as the AES decryption key, while the remaining 16 bytes form the counter value.

Phase 4 (Decompression and Output): The last phase, it's responsible for the execution of the output process. The system determines if the 0xFE compression flag is present in the decrypted payload. If the 0xFE flag is present, the payload is decompressed, using a Brotli stream decompression function. If the flag is absent, the payload remains in its raw, uncompressed form, passed forward as it is. In both cases, the final payload is then returned to the user interface. The process concludes by printing the user the recovered secret text as output.

- Sequence Diagram

Sequence Diagram of the System Hiding Pipeline (Sender)

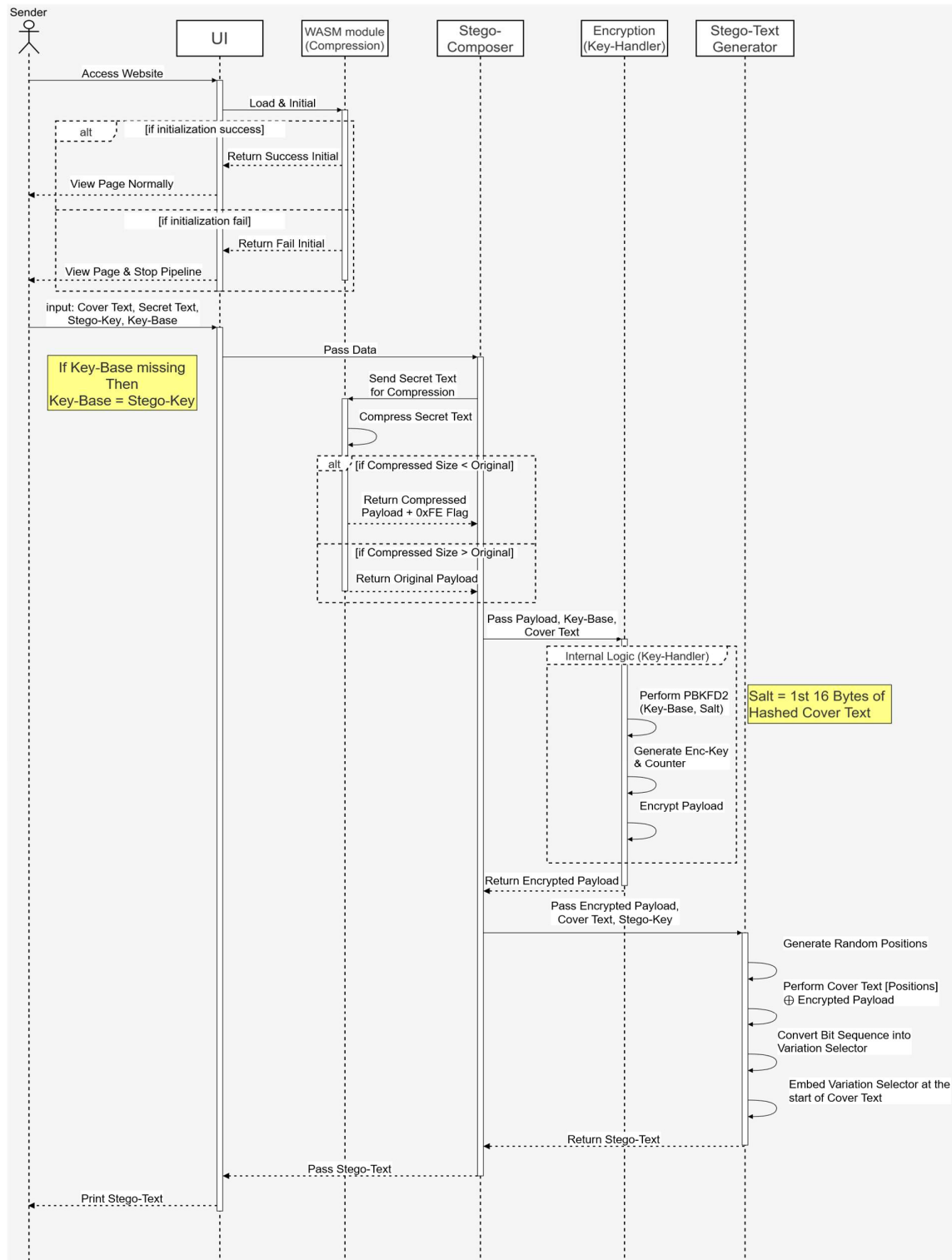


Figure 4.6 Sequence Diagram for Full System Pipeline (Sender)

The diagram shows the system execution, from the beginning of accessing the web application, to the very end, printing the Stego-Text. The full process is a continuous pipeline for the sender, visualized to show how the system works.

The process starts when the user accesses the web application. The UI immediately triggers an asynchronous call to the WebAssembly (WASM) Module, to handle execution initialization. Then the system does a check operation, if the WASM module was not initialized successfully, a failure flag is returned, prompting the UI to restrict dashboards functionality, also showing error message to the user to prevent uncompiled execution failures. Otherwise, a success confirmation is returned to the UI, the user will continue to the website normally, entering the necessary data, which are: the Cover Text, Secret Text, Stego-Key, and as an optional input: Key Base.

After the data entered, the user triggers the hidden data event. The UI sends the inputs to the stego-composer, that handles the pipeline for the rest of the sequential data transformation. First, cryptographic parameter verification is Executed by the stego-composer. If the user doesn't enter a Key-Base, the system assigns the Stego-Key value to it. In case the Key Base is entered by the user, then Key-Base value will be the input from the user.

The stego-composer dispatches the raw Secret Text payload to the Compress engine. The secret text will be compressed, using a Brotli algorithm to reduce its size. If the compressed size is less than the original raw size, the engine prefixes a unique validation identifier (0xFE) in the beginning of the payload and returns the Compressed Payload. If the original payload was smaller, the Raw Uncompressed Payload is passed forward as it is, to minimize the overhead on the cover text.

The stego-composer subsequently passes the validated payload, the Key Base, and the Cover Text, to the Encryption engine. The Encryption engine calls for the internal Key Handler. A salt value is calculated by computing the SHA-256 hash of the Cover Text, then extracting the first 16 bytes from the hash. The salt, Key Base, is then fed into the PBKDF2 function executing 100,000 iterations via SHA-256, to generate a 48-byte sequence. The first 32 bytes serve as the AES Encryption Key, while the remaining 16 bytes form the initialization Counter. The Key Handler passes these components back to the Encryption engine, which Executes the AES stream cipher in CTR mode, to transform the payload into an Encrypted Secret text returned to the stego-composer.

The stego-composer forwards the Encrypted Secret text, Stego-Key, and Cover Text to the position, and XOR-Key generator. This module mathematically generates a random position in the cover text and performs an XOR operation between the encrypted payload bits and the cover text random positions. The resulting XOR-Key sent back to the stego-composer.

In the final operation, the stego-composer drives the XOR-Key Binary Sequence, and the original Cover Text into the VS Coder engine. The binary sequence is Translated into specific Unicode Variation Selectors (VS) codes and embedded into the cover text. After a successful embedding, the VS Coder compiles and returns the final Stego-Text to the stego-composer. The stego composer passes it back to the UI. The process concludes by the UI printing the Stego-Text directly to the user.

Sequence Diagram of the Extraction Pipeline (Receiver)

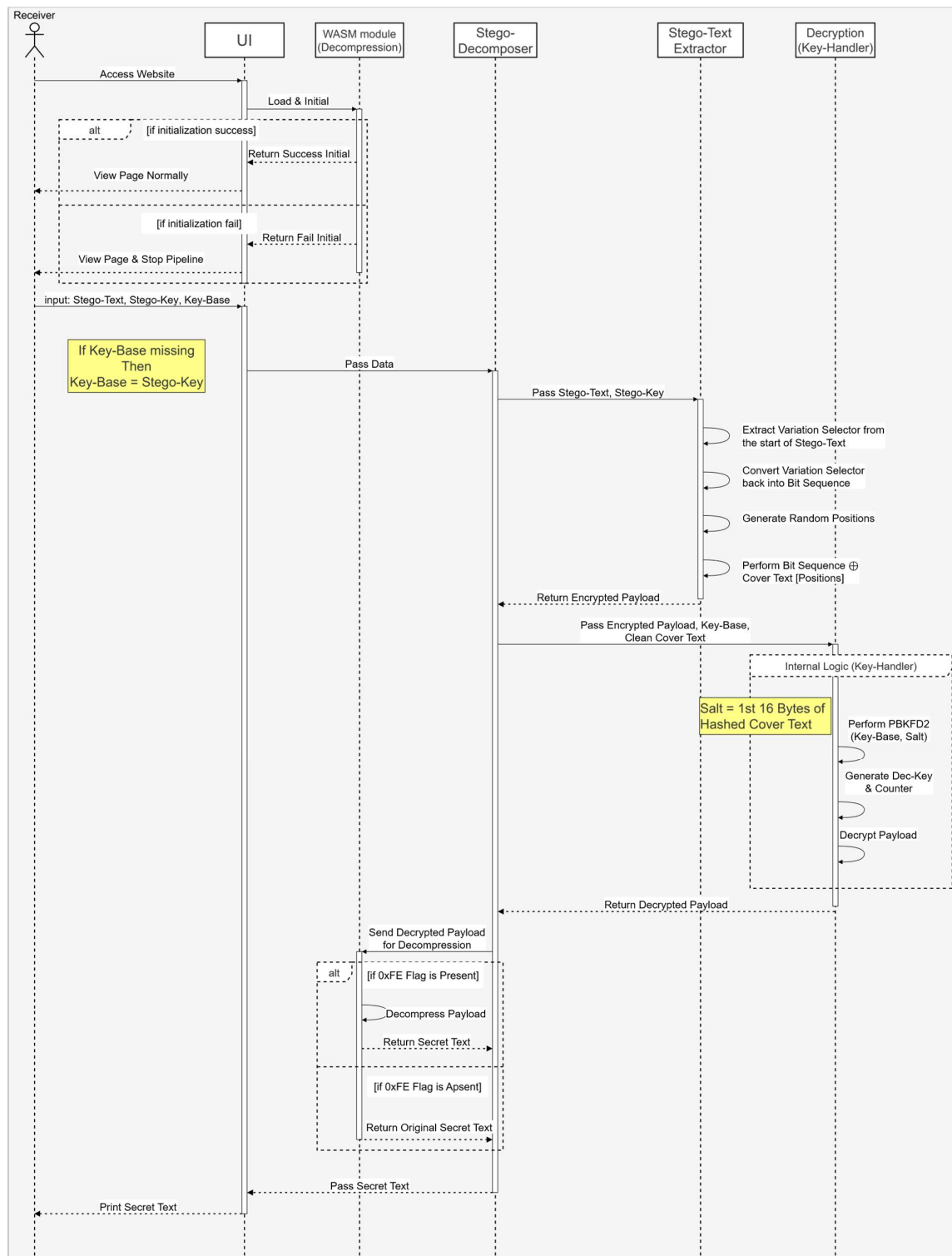


Figure 4.7 Sequence Diagram for Full System Pipeline (Receiver)

Just like the sending side, the receiving sequence starts once a person visits the web site. the interface, attempt to load and initialize the WebAssembly module inside the browser. If it was loaded successfully, the website would run smoothly with no disabling of the pipeline, but failing to build the WASM module, forces the pipeline to stop, and disabling the process, in addition to show error message to the user.

Actual process begins when user enters the secret text, cover text, stego-key, and an optional key-base. After the user presses the hide data button. The Stego-Decomposer starts initializing the key-base. If the user types a entered the key-base, the Stego-Decomposer assigns it as its value, if not entered, the fallback option took the stego-key value and assign it to the key-base.

After initializing the environment, user inputs. The stego-text goes to the Variation Selectors decoder engine, to extract the VS from cover text, then decodes the VS into its original XOR-key bits, this process isolates the hidden bits from the original cover text. both the VS, and cover text, are returned to the Stego-Decomposer.

By continuing the process, the stego-decomposer passes the extracted XOR-key, stego-key, cover text, to the position's generator. The positions are generated based on the stego-key seed, payload, and cover length. After generating the positions, a reverse XOR operation happens to retrieve the original payload. the reverse XOR, executed between the cover text bits based on the generated positions, and the XOR-key bits. The output of this operation is the encrypted payload; this payload will return to the stego-decomposer.

the stego-decomposer, pass the encrypted payload, and the cover text, to derive a decryption key. By assigning salt value, to be the first 16 bytes of the hashed cover text. Then, by executing PBKFD2, derive key function, using key-base as password, salt, 100,000 iteration, and HMAC-SHA-256, producing a hash with 48-byte length. the first 32 bytes are sliced, to be the decryption key, while the rest bytes, will be assigned as counter (IV₀), to be able to execute the AES decryption, on the encrypted payload in CTR mode.

Finally, a scan on the decrypted payload checks for a flag. If the 0xFE flag exists, then a decompress will be executed, without that flag, the code passes the decompress. The passed payload will return to the stego-Decomposer, to convert is to string, then print the secret text to the user.

Sequence Diagram of the Chat Scanner Extraction Pipeline

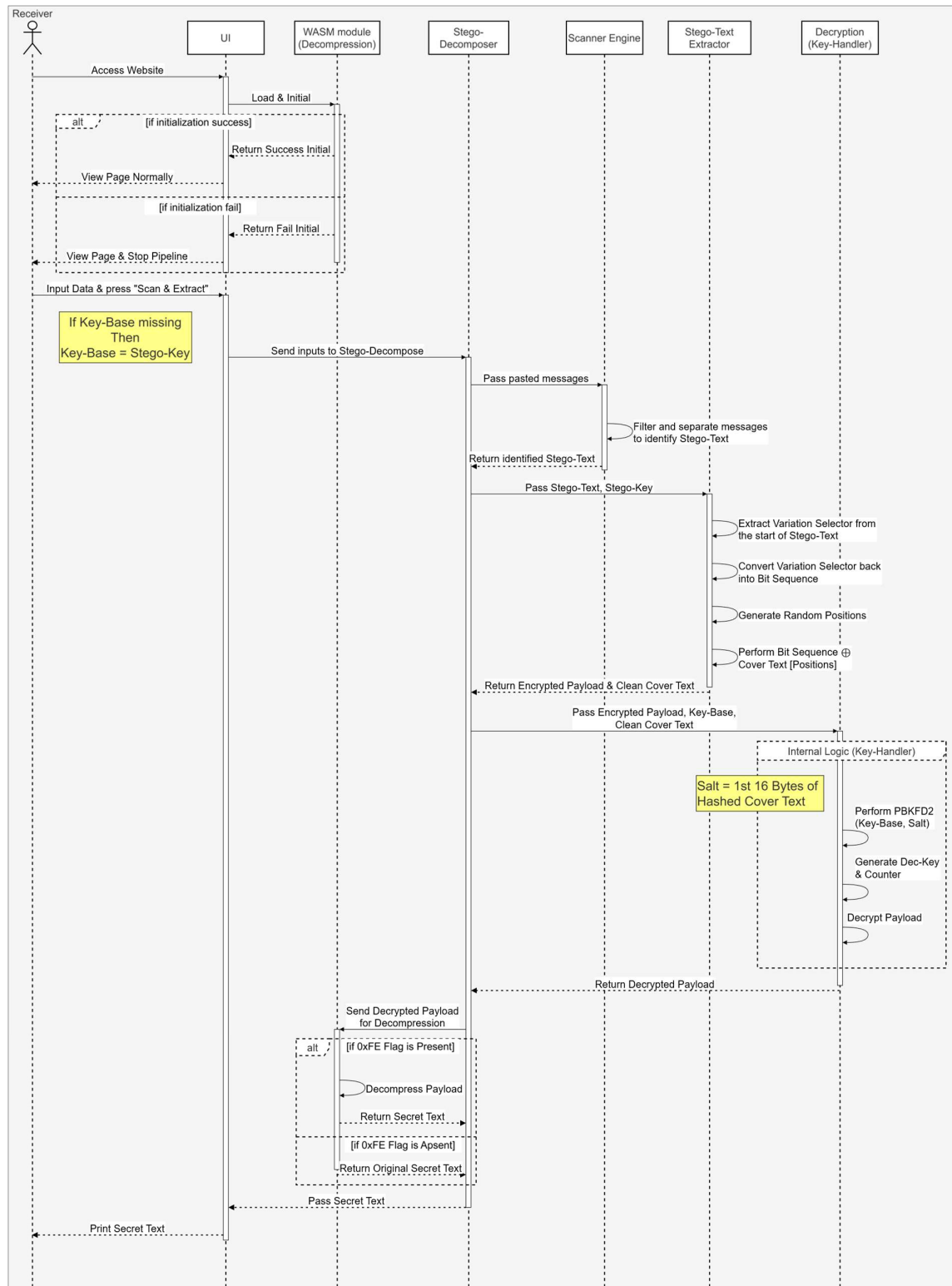


Figure 4.8 Sequence Diagram for Full Scanner Pipeline (Receiver)

The receiving scanner sequence starts once a person visits the web site. the interface, attempt to load and initialize the WebAssembly module inside the browser. If it was loaded successfully, the website would run smoothly with no disabling of the pipeline, but failing to build the WASM module, forces the pipeline to stop, and disabling the process, in addition to show error message to the user.

Actual process begins when user enters the secret text. cover text, stego-key, and an optional key-base. After the user presses the hide data button. The Stego-Decomposer starts initializing the key-base. If the user types a entered the key-base, the Stego-Decomposer assigns it as its value, if not entered, the fallback option took the stego-key value and assign it to the key-base.

The only difference between the regular extraction, and scanner extraction, is this step. The user needs to choose the platform, of pasted messages, because every social media platform has their own format, when users copy more than single message. Then enter the stego-key, and the pasted messages. The first step is to separate and filter each message individually, then check every message, to identify which message is the stego-text.

After identifying the stego-text, it goes to the Variation Selectors decoder engine, to extract the VS from cover text, then decodes the VS into its original XOR-key bits, this process isolates the hidden bits from the original cover text. both the VS, and cover text, are returned to the Stego-Decomposer.

By continuing the process, the stego-decomposer passes the extracted XOR-key, stego-key, cover text, to the position's generator. The positions are generated based on the stego-key seed, payload, and cover length. After generating the positions, a reverse XOR operation happens to retrieve the original payload. the reverse XOR, executed between the cover text bits based on the generated positions, and the XOR-key bits. The output of this operation is the encrypted payload; this payload will return to the stego-decomposer.

the stego-decomposer, pass the encrypted payload, and the cover text, to derive a decryption key. By assigning salt value, to be the first 16 bytes of the hashed cover text. Then, by executing PBKFD2, derive key function, using key-base as password, salt, 100,000 iteration, and HMAC-SHA-256, producing a hash with 48-byte length. the first 32 bytes are sliced, to be the decryption key, while the rest bytes, will be assigned as counter (IV₀), to be able to execute the AES decryption, on the encrypted payload in CTR mode.

Finally, a scan on the decrypted payload checks for a flag. If the 0xFE flag exists, then a decompress will be executed, without that flag, the code passes the decompress. The passed payload will return to the stego-Decomposer, to convert is to string, then print the secret text to the user.

- **Physical Diagram**

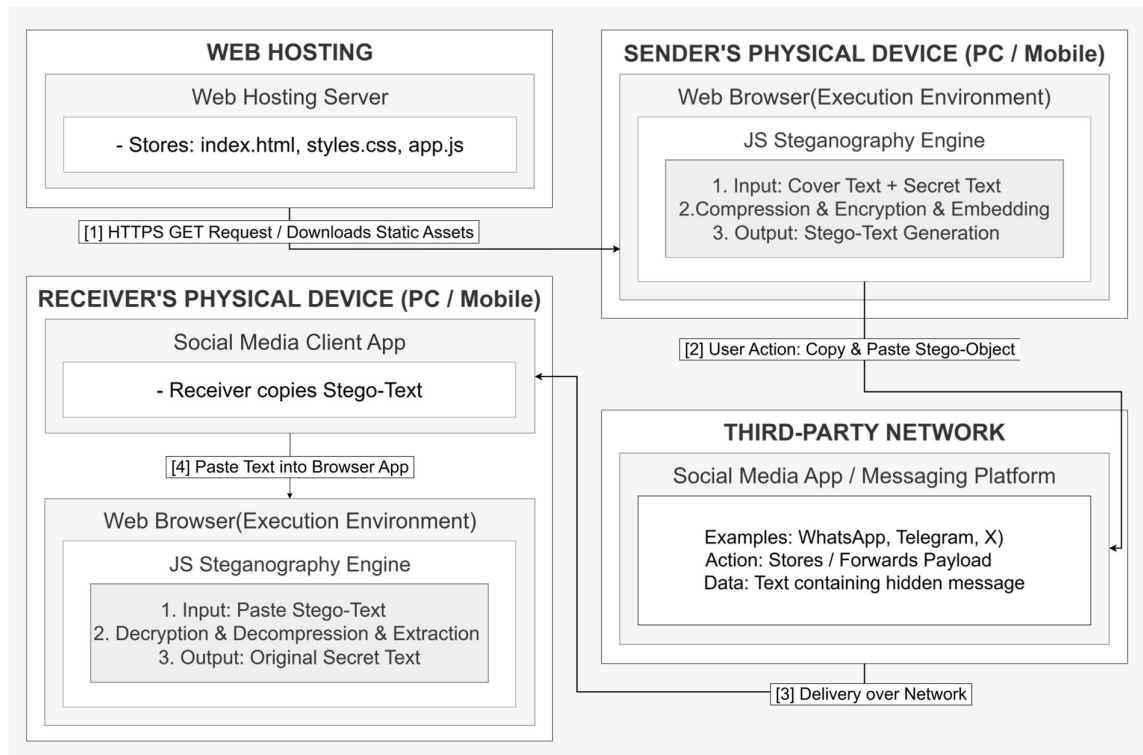


Figure 4.9 Physical Diagram of The System

This diagram shows how the process happens, but on physical layer, how the user requests the website from the hosting server, how the sender embed process happens, and how the receiver extracts it.

web hosting: the hosting layer, deployed via static pages hosting, does not compute, process, or stores users' data, only hosting the website to deliver it to users. When a user accesses the page, the local device sends an HTTPS GET REQUEST, to fetch and download the static assets into the local web browser environment.

Sender Side local hardware and execution: when downloading is finished, execution will be only on the personal user device, within the safe sandbox environment, that provided by the browser. The WASM module starts initializing on the user browser, to provide Brotli stream compression, to the secret text, then, by using the web crypto Api embedded in the browser, AES encryption is executed in CTR mode, with the help of PBKFD2, to derive the cryptographic key, and the counter (IV₀). Finally, an XOR operation between the cover, and encrypted payload, resulting in XOR-Key binary, that will be converted into VS, to be embedded with cover text.

Third Party network transport medium: the output, which is the stego-text, will be sent to the receiver via social media app, these third-party applications work as a medium between the sender and the receiver,

Receiver Side Client Application and Extraction: on the receiver side, when he receives the stego-text, which mirrors the sender side operation, from accessing the website, with reverse process for the extraction process, within the safe environment, the sandbox provided by the browser. Extracting the secret text and prints it to the user.

Trust Boundary Diagram

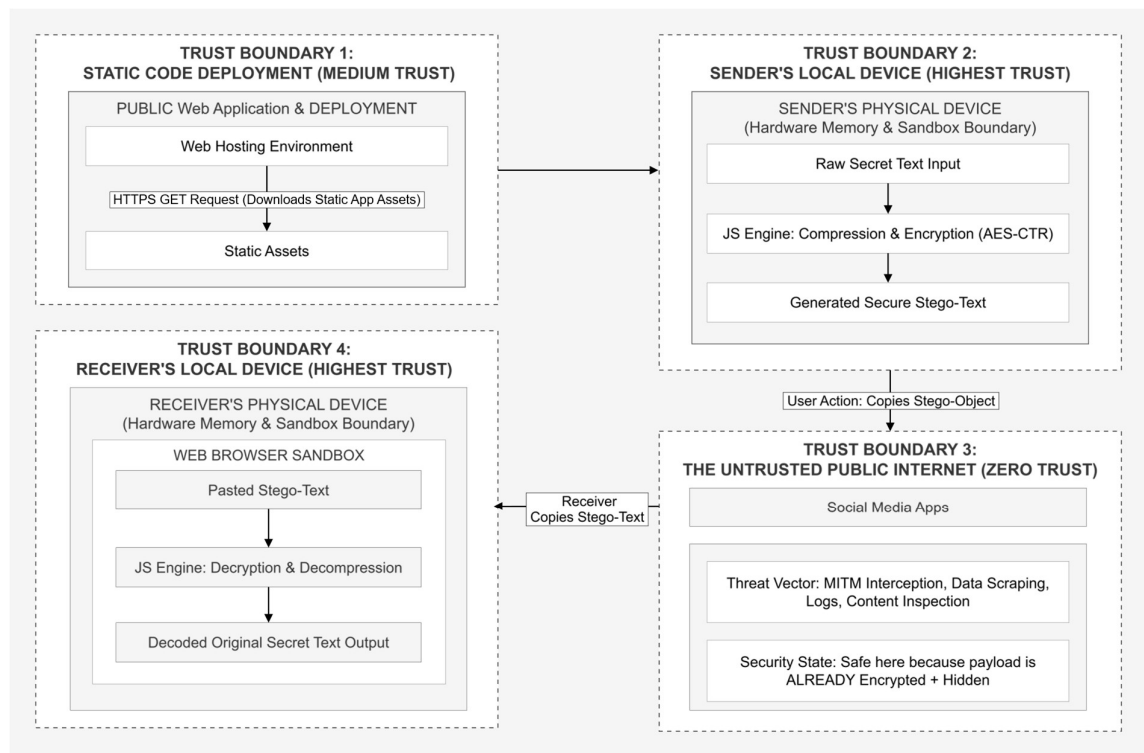


Figure 4.10 Trust Boundary Diagram

This diagram shows the trust boundaries between the hosting server, sender device, social media applications, and receiver device. Visualizing which part of them considered safe, and which is not.

Trust Boundary 1: the web hosting contains the hosting service for the web application. Since there is no private data, Encryption key, or any text processed on these services. and there is no real time process of the embedding operation. These can be considered as medium trust; their only purpose is to store the website and send the website to users when they request it.

Trust Boundary 2: The sender's local device, representing the highest trust zone. This boundary is highly constrained, within the hardware of the sender device, and the strict isolation of the web browser sandbox. Inside this secure environment, the JavaScript engine captures the secret text and executes the Brotli compression. Then performing an encryption, with the help of PBKDF2, for key derivation. using AES in CTR mode. The sensitive plain texts, and operational cryptographic keys, happen only within the volatile memory. The output of this boundary is the stego-text, which means no unencrypted data ever go out of the environment.

Trust Boundary 3: The untrusted public internet, classified as a zero-trust zone. When the sender copies, and sends the stego-text, it goes into public networks and social media platforms. This boundary has severe threats. Still, the system is designed to reduce these threats, the data moving in this environment is safe. because the secret payload is encrypted by AES, and visually obfuscated within the cover text, making it imperceptible to external monitors.

Boundary 4: The receiver's local device, as the sender's environment, representing as the highest trust zone. When receiving the stego-text, the receiver pastes it into his web browser sandbox. Safely, isolated from the external threats, the local JavaScript engine, reversing the cryptographic and steganographic. The engine extracts the VS, executes local decryption, and performs decompression, entirely within the browser's volatile memory. securing the secret text. With high strict to the authorized receiver, without exposing data to external entities.

4.1.2 Design Principles

The system we are building inherits the base framework of cyber security, the CIA triad (Confidentiality, Integrity, Availability,) as the foundation, of how we implemented the design principles on our system. We divided each principle under its component from the CIA triad.

Confidentiality

- **Defense in Depth**

The system provides a tri security layer, during the embedding process:

- Layer 1 (Cryptographic): The AES-256-CTR stream cipher, used to encrypt the secret payload.
- Layer 2 (Key Diversification): PBKDF2 stretches the password, through 100,000 hashing rounds. Even if an attacker discovers the hidden VS, they cannot read the secret text, without breaking the independent cryptographic, and key stretching layers.

- Layer 3 (Steganographic): this layer generates a random position at the cover text, based on the stego-key, to produce a bitwise XOR-Mapping, between the cover bits and the secret bits. Resulting in XOR-Key, that will encode to VS, then embed into the cover.

Secure by Defaults

To provide sustainable security, a default value, which is the Stego-Key value, will be assigned to be as Key-Base, if the user didn't provide it. That provides secure password to feed it into the PBKFD2, not a pre-defined, or static value to be used.

Integrity

Don't Trust Services

The system is designed to run entirely on the client side. With no interference of third parties, to execute any process of the system. From the compression algorithm, provided by loading and initializing the WASM module on the user browser. Then by executing the AES in CTR mode using Web Crypto API, with the Key Diver technique, using PBKFD2 algorithm, to produce the cryptographic key. To the embedding process, with every step happens on the embedding pipeline.

Availability

Minimize attack surface area

Since the system was designed to run on client side. This achieves a near zero attack surface. With no API exposed, and no backend server, traditional web attacks, such as SQL injections, are eliminated.

Fail Securely

By using Try and Catch on almost every side of the website. Handling the unexpected error, wither it was by wrong user inputs, unexpected process, or any other unexpected event. There will always be an error message shown to the user, preventing the system from colliding.

4.2 Components

4.2.1 System Modules

The system is divided into five core functional modules, each responsible for a specific stage of the data processing pipeline. Under each model, there will be a pseudo code, to show the logic behind the model.

User Interface (UI):

This model is considered as the primary interaction interface. Built with HTML, CSS, and JS. It takes the user inputs, handling the capacity meter, showing the user the accepted length of secret text, based on the length of the cover text. In addition to prevent the process from starting, if the required fields are not entered.

Pseudo code:

Sender Side

```
Input: Secret_Text (String), Cover_Text (String), User_Stego_Key (String), User_Aes_Key (String)
Begin
  // Step 1: WebAssembly Environment Verification (On Page Load)
  WASM_Status ← Initialize_WebAssembly_Module()
  If WASM_Status is FAILED Then
    Disable_UI_Controls()
    Display_Error("Initialization failed: WebAssembly is required.")
    Stop_Execution()
  EndIf
  // Step 2: Payload Capacity Validation (On "Hide Data" Trigger)
  If Length(Secret_Text) > Length(Cover_Text) Then
    Display_Error("Capacity Exceeded: Cover text is too short for the secret message.")
    Stop_Execution()
  EndIf
  // Step 3: Automated Key Configuration and Fallback Logic
  If User_Aes_Key is Empty Then
    Active_Aes_Key ← Active_Stego_Key
  Else
    Active_Aes_Key ← User_Aes_Key
  EndIf
  // Step 4: Proceed to Execution Pipeline to Core Processing Modules
  PassToEmbeddingProcess (Secret_Text, Cover_Text, Active_Stego_Key, Active_Aes_Key)
End
```

Figure 4.11 Pseudo code for UI Module (Sender Side)

Receiver Side

```
Input: Stego_Input (String), User_Stego_Key (String), User_Aes_Key (String)
Begin
  // Step 1: WebAssembly Environment Verification (On Page Load)
  WASM_Status ← Initialize_WebAssembly_Module()
  If WASM_Status is FAILED Then
    Disable_UI_Controls()
    Display_Error("Initialization failed: WebAssembly context is required for core streaming.")
    Stop_Execution()
  EndIf
  // Step 2: Input Presence Validation (On "Extract Data" Trigger)
  If Stego_Input is Empty Then
    Display_Error("Extraction Error: The Stego-Textext input field cannot be empty.")
    Stop_Execution()
  EndIf
  // Step 3: Preliminary Structural Isolation
  // Isolate invisible VS characters from the text to retrieve the Clean Cover Context
  Isolated_VS_Bytes, Clean_Cover_Text ← Isolate_And_Decompose_Variation_Selectors(Stego_Input)
  If Length(Isolated_VS_Bytes) == 0 Then
    Display_Error("Integrity Error: No hidden variation selector payload detected in the provided text.")
    Stop_Execution()
  EndIf
  // Step 4: Automated Key Configuration and Fallback Logic (Symmetric Mirroring)
  If User_Aes_Key is Empty Then
    // Fallback: Bind the cryptographic layer directly to the resolved stego-key context
    Active_Aes_Key ← Active_Stego_Key
  Else
    Active_Aes_Key ← User_Aes_Key
  EndIf
  // Step 5: Proceed to Core Extraction Pipeline
  PassToExtractionProcess(Isolated_VS_Bytes, Clean_Cover_Text, Active_Stego_Key, Active_Aes_Key)
End
```

Figure 4.12 Pseudo code for UI Module (Receiver Side)

Scanner Engine:

The scanner engine is an extracting method, choose by the user, when he doesn't know which message is the stego-text, copying more than one message from social media apps, adds more information to the copied messages. That prevents our system from running correctly. The scanner engine is and separation and filtering layer, running at the beginning of the extraction pipeline. By separating each message individually, then check each message to identify which message is the stego-text. After the identification, the process continues normally to the end.

Full Pseudo code for Scanner Engine:

```
Input: Raw_Chat_Transcript (String), Stego_Key (String), Optional_AES_Key (String)
Output: Extracted_Secret_Message (String), Secret_Hint (String)
Begin
    // Step 1: Automated Platform Signature Detection and Parsing
    Platform_Signature ← Identify_Chat_Platform_Format(Raw_Chat_Transcript)
    Structured_Messages[] ← Filter_And_Segment_Transcript(Raw_Chat_Transcript, Platform_Signature)
    // Step 2: Component Separation and Stego-Text Isolation
    Clean_Messages[], VS_Payload_Bytes, Carrier_Index ←
    Isolate_Steganographic_Payload(Structured_Messages[])
    If VS_Payload_Bytes is Null Then
        Raise_Error("Security Exception: No embedded steganographic components detected.")
    EndIf
    // Translate extracted Variation Selectors into a binary stream for alignment matching
    XOR_Key_Binary ← Convert_Bytes_To_Binary_String(VS_Payload_Bytes)
    // Step 3: Primary Candidate Evaluation Loop (Iterating over non-carrier messages)
    For i ← 0 To Length(Clean_Messages) - 1 Do
        If i == Carrier_Index Then Continue EndIf // Skip active carrier during candidate matching
        Candidate_Cover ← Clean_Messages[i]
        // Execute dynamic inverse bit-mapping and XOR decryption
        Target_Positions ← Generate_Deterministic_Positions(Candidate_Cover_Length,
        XOR_Key_Binary_Length, Resolved_Stego_Key)
        Recovered_Binary ← Reverse_Bitwise_XOR(Candidate_Cover, Target_Positions, XOR_Key_Binary)
        Payload_Bytes ← Convert_Binary_To_Bytes(Recovered_Binary)
        // Execute Cryptographic Decryption if key parameters exist
        If Optional_AES_Key is Not Empty Then
            Payload_Bytes ← Execute_AES_CTR_Decryption(Payload_Bytes, Optional_AES_Key, Candidate_Cover)
        EndIf
        // Evaluate decompression verification flag (0xFE)
        If Payload_Bytes[0] == 0xFE Then
            Payload_Bytes ← Execute_Brotli_Stream-Decompression(Payload_Bytes[1..End])
        EndIf
        // Verify structural integrity and language readability
        (Secret_Message, Secret_Hint, Valid) ← Validate_And_Parse_UTF8_Payload(Payload_Bytes)
        If Valid is TRUE Then
            Return Secret_Message, Secret_Hint
        EndIf
    EndFor
    // Step 4: Carrier-As-Cover Fallback Execution Phase
    // (Triggered only if secret text cannot be mapped to other surrounding messages)
    If Carrier_Index ≥ 0 Then
        Fallback_Cover ← Clean_Messages[Carrier_Index]
        Target_Positions ← Generate_Deterministic_Positions(Fallback_Cover_Length, XOR_Key_Binary_Length,
        Resolved_Stego_Key)
        Recovered_Binary ← Reverse_Bitwise_XOR(Fallback_Cover, Target_Positions, XOR_Key_Binary)
        Payload_Bytes ← Convert_Binary_To_Bytes(Recovered_Binary)
        If Optional_AES_Key is Not Empty Then
            Payload_Bytes ← Execute_AES_CTR_Decryption(Payload_Bytes, Optional_AES_Key, Fallback_Cover)
        EndIf
        If Payload_Bytes[0] == 0xFE Then
            Payload_Bytes ← Execute_Brotli_Stream-Decompression(Payload_Bytes[1..End])
        EndIf
        (Secret_Message, Secret_Hint, Valid) ← Validate_And_Parse_UTF8_Payload(Payload_Bytes)
        If Valid is TRUE Then
            Return Secret_Message, Secret_Hint
        EndIf
    EndIf
    Raise_Error("Extraction Exception: Failed to decode or map payload against the chat log matrix.")
End
```

Figure 4.13 Pseudo code for Scanner Engine

Compression/Decompression Engine (WASM Brotli):

Since capacity is one of the three most important criteria, that measures the quality of each steganography system. We, in our system, used a high, new improvement algorithm to compress the secret text. We used the Brotli algorithm, originally, it's a library that wrote in Rust Language, since this algorithm is developed to run on servers, and its open source, we have managed to load the WASM module on the user browser the moment he accesses the website. By encoding the entire WASM module into base64 in the developing, then converting it to its original format as bytes, then initializing it to run smoothly during the pipeline.

Pseudo code:

```
--- ROUTINE 1: RUNTIME WEBASSEMBLY INITIALIZATION (Executed On Page Load) ---
Procedure Initialize_Brotli_WASM_Context()
Begin
  If WASM_Ready_Status is TRUE Then
    Exit // Prevent redundant memory instantiation
  EndIf
  // Abstracted Base64 decoding and runtime compilation
  WASM_Binary_Buffer ← Decode_Base64_To_ArrayBuffer(EMBEDDED_WASM_BASE64_STRING)
  WASM_Instance ← WebAssembly_Compile_And_Instantiate(WASM_Binary_Buffer)
  Expose_Global_Streaming_Functions(WASM_Instance.compress, WASM_Instance.decompress)
  WASM_Ready_Status ← TRUE
End
// --- ROUTINE 2: DIRECT DATA COMPRESSION ---
Function Execute_Brotli_Compression(Raw_Payload_Bytes)
Input: Raw_Payload_Bytes (Byte Array)
Output: Compressed_Output_Bytes (Byte Array)
Begin
  // Process input data linearly through the globally pre-initialized WASM stream buffer
  Compressed_Output_Bytes ← Execute_WASM_Stream_Compress(Raw_Payload_Bytes, Quality=11,
Chunk_Size=4096)
  Return Compressed_Output_Bytes
End
// --- ROUTINE 3: DIRECT DATA DECOMPRESSION ---
Function Execute_Brotli-Decompression(Compressed_Bytes)
Input: Compressed_Bytes (Byte Array)
Output: Restored_Original_Bytes (Byte Array)
Begin
  // Execute raw stream decompression via the globally pre-initialized WASM context
  Restored_Original_Bytes ← Execute_WASM_Stream-Decompress(Compressed_Bytes, Output_Size=4096)
  Return Restored_Original_Bytes
End
```

Figure 4.14 Pseudo code for Co/Decompression Engine

Cryptographic Engine (Key Handler & En/Decryption):

This module enforces the data confidentiality. It consists of the Key Handler, which derives a cryptographic key, using PBKDF2 (Key-Base, 100,000 iterations using a SHA-256 Salt extracted from the Cover Text), and the Encryption module, which executes the AES-CTR stream cipher to encrypt/decrypt the compressed byte arrays.

Pseudo code:

```
// --- ROUTINE 1: CRYPTOGRAPHIC PARAMETER DERIVATION ---
Function Derive_Cryptographic_Parameters(User_Password, Cover_Text)
Begin
    // Step 1: Derive a deterministic 16-byte Salt dynamically from the Cover Text
    Cover_Hash ← SHA256(Cover_Text)
    Cryptographic_Salt ← Cover_Hash[0 .. 15]
    // Step 2: Prepare raw string into standard cryptographic key material
    Raw_Key_Material ← Import_Raw_Cryptographic_Key(User_Password)
    // Step 3: Execute PBKDF2 key stretching (100,000 iterations via SHA-256)
    Derived_48_Bytes ← PBKDF2(Raw_Key_Material, Cryptographic_Salt, Iterations=100000, Hash=SHA-256)
    // Step 4: Split the extended key material into operational AES parameters
    AES_256_Key ← Derived_48_Bytes[0 .. 31] // First 32 Bytes (256 bits)
    CTR_Initialization_Vector ← Derived_48_Bytes[32 .. 47] // Remaining 16 Bytes (128 bits)
    Return AES_256_Key, CTR_Initialization_Vector
End

// --- ROUTINE 2: PAYLOAD ENCRYPTION ---
Function Encrypt_Payload_CTR(Plain_Bytes, User_Password, Cover_Text)
Input: Plain_Bytes (Byte Array), User_Password (String), Cover_Text (String)
Output: Cipher_Bytes (Byte Array)
Begin
    AES_256_Key, CTR_IV ← Call Derive_Cryptographic_Parameters(User_Password, Cover_Text)
    // Execute AES Encryption in Counter Mode
    Cipher_Bytes ← AES_CTR_Encrypt(Key: AES_256_Key, Counter: CTR_IV, Plaintext: Plain_Bytes)
    Return Cipher_Bytes
End

// --- ROUTINE 3: PAYLOAD DECRYPTION ---
Function Decrypt_Payload_CTR(Cipher_Bytes, User_Password, Cover_Text)
Input: Cipher_Bytes (Byte Array), User_Password (String), Cover_Text (String)
Output: Plain_Bytes (Byte Array)
Begin
    AES_256_Key, CTR_IV ← Call Derive_Cryptographic_Parameters(User_Password, Cover_Text)
    // Execute AES Decryption in Counter Mode
    Plain_Bytes ← AES_CTR_Decrypt(Key: AES_256_Key, Counter: CTR_IV, Ciphertext: Cipher_Bytes)
    Return Plain_Bytes
End
```

Figure 4.15 Pseudo code for En/Decryption Engine

Steganographic Engine (XOR Generator & VS Co/Decoder):

This module executes the core bitmapping and embedding logic. It executes a random positions Generator. These random positions will be mapped to the cover text bits. To perform bitwise XOR operations. The VS Co/Decoder then translates this randomized binary sequence into invisible Unicode Variation Selectors (VS), and embedding them into the cover text, resulting in the Stego-Text.

Pseudo code:

```
// --- ROUTINE 1: DETERMINISTIC SPATIAL MAPPING (PRNG) ---
Function Generate_Bit_Positions(Cover_Capacity, Payload_Bit_Length, Stego_Key)
Begin
    // Hash the Stego-Key to create a deterministic 32-bit PRNG seed
    Numeric_Seed ← Hash_To_32Bit_Integer(Stego_Key)
    // Initialize Mulberry32 PRNG and execute a partial Fisher-Yates shuffle
    PRNG_State ← Initialize_Mulberry32(Numeric_Seed)
    Target_Positions[] ← Execute_Fisher_Yates_Shuffle(Cover_Capacity, Payload_Bit_Length, PRNG_State)
    Return Target_Positions[]
End

// --- ROUTINE 2: PAYLOAD EMBEDDING (XOR & VS INJECTION) ---
Function Embed_Data_Into_Cover(Formatted_Payload_Bytes, Cover_Text, Stego_Key)
Input: Formatted_Payload_Bytes (Byte Array), Cover_Text (String), Stego_Key (String)
Output: Final_Stego_Object (String)
Begin
    Payload_Binary ← Convert_Bytes_To_Binary(Formatted_Payload_Bytes)
    Cover_Binary ← Convert_String_To_Binary(Cover_Text)
    // Retrieve deterministic mapping indices
    Target_Positions[] ← Call Generate_Bit_Positions(Length(Cover_Binary), Length(Payload_Binary), Stego_Key)
    // Execute Inverse Bit-Mapping (Generate the XOR Key)
    XOR_Key_Binary ← Empty String
    For i ← 0 To Length(Payload_Binary) - 1 Do
        If Cover_Binary[Target_Positions[i]] == Payload_Binary[i] Then
            Append '0' to XOR_Key_Binary // No bit-flip required
        Else
            Append '1' to XOR_Key_Binary // Bit-flip required to match payload
        EndIf
    EndFor
    // Map the binary XOR key into invisible Unicode Variation Selector (VS) characters
    Invisible_VS_String ← Encode_Binary_To_Variation_Selectors(XOR_Key_Binary)
    // Prepend the invisible key to the visible cover text
    Final_Stego_Object ← Invisible_VS_String || Cover_Text
    Return Final_Stego_Object
End

// --- ROUTINE 3: PAYLOAD EXTRACTION (VS DECODING & XOR RECOVERY) ---
Function Extract_Data_From_Cover(Stego_Object, Stego_Key)
Input: Stego_Object (String), Stego_Key (String)
Output: Recovered_Payload_Bytes (Byte Array), Clean_Cover_Text (String)
Begin
    // Isolate invisible VS characters from the human-readable text
    VS_Bytes, Clean_Cover_Text ← Isolate_And_Decode_Variation_Selectors(Stego_Object)
    XOR_Key_Binary ← Convert_Bytes_To_Binary(VS_Bytes)
    Cover_Binary ← Convert_String_To_Binary(Clean_Cover_Text)
    // Retrieve the exact same deterministic mapping indices used during embedding
    Target_Positions[] ← Call Generate_Bit_Positions(Length(Cover_Binary), Length(XOR_Key_Binary), Stego_Key)
    // Execute Bitwise XOR to recover the original payload bits
    Recovered_Payload_Binary ← Empty String
    For i ← 0 To Length(XOR_Key_Binary) - 1 Do
        If Cover_Binary[Target_Positions[i]] == XOR_Key_Binary[i] Then
            Append '0' to Recovered_Payload_Binary
        Else
            Append '1' to Recovered_Payload_Binary
        EndIf
    EndFor
    Recovered_Payload_Bytes ← Convert_Binary_To_Bytes(Recovered_Payload_Binary)
    Return Recovered_Payload_Bytes, Clean_Cover_Text
End
```

Figure 4.16 Pseudo code for Steganographic Engine

4.2.2 System Interfaces

Because the architecture is decentralized, and operates entirely locally, within the browser sandbox, it avoids traditional external networks API, relying instead on internal component interfaces:

Internal interfaces: The communication between the modules, UI, WASM module, En\Decryption engine, embed and extract engine, is managed via JS function calls. With no dependencies on external APIs, services, to help executing the process.

4.3 Security Controls

4.3.1 Preventive controls

Prevent unauthorized access to the secret text: the system requires two password fields as input from the user. The stego-key, that generates random positions, and the Key-Base, works as password to generate the crypto key using the PBKDF2. This composed system, requiring the two passwords to be correct, if one is false, then the full extraction process will fail.

4.4 Implementation details

4.4.1 Languages and frameworks

The system is built using a combination of static assets, with native & low-level libraries.

- **HTML5:** Used to construct, build the structure of the website. accessible, and structured Document Object Model (DOM) for both the sender and receiver interfaces.
- **CSS3:** Used to style the website. Optimized to provide a clean, responsive, modern User Interface (UI).
- **Vanilla JavaScript:** Works as the primary execution language. It manages the runtime and binds the stego-co\decomposer directly to the processing cores.
- **Native & low-level libraries:**
 - **Web Crypto API:** A cryptographic, secure, browser native API. used to execute high-performance AES in CTR mode. with PBKDF2 key derive function.
 - **WebAssembly (WASM) Brotli Core:** A low-level, compiled, binary module, embedded as a static Base64 string, executed on the client-side to handle stream compression, without requiring external server resources.

4.4.2 Testing

Unit testing & integration

To conduct mathematical verification of the STEGNOLINES architectural integrity and validate the correctness and reliability of the security mechanisms used in our text steganography solution, an advanced automated validation phase was undertaken. Considering the fully client-side nature of the implementation, the Jest testing framework was chosen for conducting rigorous unit and integration testing of the software's JavaScript codebase. It should be noted that the Jest framework distinguishes between two types of results: Suites and Tests. A Suite refers to a single JavaScript file containing results from a set of checks of one architectural component. Test refers to a particular condition asserted for that suite. The overall result showed a perfect 100% success rate, with 9 out of 9 Suites and 139 out of 139 Tests being successful without any failures or pending tasks. Therefore, the baseline stability of the entire steganographic pipeline is validated at this point.

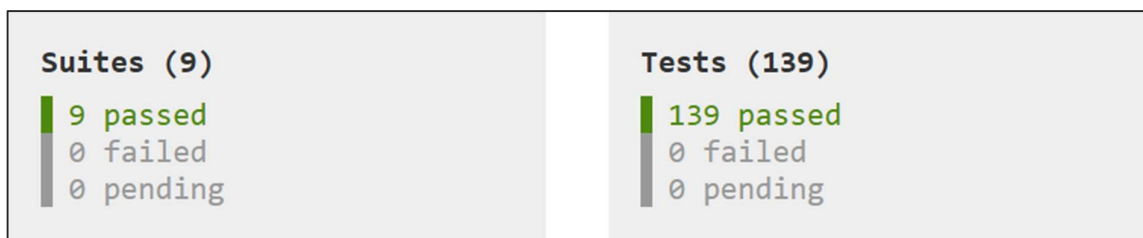


Figure 4.17 Suites & Tests Testing

XOR Mask Engine (xor-mask.test.js)

This test suite for the unit verifies the layer that performs bit level operations to mask and optimize the concealment of payloads. This test suite checks that `generateXORKey` and `recoverPayloadBits` perform their respective functions deterministically, guaranteeing that the data is restored perfectly regardless of whether the bits are placed sequentially or non-sequentially. The tests are able to prove the working of the boundary cases, including when the same bit streams generate all zeros keys or all one keys.

PRNG Position Generator (prng-generator.test.js)

This test suite checks the pseudo-random placement methods for hiding information within the text carrier. The mulberry32 algorithm is verified thoroughly for its ability to produce bounded, deterministic, and uniformly distributed sequences within the range of [0, 1) based on the specified seed. In addition, the test suite verifies that the system is capable of translating the text stego-key into a 32-bit unsigned integer seed, avoids duplicates (generatePositions), properly adjusts the bounds in case the requested number is more than the available one, and automatically generates keys from the cover text.

SHA-256 Hashing (sha256.test.js)

This set of tests confirms the cryptographic security and key generation layer of the program. These tests confirm that the hashing algorithm produces predictable results in the form of 64-character hexadecimal strings. More importantly, in order to meet the demands of globalizing, the test suite confirms the deterministic nature of the hashing of multibyte character sequences, including Arabic strings and emojis, and empty strings without leading to memory fault or null pointers.

VS Codec — Variation Selector Encoding (vs-codec.test.js)

This test suite aims at testing the core mechanism of information embedding into hidden Unicode characters. First of all, it verifies strict compliance with Unicode limits both Base Variation Selectors (U+FE00 to U+FE0F) and Supplementary Variation Selectors (U+E0100 to U+E01EF). This test suite verifies the proper byte-to-binary structure alignment, exact byte mapping for all possible values (256), multi-bytes extraction from the mixed stego-text, and finally, structure assembly of the hidden information payload.

Text Codec — Binary ↔ Text Conversion (text_codec.test.js)

This suite tests the bidirectional data serialization process from strings to binary arrays and vice versa. This process is being tested on string data under highly volatile conditions. This execution shows that there is no loss in the process of translating data between standard ASCII characters, multi-byte Arabic scripts, and graphic emojis. This test ensures that the trailing bits are handled securely and large arbitrary byte arrays are structurally aligned during full roundtrips.

Payload Codec — Build & Parse (payload-codec.test.js)

This suite tests the process of formatting, assembling and parsing of the last data packaging. This suite tests how the system forms the raw hidden message in comparison to structures that contain specialized recovery hints in a unique delimiter format. These tests have successfully proven that there is structural compliance for different payloads (Arabic text and emojis), correct error boundaries for empty messages and ensure safe extraction of the original message.

Brotli Compression Service (brotli-service.test.js)

In this suite, the optimization engine based on WebAssembly (WASM) is tested. It checks that Brotli compression yields appropriate ratios for very repetitive data sets and preserves byte accuracy during decompression. Moreover, it checks whether the decision process inside the system works well, whether the compression process is skipped if the data set is very short and whether the correct hexadecimal symbol of 0xFE is embedded.

Integration — Full Stego Pipeline (stego-pipeline.test.js)

This suite, acts as the main layer, for the integration testing of the framework

(Embed $\longleftrightarrow$ Extract). It shows successful embedding and extraction of the English and Arabic messages and proves that the appearance of the cover text is not changed after the structure injection. Most importantly, it confirms the reliability of the security mechanisms of the system, capacity exhaustion exceptions are raised appropriately and data extraction fails if the malicious user provides the wrong stego-key or password.

AES-256-CTR Encryption Engine (aes-ctr.test.js)

The tests in this suite verify the confidentiality layer in the framework, responsible for encrypting payloads before the steganographic embedding process. This is done by testing the PBKDF2 based extraction of 32-byte keys and 16-byte IVs from the user password and the cover text. Tests also reveal that the implementation of AES-256-CTR works without overhead (the output message has exactly the same length as the input message), correctly handles unencrypted bypass mode on empty passwords, and rejects decryption with invalid passwords or modified carriers.

4.5 Threat modeling

Threat	Impact	Reasoning	Mitigation	Residual Risks
Uncompressed Fallback Vulnerability	Freezing or crashing of the sender/receiver browser (Denial of Service) due to processing millions of hidden characters.	The presence of fallback code inside a catch block that reads the bytes of the original file as an uncompressed file in case AVIF compression fails.	The fallback mechanism was completely removed. If the compression process fails for any reason, the operation is immediately halted, the upload interface is reset, and the user is alerted to retry.	None: Any compression failure immediately stops the embedding process and completely rejects the file.
	Possibility of embedding huge executable files within texts.			
File Signature Bypass	Using the platform to hide and pass malicious software or unauthorized files (Payload Stego-hiding) via fake image extensions.	The old code relied solely on the apparent file name and the MIME type sent by the browser using the OR operator, without verifying the actual content.	Subjecting all programmatically received files to strict extension validation and Magic Bytes checking at the first entry point (handleImgFileSelect).	None: The browser does not process any file until it mandatorily passes the JavaScript checks.
HTML Filter Bypass (accept Attribute Bypass)	Directly injecting unsupported files into the JavaScript environment for processing.	The accept attribute in HTML tags does not support filtering when a user drag-and-drops a file, and it can also be bypassed by selecting "All Files" in the browser dialog.	Subjecting all programmatically received files—regardless of the input method—to strict extension validation at the first entry point (handleImgFileSelect).	None: The browser does not process any file until it mandatorily passes the JavaScript checks.
Object URL (Uniform Resource Locator) Memory Leak	Increased RAM consumption that may lead to browser slowdowns or complete crashes during extended use.	Failing to use URL.revokeObjectURL() to clean up temporary preview links generated during repeated image uploading	Created the clearPreviousImgUploadedFile function to immediately revoke previous preview links before generating new	Very Low: Memory is only allocated for the current active image and is fully released when the tab is

		and compression in a single session.	ones or when an error occurs.	closed or the page is refreshed.
DOM-Based Cross-Site Scripting (XSS) (Code Injection Vulnerability)	Executing malicious scripts in the user's browser. Stealing session data and LocalStorage.	Displaying the Hint text or Emoji field in the UI directly via the innerHTML property without any sanitization or encoding of special characters.	Using the escape Html (hintText) sanitization function to encode all hint texts before programmatically injecting them into the DOM.	Very Low: Limited only to the scenario where another developer adds new injection points in the future without utilizing the sanitization function.
Freezing the Tab's UI	Completely freeze the browser and suspend the page. Exhausting RAM capacity, leading to a tab crash.	The ability to upload a massive JSON file. Injecting thousands of structurally valid hints to generate thousands of repeated DOM elements. Injecting extremely long hint texts.	Enforcing a maximum upload file size limit of 50 KB. Setting a maximum display limit of 40 hints and truncating any excess. Imposing strict constraints on field lengths (Hint ≤ 60 characters, Timestamp ≤ 24 characters).	No Tangible Risks: The browser is fully protected against crashing due to massive data sizes or DOM element bloat within this tab.
Data Loss & Corruption	Completely disabling the hints tab due to read/write errors. Sudden, complete loss of the user's hint history.	Saving new modifications directly to the local JSON file or LocalStorage without programmatically verifying the current file's integrity, which	Wrapping all read, write, and parse operations inside try-catch blocks. Calling the validateHints function to verify full structural integrity before	External Factors: File corruption caused by factors outside the browser environment (e.g., sudden system shutdown by

		leads to overwriting an already corrupted file or writing a malformed structure.	allowing any new input to be saved.	the OS during a write operation).
Logic & Integrity Tampering	Injecting malformed values into the UI. Sabotaging the filtering and display logic of incoming/outgoing hints. Hiding crucial information from the user.	Lack of structural logic validation for fields during file reading. The code only required a valid JSON format to read and display data, even if sub-fields (like type) contained random values.	Upgrading validate Hints to strictly check each element, ensuring the type is exclusively 'sent' or 'received' with fixed lengths. Building a custom security alert (showHintsErrorModal) that accurately identifies the tampering type to prevent displaying tainted data and to protect system logic.	Manual Edits: The user can manually edit the file to change dates or texts, provided they still adhere to the programmatic constraints (accepted field formats and structures).
Accidental Data Deletion	Permanently erasing the entire hint history without the possibility of recovery due to a mistaken button click.	Relying on the browser's default confirm(...) dialog, which looks poor and is easily bypassed by mistake or triggered rapidly and unintentionally via the keyboard.	Replacing the default dialog with an attractive, custom confirmation modal featuring solid borders and a blurred background to prevent random clicks and require conscious user response. Linking the action button to a warning color (--color-error) to signify the operation's severity.	Intentional Deletion: Deletion deliberately executed by the user. The action cannot be undone once confirmed, but accidental triggering has been eliminated.

Table 4.1 threat modeling

4.6 System Documentation

in this section we will show a screenshot of the system, from the embedding phase to the extraction phase.

4.6.1 Embedding Workflow

This is the main page that the user enters to it when he accesses the application. This page, is responsible for the input handling, if there is an empty field, incorrect input, or anything that could make the process won't run.

Figure 4.18 Embedding Page

The input field in this page are: Cover Text, and Secret Message, and the Pre-Shared Key (Stego-Key). These three inputs are mandatory and the process will not start until those three fields are entered correctly.

After the user end from entering the inputs, he will press the Hide Data button shown in the left bottom of the picture, the result, which is the Stego-Text, is shown as bellow:

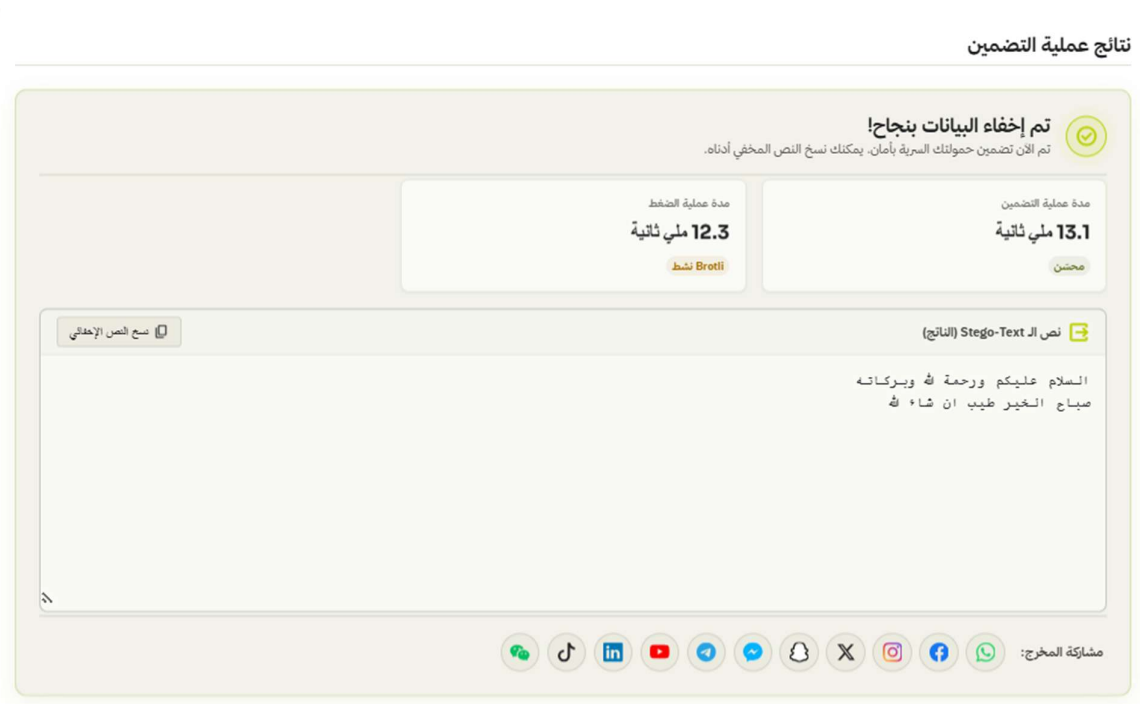


Figure 4.19 Embedding Result

This picture, shows the output of the embedding process, with the ability to choose which platform the user wants to use and send the Stego-Text.

Then, by sending the Stego-Text to the person or entity concerned.

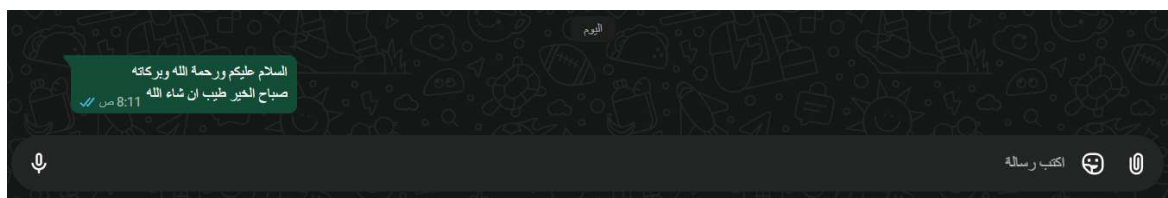


Figure 4.20 Sending the Stego-Text

4.6.2 Extraction Workflow

The receiver side copy the message from the chat,



Figure 4.21 Receiving The Stego-Text

and paste it in the Extraction Page

واجهة الاستخراج
كشف الرسالة السرية المخفية من النص الإخفاقي باستخدام المفاتيح والمعاملات الصحيحة.

استخراج قياسي مسح واستخراج تلميحات الاستخراج استخراج الصور

مراحل عملية الاستخراج

النص الإخفاقي  **الحامل**

السلام عليكم ورحمة الله وبركاته
صباح الخير طيب ان شاء الله

المفتاح المشترك مسبقاً  chars 8

Abcd4321

مفتاح التشفير **AES**

أدخل مفتاح خوارزمية تشفير AES-256-CTR...

استخراج البيانات 

Figure 4.22 Extracting Page

Then entering the Pre-Shared Stego-Key, and pressing the Extract Data button, Showing the Secret message as shows bellow:

نتائج عملية الاستخراج

 **تم استخراج السر بنجاح!**
تم استرداد الحمولة المخفية وفك تشفيرها بنجاح. رسالتك السرية تظهر أدناه.

مدة عملية فك الضغط 2.0 ملي ثانية  Brotli نشط	مدة عملية الاستخراج 102.3 ملي ثانية ناجح
---	--

الرسالة المستخرجة  **ناجح**

في ملف ببوصلك، راجعه، نشك ان في احد يختلس من اموال الشركة

Figure 4.23 Extraction Result

4.6.3 Technical Report for Technical People

When finishing the embedding phase, a report is shown for technical people who want to understand the logic behind the process.

The report shows three main analyses for the embedding process, first one is the Steganography Hex Stream, showing the final VS that embedded at the Cover Text.



Figure 4.24 Steganography Hex Stream

The second one, is the PRNG Random Base Positions.

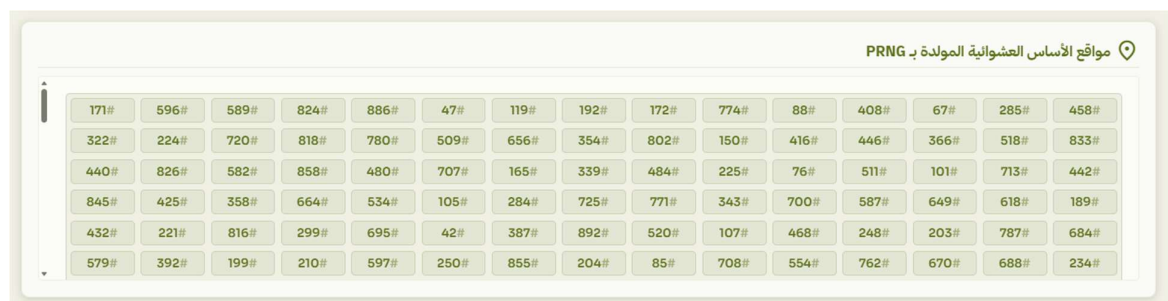


Figure 4.25 PRNG Random Base Positions

The third one is Binary XOR Key Difference Mask



Figure 4.26 Binary XOR Key Difference Mask

5. Results

5.1 Experimental setup recap

5.1.1 Baseline

This paragraph aims to determine the state of the cover text and the secret text before the hiding process, to measure the specific impact in the Test cases section and analyze the results in the Analysis section. To achieve this, several cases or reference points were determined for testing.

Note: The hiding and encryption key used in all reference cases is (123).

The first case is hiding a small message. The secret text is a link with a length of 53 characters and a size of (53 byte) and the cover text is written in Arabic with a length of (48) characters and a size of (88 byte)

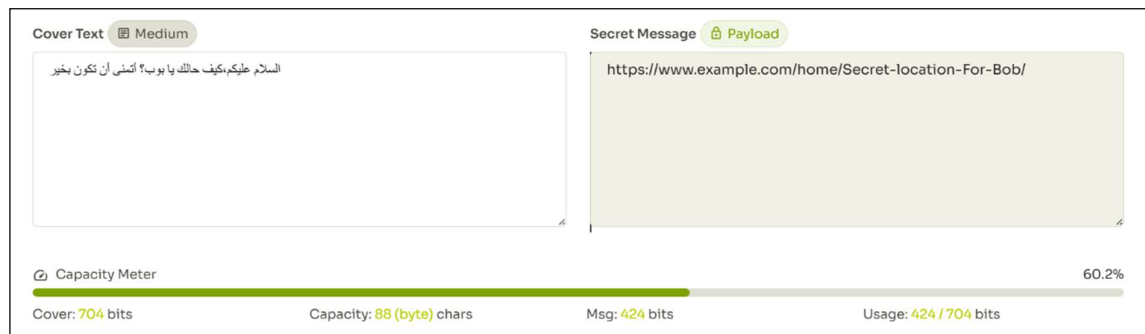


Figure 5.1 when cover text size is bigger than secret text size

The second case is hiding a large message (paragraph) written in English with a length of (707) characters and a size of (707 byte) in a large cover text written in English with a length of (803) characters and a size of (803 byte).

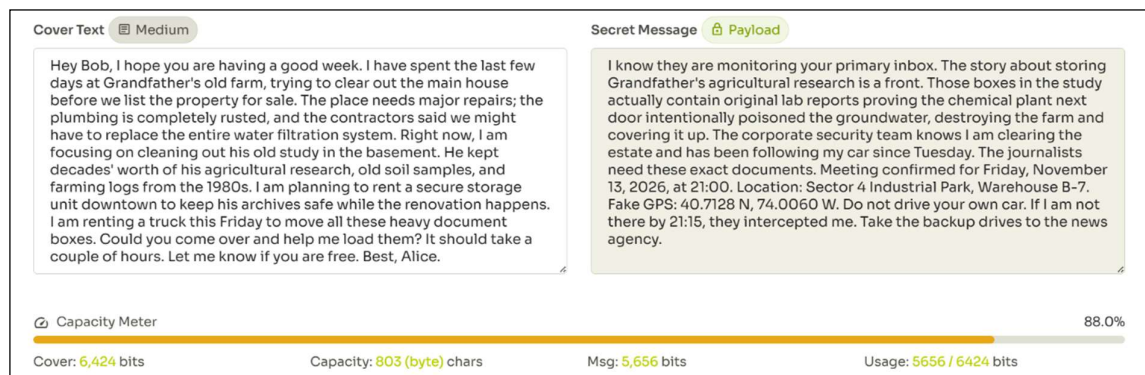


Figure 5.2 when cover text size is slightly bigger than secret text size

The third case: Hiding a small message equal to the size of the cover, written in Arabic with a length of (31) characters and a size of (57 bytes), in a small text cover written in English with a length of (57) characters and a size of (57 bytes)

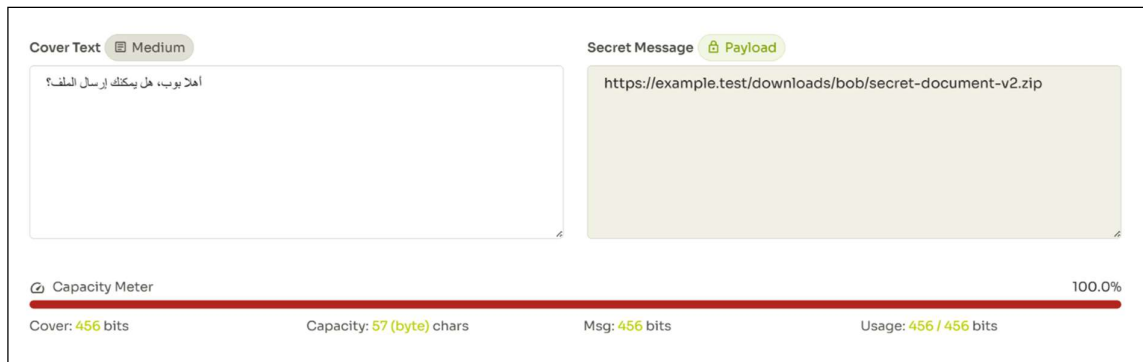


Figure 5.3 when both sizes are equals

The fourth case: Hiding a very large message written in English with a length of (141) characters and a size of (141 bytes), in a small text cover written in English with a length of (20) characters and a size of (20 bytes)

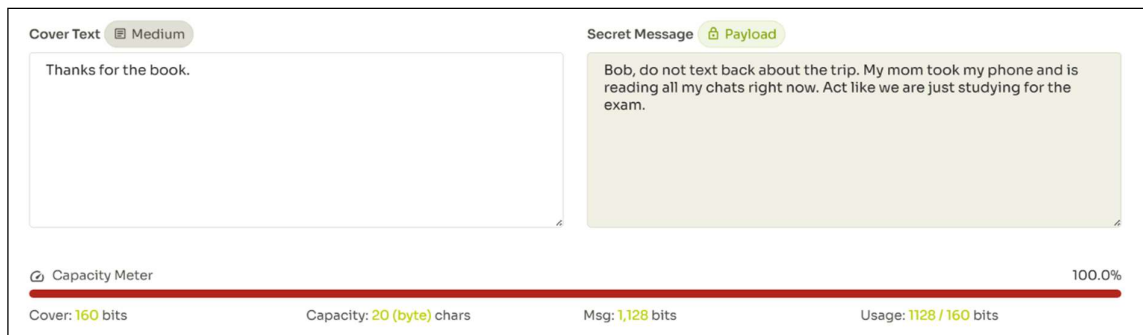


Figure 5.4 when secret text size is bigger than cover text size

5.1.2 Test cases

The system was evaluated through several systematic test cases to verify hiding efficiency. These tests branch out to cover the functional and security aspects of the system. All tests included in this section are based on several pre-defined reference cases (Baseline) to ensure the standardization of measurement criteria during the extraction of indicators and the documentation of the system's behavior under different usage scenarios.

Below is a detailed breakdown of all applied tests, including the execution mechanism and the metrics used for each case:

A. Capacity Test:

This test aims to measure the amount of data that the cover text can hold by applying the hiding process to all reference cases that covered all possible cases, and documenting all results in the form of a table consisting of seven columns (Tested Case, Cover Text Length, Secret text Length, Cover Text Size, Compressed Message Size, Capacity Ratio, Result). This table will display the capacity indicator along with the successful cases and the cases that reached the failure point, which is the point where it becomes impossible to hide any more bits.

$$\text{Capacity Ratio} = \left(\frac{\text{Secret text Size}}{\text{Cover Size}} \right) \times 100$$

B. Compression Test:

This test aims to measure the Space Savings ratio by testing the compression of the secret text (Payload) in all reference cases using the utilized algorithm (Brotli) during the hiding process. The ratio is calculated using the following mathematical equation:

$$\text{Space Savings Ratio} = \left(1 - \frac{\text{Compressed Size}}{\text{Uncompressed Size}} \right) \times 100\%$$

C. Stego-text Expansion Ratio:

This test aims to measure the percentage increase occurring on the cover text after the hiding process for all reference cases, by recording the size of the cover text in bytes before the hiding process, and the size of the resulting text (stego-text) after completing the embedding process and applying the following mathematical equation:

The lower this ratio, the more it indicates the efficiency of the algorithm in hiding data without causing inflation that may prevent sending the message to the specified platform or lead to exposing the hiding process due to the excessive amount.

$$\text{Size Expansion Ratio} = \left(\frac{\text{Stego-text Size} - \text{Cover Text Size}}{\text{Cover Text Size}} \right) \times 100\%$$

D. Robustness Test:

This test aims to evaluate the endurance of the (stego-text) against text processing and sanitization mechanisms (Text Sanitization) applied by input fields in social media platforms, which may lead to the deletion of invisible characters. The test is executed based on the pre-defined reference cases (Baseline), where the resulting text is sent across several specified communication platforms: (WhatsApp, Telegram, X/Twitter, Facebook, LinkedIn, Instagram, TikTok, YouTube, Snapchat)

For analysis purposes, the test outputs will be displayed within a table consisting of two columns (Platform Name, Result). The result determines the (success/failure) of the hiding for each platform.

E. Imperceptibility and Distortion Test:

This test aims to measure the extent of the visual and structural change occurring on the cover text after the completion of the embedding process, to ensure the absence of visible distortions that attract attention. The test is executed on the pre-defined reference cases (Baselines) by conducting a comparison between the cover text and the resulting text (stego-text). The results will be reviewed within a visual comparison table that includes screenshots of the texts before and after hiding within the previously specified communication platforms.

F. **Security Test:**

This test aims to evaluate the system's immunity against statistical detection attempts (Steganalysis). The test will be applied to a conversation consisting of 50 text messages with a size of 6KB, separately from the reference cases. The maximum-security stages will be applied by activating the encryption key along with using the fake cover (FC). The evaluation is carried out in two consecutive stages; the first stage aims to prove or suspect the presence of a hidden message in the conversation by analyzing the entire conversation to discover any hidden symbols. The detection time will be measured to determine the difficulty of detection, while the second stage aims to calculate the amount of time the process will take to extract the message. This is to mathematically simulate statistical attacks (Statistical Attacks), as one operation will be calculated and measured against a million trials to extract the Estimated Cracking Time.

$$\text{Total Seconds} = \text{Number of Trials} \times \left(\frac{\text{Time in milliseconds}}{1000} \right)$$

5.2 Findings

This paragraph reviews the results of the tests applied to the baselines without analyzing and interpreting them. The results are divided into two main cases:

5.2.1 Quantitative results

In this section, the test results extracted from the stego-system are reviewed, which provide numerical metrics enabling comparison and analysis. The results of the capacity, compression, stego-text size, and security tests are presented in tables to methodically evaluate the embedding algorithm.

Capacity Test:

Tested Case	Cover Text Length	Secret text Length	Cover Text Size	Compressed Message Size	Capacity Ratio	Result
Case 1	48 characters	53 characters	88 bytes	43 bytes	48.9%	Success
Case 2	803 characters	707 characters	803 bytes	314 bytes	39.1%	Success
Case 3	31 characters	31 characters	57 bytes	57 bytes	100%	Success
Case 4	20 characters	141 characters	20 bytes	141 bytes	705%	Failure

Table 5.1 Capacity test results

Compression Test:

Tested Case	Uncompressed Size	Compressed Size	Space Savings
Case 1	53 bytes	43 bytes	18.8%
Case 2	707 bytes	314 bytes	55.5%
Case 3	57 bytes	57 bytes	0%

Table 5.2 Compression test results to evaluate the efficiency of the Brotli algorithm

Stego-text Expansion Ratio:

Tested Case	Cover Text Size	Secret text Size	Stego-text Size	Size Expansion Ratio
Case 1	88 bytes	53 bytes	255 bytes	189.77%
Case 2	803 bytes	707 bytes	2040 bytes	154.05%
Case 3	57 bytes	57 bytes	280 bytes	391.23%

Table 5.3 Stego-text Expansion Ratio test results

Security Test:

Description	Details
Sample	Arabic language conversation, 50 messages in length, 6KB in size
Hidden symbols detection time	3.0 MS
Message extraction time	1510 MS
Number of simulation attempts	1,000,000 attempts

Table 5.4 Security Test Parameters

To simulate Statistical Attacks, the Estimated Cracking Time was calculated over one million trials based on the following equation:

$$\text{Total Seconds} = 1,000,000 \times \left(\frac{1510}{1000} \right) = 1,510,000$$

Days (D)	Hours (H)	Minutes (M)	Seconds (S)
17 Days	11 Hours	26 Minutes	40 Seconds

Table 5.5 Estimated Cracking Time Breakdown

5.2.2 Qualitative insights

Robustness Test:

Digital Platform	Procedural Result
WhatsApp	Success
Facebook	Success
Instagram	Success
X / Twitter	Success
Telegram	Success
TikTok	Success
YouTube (Comments)	Success
LinkedIn	Success
Snapchat	Success

Table 5.6 Estimated Cracking Time Breakdown

Imperceptibility and Distortion Test:

Digital Platform	Tested Case	Image Before Embedding	Image After Embedding
WhatsApp	1		
Facebook	2		
Instagram	1		
X / Twitter	1		
Telegram	2		
TikTok	1		
YouTube (Comments)	1		
LinkedIn	1		
Snapchat	1		

Table 5.7 Estimated Cracking Time Breakdown

6. Analysis & Discussion

6.1 Analysis

This section aims to provide an analysis and interpretation of the quantitative and qualitative outputs for all the tests observed in the "Findings" section.

6.1.1 Effectiveness

The overall results prove the system's success in maintaining 100% Data Integrity across all identified platforms without the loss or corruption of any characters, along with an excellent space-saving ratio and achieving complete visual stealth thanks to the unique characteristics of Variation Selectors, as demonstrated by the following tests:

Capacity Test

The capacity test, as shown in Table No 5.1, revealed a direct relationship between the size of the cover text (Cover Text) and the secret text.

Successful cases (Cases 1, 2, 3): The system succeeded in passing the secret texts even when the lengths of the cover and the payload were equal, as in Case 3, with a capacity ratio of (100%). This reflects the flexibility of the algorithm and its ability to excellently exploit the space, significantly outperforming the studies presented by Malik [4], which achieved a capacity of only 13.43%, and the model proposed by Priyansh Sharma [18], whose capacity did not exceed 16% of the total cover size.

Failure point (Failure Point - Case 4): The fourth case represents the critical capacity limit of the system; where the required capacity ratio reached 705% (a 141-character secret text against a cover text of only 20 characters). The reason for the failure here is due to running out of the available "embedding capacity" in the very short cover, which means that it does not allow the message to exceed the length of the cover.

Compression Test

The performance of the (Brotli) compression algorithm demonstrates varying efficiency that is directly related to the size of the input payload (please refer to Table No 5.2

In large texts (Case 2), the algorithm achieved an excellent space-saving ratio of 55.5% (where the payload was reduced from 707 bytes to 314 bytes). This is due to Brotli's high capability to build custom dictionaries and exploit linguistic repetitions in long texts.

Conversely, efficiency decreased in short texts (Case 1) to 18.8% and vanished completely in (Case 3). Technically, this behavior indicates that when the payload is very small, the compression process is canceled due to the overhead resulting from the dictionaries, which means that the compression process is unfeasible in small texts.

This result presented by the Brotli algorithm outperforms studies that relied on older compression algorithms; such as the (Huffman) algorithm used in Kwan's study [21], and the (LZW) algorithm in the study by Ahvanooey et al. [32]. The proposed system also provided an absolute advantage over studies that fundamentally lack compression mechanisms, such as Butler [17] and Sharma [18].

stego-text Expansion Ratio Test

The Size Expansion Ratio recorded in Table No 5.3 reveals the actual cost of visual concealment: Case 3 recorded the highest expansion ratio at 391.23%, and this is due to the payload not being affected by compression. This ratio decreases in Case 2 to 154.05% thanks to the role of compression, which reduced the payload size before concealment. The technical explanation for this expansion lies in the fact that Variation Selectors consume 2 to 4 bytes in (UTF-8) encoding for each hidden symbol, which leads to the doubling of the final Stego-Text size in bytes. However, most social media platforms rely on the maximum number of symbols instead of the actual size in bytes. This means that this expansion in size will not hinder the passing of texts within the allowed limits for inputs.

Based on the above, this proposed mechanism practically outperformed the challenges faced by previous studies; as the (ZWC) symbols used in the studies of Sharma [18] and Ahvanooey et al. [32] cause a significant expansion in the number and size of symbols, since hiding a single byte requires 4 symbols, and each of these symbols consumes 3 bytes in (UTF-8) encoding. In contrast, the concealment mechanisms of Naharuddin et al. [22] and Shivani et al. [23] suffer from external expansion; because the size of the "key" they proposed to send alongside the secret text is much larger than the message itself. Consequently, the proposed system circumvents these technical flaws.

Security Test

The results of the security test, as shown in Tables No 5.4 and 5.5, indicate that the system achieved an excellent level of resistance to Statistical Attacks during the extraction attempt. However, despite all the advantages offered by (Variation Selectors), they are easy to inspect. The test revealed that the duration of inspecting for the presence of hidden symbols in the conversation took about 3.0 milliseconds, which is a low time reflecting the ease of conducting an initial inspection. As for the extraction phase, the message extraction time was estimated at approximately 17 days, which is an indicator of a temporal obstacle facing the attacker, taking into consideration that the test simulation was limited to one million trials, which is considered a simplified assumption. If we were to assume that the conversation length exceeds fifty messages like natural conversations, or in the event of multiple hidden messages scattered within a fake cover, the computational complexities would multiply tremendously and require a massive depletion of the attacker's resources.

These cumulative factors, coupled with the need for billions of attempts to guess the hiding key and the encryption key, indicate that the extraction process is not an actual threat; since the probability of the attacker succeeding in extracting the data is slim and does not justify the effort and time expended. These results gain their importance when considering the shortcomings suffered by previous studies in the security aspect; as few of them conducted security testing. On the one hand, the studies of Butler [17] and Ahmad et al. [30] did not focus on addressing any security factors, while Malik's [28] methodology did not include any data encryption layer. Even the studies that employed encryption relied on old algorithms that are easy to break; such as the (RC4) algorithm in Sharma's study [18], and the (ICE) algorithm in Kwan's study [21], which confirms the superiority of the proposed system in providing multi-layered protection.

Robustness Test

The system demonstrated excellent robustness and stability, as mentioned in Table 5.5. The system passed transmission tests across social media platforms without these platforms corrupting data integrity or filtering out the Variation Selectors. In contrast, some previous studies suffered damage during transmission on social media platforms due to their reliance either on a (Rich Text Environment) such as Kumar et al. [28], who relied on an encoding table of fonts, and Malik [22], who relied on color coding, and similarly, techniques that relied on white spaces, as in Matthew's tool [21], are filtered out by social media platforms.

Imperceptibility and Distortion Test

This test proved the efficiency of the system from a visual perspective; as the screenshots in Table No 5.6 demonstrated that the texts completely retained their natural appearance, without any distortion or flaw to the stego-text across all tested social media platforms, by comparing the appearance of the cover text before and after embedding. This success is due to the nature of Variation Selectors; when these symbols are not paired with a symbol that interacts with them, they are treated as invisible instructions that do not possess any graphical representation (Glyphs). This behavior ensures achieving the goal of "Human Imperceptibility" by 100%, which obscures any visual suspicion.

This result overcomes the distortion problems that appeared in previous studies; as the symbols used in the study by Majid et al. [19] (such as Left-To-Right Embedding) caused the text to invert and distort when using the Arabic language as a cover. This proves the ability of the proposed system to hide data while completely maintaining the appearance of the original text without raising any visual suspicions.

6.2 Validation

This section aims to evaluate the efficiency of the proposed system by conducting a comparative analysis (Comparative Analysis) that places its output against the results of a group of previous applications and techniques in the same field. This comparison will be presented in the form of a table to facilitate the comprehension of technical differences and to highlight the added value provided by this research.

Reference	Capacity	Imperceptibility	Robustness	Security
Butler [15]	Open (may exceed platform limits, causing message loss)	No distortion	Unaffected during transmission	Easy to detect and extract (No encryption)
Sharma [16]	16% of cover size	No distortion	Unaffected during transmission	Encrypted (but uses an outdated algorithm)
Ahvanooey et al. [18]	Limited by platform input constraints	Distortion in Arabic (due to LRO character)	Unaffected during transmission	Strong
Malik [26]	13.43% of cover size	No distortion	Affected and fails outside Rich Text Environment	Uses a stego-key (No encryption)

Reference	Capacity	Imperceptibility	Robustness	Security
Majid et al. [21]	18.4% of cover size	No distortion	Unaffected during transmission	Encrypted (but uses an outdated algorithm)
Proposed System	100% cover size, subject to maximum character limits	No distortion	Unaffected	Resists extraction (Encrypted)

Comparison of the Proposed System's Performance with Previous Solutions

The results demonstrated the superiority of the proposed system, surpassing the limitations that previous applications suffered from. The system succeeded in achieving an excellent balance between capacity efficiency and encryption strength, with complete preservation of the natural appearance of the cover text without any distortion. Based on these findings, this research presented an exceptional model committed to the fundamental data hiding standards, making it a suitable tool for securing information exchange across communication platforms.

7. Conclusion

The goal of this research was to develop a compatible and secure data hiding method, for transmission across modern social media platforms, in addition to maximizing the capacity of the cover text. To achieve this objective, the paper proposed a hybrid, structural, text steganography system. This system integrates Brotli compression, AES-CTR encryption, and a bit-mapping technique utilizing Unicode VS. By generating pseudorandom positions and performing an XOR operation between the cover text and the secret text, producing a sequence of bits, that converted into VS, then embedded at the beginning of the cover text. The results show a high-capacity efficiency, with the possibility to achieve a 100% capacity ratio, without visually affecting the cover text. Also, a set of tests across multiple social media platforms, confirmed that the Stego-Text can be sent and received without triggering platform restrictions, or visual suspicion.

The system does provide a practical implication, for real-world deployment, the system operates entirely on the client side, within the user's web browser. Using WebAssembly, and WEB CRYPTO API. This decentralized architecture ensures zero backend infrastructure costs, eliminating the need to a high technical knowledge from users, guarantees a high operational fit across both mobile and desktops. Most of all, a massive security enhancement is implemented by keeping sensitive data, and cryptographic keys only in the browser's sandbox. Mitigating the risks of Man in The Middle (MITM) attacks.

The proposed system does have limitations. The critical one, when modifying the cover text, if a user or intermediary platform adds, deletes even a single character or space, the extraction process fails. This occurs because the extraction relies on the exact original cover text positions to regenerate the correct positions again. Another limitation is that the Stego-Text is vulnerable to strict text sanitization process, that implemented by certain platforms, that might strip out the non-visible Unicode characters. Leading to the loss of secret data. In addition to the stego-text size expansion caused by the UTF-8 encoding of VS.

For future work, it'll focus on enhancing the robustness of the system, specifically the critical limitation related to the cover text altering. Ensuring that the extraction process successfully retrieves hidden data.

8. References

- [1] BBC News, "Facebook security breach: Up to 50 million accounts affected," BBC News, 28 september 2018. [Online]. Available: <https://www.bbc.com/news/technology-45686890>.
- [2] CNN Business, "WhatsApp says a security flaw allowed hackers to install spyware on phones," CNN, 14 May 2019. [Online]. Available: <https://edition.cnn.com/2019/05/14/tech/whatsapp-attack/index.html>. [Accessed 07 March 2026].
- [3] Chris Perez, "Snapchat employees abused data access, spied on users: report," Fox News, 27 May 2019. [Online]. Available: <https://www.foxnews.com/tech/snapchat-employees-abused-data-access-spied-on-users-report>.
- [4] A. M. G. S. and H. K. Verma, "A high capacity text steganography scheme based on LZW compression and color coding," *Engineering Science and Technology, an International Journal*, p. 72–79, 2018.
- [5] N. F. Johnson and S. Jajodia, "Exploring steganography: Seeing the unseen," *IEEE Computer*, pp. 26 - 34, 28 February 1998.
- [6] M. Kaur, V. Kumar and D. Singh, "An efficient image steganography method using multiobjective differential evolution," in *Digital Media Steganography: Principles, Algorithms, and Advances*, Cambridge, MA, Academic Press, 2020, pp. 65-85.
- [7] MahmoudHassaballah, Digital Media Steganography, Qena, Egypt: Stacy Masucci, 2020, pp. 32 - 44.
- [8] A. Pradhan, A. K. Sahu, G. Swain and K. R. Sekhar, "Performance Evaluation Parameters of Image Steganography Techniques," in *International Conference on Research Advances in Integrated Navigation Systems (RAINS - 2016)*, Bangalore, 2016.
- [9] M. A. Majeed, R. Sulaiman, Z. Shukur and M. K. Hasan, "A Review on Text Steganography Techniques," *Mathematics*, vol. 9, no. 21, p. 2829, 2021.
- [10] M. Hassaballah, M. Abdel Hameed and M. H. Alkinani, in *Digital Media Steganography: Principles, Algorithms, and Advances*, cambridge, MA, Academic Press, 2020, pp. 1-17.
- [11] M. T. Ahvanooy, Q. Li, J. Hou, A. R. Rajput and Y. Chen, "Modern Text Hiding, Text Steganalysis, and Applications: A Comparative Analysis," *Entropy*, vol. 21, no. 4, p. 329, 2019.
- [12] T.-C. Lu and T. N. Vo, "Reversible steganography techniques: A survey," in *Digital Media Steganography: Principles, Algorithms, and Advances*, MA, Academic Press, 2020, pp. 189-213.
- [13] M. Hassaballah, M. Abdel Hameed and M. H. Alkinani, "Introduction to digital image steganography," in *Digital Media Steganography: Principles, Algorithms, and Advances*, Cambridge, MA, Academic Press, 2020, pp. 1-17.
- [14] V. Haridas, "Steganography-A Data Hiding Technique," *International Journal of Engineering Research & Technology (IJERT)*, p. 1, 30 July 2018.

- [15] Q. L. H. J. S. Y. H. Milad Taleby Ahvanooy, "A Comparative Analysis of Information Hiding Techniques for Copyright Protection of Text Documents," *Security and Communication Networks*, p. 5, 17 April 2018.
- [16] T. A. Z. I. N. H. A. M. A. M. J. B. Yazeed Yasin Ghadi1, "Enhanced payload volume in the least significant bits image steganography using hash function," *PeerJ Computer Science*, p. e1606, 13 November 2023.
- [17] P. Butler, "Emoji Encoder," 26 يناير 2025. [Online]. Available: <https://github.com/paulgb/emoji-encoder>. [Accessed 1 أبريل 2026].
- [18] P. Sharma, "Steganography-Tools," GitHub, 2023. [Online]. Available: <https://github.com/priyansh-15/steganography-tools>. [Accessed 02 February 2026].
- [19] M. A. Majeed, R. Sulaiman and Z. Shukur, "New Text Steganography Technique based on Multilayer Encoding with Format-Preserving Encryption and Huffman Coding," *International Journal of Advanced Computer Science and Applications*, vol. 13, no. 12, pp. 163-172, 2022.
- [20] Q. L. J. H. H. D. M. a. J. Z. M. Taleby Ahvanooy, "AITSteg: An Innovative Text Steganography Technique for Hidden Transmission of Text Message via Social Medi," 20 august 2018. [Online]. Available: https://www.researchgate.net/profile/Milad-Taleby-Ahvanooy-2/publication/327034906_AITSteg_An_Innovative_Text_Steganography_Technique_for_Hidden_Transmission_of_Text_Message_via_Social_Media/links/5bfea54245851523d151b8f5/AITSteg-An-Innovative-Text-Stega. [Accessed 19 february 2026].
- [21] M. Kwan, "SNOW – whitespace steganography program," 16 June 2013. [Online]. Available: <https://darkside.com.au/snow/>. [Accessed 07 02 2026].
- [22] G. S. H. K. V. Aruna Malik, "A high capacity text steganography scheme based on LZW compression and color coding," *Engineering Science and Technology, an International Journal*, pp. 72-79, Jan 2018.
- [23] Shivani, V. K. Yadav and S. Batham, "A Novel Approach of Bulk Data Hiding using Text Steganography," *Procedia Computer Science*, vol. 57, no. (Leave blank—Procedia volumes generally don't use issue numbers), pp. 1401-1410, 2015.
- [24] V. Chu, "ASCII Art Steganography," 2009. [Online]. Available: <https://pictureworthsthousandwords.appspot.com/>. [Accessed 3 February 2026].
- [25] R. Gurunath and D. Samanta, "A New 3-Bit Hiding Covert Channel Algorithm for Public Data and Medical Data Security Using Format Based Text Steganography," *ournal of Database Management (JDM)*, pp. vol. 34, no. 2, pp. 1-17, June 2023.
- [26] D. A. K. D. A. H. Syed Tahir Ali Shah, "Text Steganography Using character Spacing after Normalization," *International Journal of Scientific & Engineering Research*, pp. 949-957, February 2020.
- [27] B. Khosravi, B. Khosravi, B. Khosravi and K. Nazarkardeh, "A new method for pdf steganography in justified texts.," *Journal of Information Security and Applications*, pp. Vol 45, Pages 61-70, 3 January 2019.
- [28] R. Kumar, A. Malik, S. Singh, B. Kumar and S. Chand, "A Space based reversible high capacity text steganography scheme using Font type and style," *International Conference on Computing, Communication and Automation (ICCCA)*, pp. 1090 - 1094, 29-30 April 2016.
- [29] S. Roy and M. Manasmita, "A Novel Approach to Format Based Text Steganography," in *Proceedings of the 2011 International Conference on Communication, Computing & Security (ICCCS '11)*, Rourkela, Odisha, 2011.

- [30] G. S. H. S. W. W. R. M. I. Tohari Ahmad, "Emoticon-based Steganography for Securing Sensitive Data," *International Conference on Information Technology and Electrical Engineering (ICITEE)*, pp. 79-84, 7 October 2014.
- [31] M. Fateh and M. Rezvani, "An email-based high capacity text steganography using repeating characters," *International Journal of Computers and Applications*, vol. 43, no. 3, pp. 226-232, 2021 (Note: While it appeared online in 2018, it was officially allocated to the 2021 print volume).
- [32] M. T. Ahvanooy, Q. Li, J. Hou, A. R. Rajput and Y. Chen, "Modern Text Hiding, Text Steganalysis, and Applications: A Comparative Analysis," *Entropy*, vol. 21, no. 4, p. 329, 2019.
- [33] AnuIyer, "Image Steganography in Cryptography," geeksforgeeks.org, 22 Aug 2023. [Online]. Available: <https://www.geeksforgeeks.org/computer-networks/image-steganography-in-cryptography/>. [Accessed 25 January 2026].
- [34] K. O'Riordan, "ASCII art," 4 December 2025. [Online]. Available: <https://www.britannica.com/topic/ASCII-art>. [Accessed 25 January 2026].
- [35] S. Chaudhary, M. Dave and A. Sanghi, "Aggrandize text security and hiding data through text steganography.," *IEEE*, pp. 1-5, 25-27 November 2016.
- [36] S. R. a. M. Manasmita., "(PDF) A novel approach to format based text steganography," 12-14 februray 2011. [Online]. Available: https://www.researchgate.net/publication/220846511_A_novel_approach_to_format_based_text_steganography.
- [37] M. Hassaballah and A. I. Awad, *Digital Media Steganography: Principles, Algorithms, and Advances*, Cambridge, MA: Academic Press, 2020.
- [38] M. Kaur, V. Kumar and D. Singh, "An efficient image steganography method using multiobjective differential evolution," in *Digital Media Steganography: Principles, Algorithms, and Advances*, Cambridge, MA, Academic Press, 2020, pp. 65-85.

9. Appendices

9.1 Source code excerpts

<https://github.com/AkazaUI/Stegnolines>

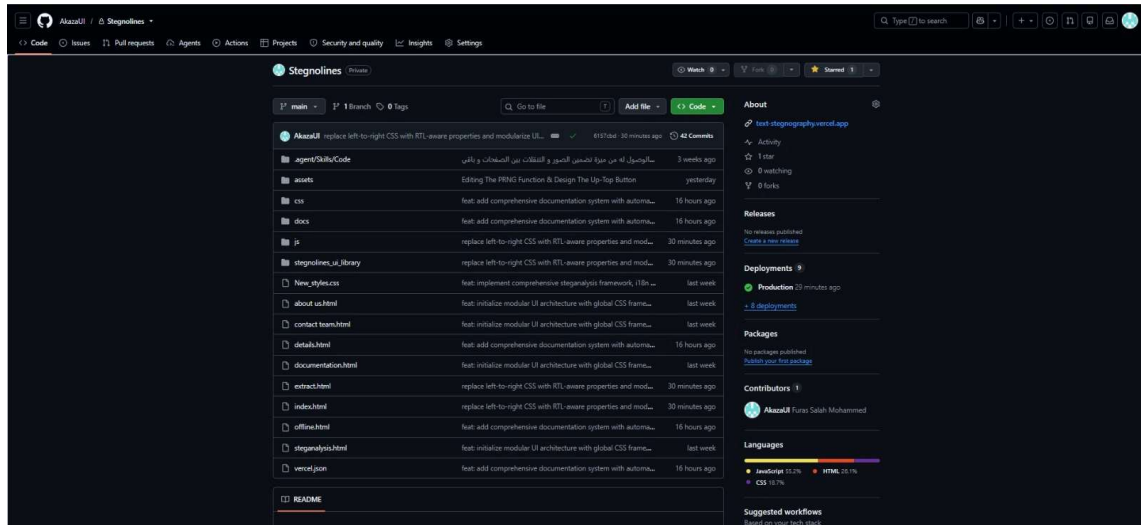


Figure 0.1 repo

Unit & Integration Tests

The automated test suite was executed using the Jest testing framework to validate the correctness and reliability of all core system modules. A total of 9 test suites comprising 139 individual test cases were executed, and all tests passed successfully with zero failures.

XOR Masking Tests

v E:\Project App\Embding-Algo\tests\unit\stego\xor-mask.test.js			0.498s
XOR Mask Engine > generateXORKey	should return all 0s when cover and message bits match	passed	0.011s
XOR Mask Engine > generateXORKey	should return all 1s when cover and message bits are inverted	passed	0.001s
XOR Mask Engine > generateXORKey	should handle non-sequential positions	passed	0.003s
XOR Mask Engine > generateXORKey	should generate key of same length as message bits	passed	0.001s
XOR Mask Engine > recoverPayloadBits	should recover original message bits when key is all 0s	passed	0.001s
XOR Mask Engine > recoverPayloadBits	should recover inverted bits when key is all 1s	passed	0.001s
XOR Mask Engine > Roundtrip: generateXORKey → recoverPayloadBits	should perfectly recover the original message bits	passed	0.001s

XOR Mask Engine > Roundtrip: generateXORKey → recoverPayloadBits	should roundtrip with random-like data	passed	0.001s
XOR Mask Engine > Roundtrip: generateXORKey → recoverPayloadBits	should roundtrip with single bit	passed	0.001s

Figure 0.2 XOR Masking Tests

PRNG Position Generator Tests

v E:\Project App\Embdng-Algo\tests\unit\stego\prng-generator.test.js			0.576s
PRNG Position Generator > stegoKeyToSeed	should return a number for any string	passed	0.01s
PRNG Position Generator > stegoKeyToSeed	should return same seed for same input (deterministic)	passed	0.001s
PRNG Position Generator > stegoKeyToSeed	should return different seeds for different inputs	passed	0.001s
PRNG Position Generator > stegoKeyToSeed	should return an unsigned 32-bit integer	passed	0.005s
PRNG Position Generator > stegoKeyToSeed	should handle empty string	passed	0.001s
PRNG Position Generator > mulberry32	should return a function	passed	0.001s
PRNG Position Generator > mulberry32	should produce values in [0, 1)	passed	0.02s
PRNG Position Generator > mulberry32	should produce deterministic sequence from same seed	passed	0.012s
PRNG Position Generator > mulberry32	should produce different sequences for different seeds	passed	0.001s
PRNG Position Generator > generatePositions	should return correct number of positions	passed	0s
PRNG Position Generator > generatePositions	should return unique positions (no duplicates)	passed	0.001s
PRNG Position Generator > generatePositions	should return positions within valid range [0, maxLength-1]	passed	0.003s
PRNG Position Generator > generatePositions	should be deterministic for same key	passed	0.003s
PRNG Position Generator > generatePositions	should produce different positions for different keys	passed	0.001s

PRNG Position Generator > generatePositions	should handle count = maxLength (select all positions)	passed	0.001s
PRNG Position Generator > generatePositions	should handle count > maxLength (clamp to maxLength)	passed	0.001s
PRNG Position Generator > generatePositions	should handle count = 0	passed	0.001s
PRNG Position Generator > resolveStegoKey	should use user-provided key when not empty	passed	0.001s
PRNG Position Generator > resolveStegoKey	should auto-generate key from cover text when key is empty	passed	0.007s
PRNG Position Generator > resolveStegoKey	should auto-generate key when key is only whitespace	passed	0.001s
PRNG Position Generator > resolveStegoKey	should produce deterministic auto-key for same cover text	passed	0.002s

Figure 0.3 PRNG Position Generator Tests

SHA-256 Hashing Tests

v E:\Project App\Embding-Algo\tests\unit\crypto\sha256.test.js			0.581s
SHA-256 Hashing	should hash empty string correctly	passed	0.017s
SHA-256 Hashing	should hash "hello" correctly	passed	0.001s
SHA-256 Hashing	should hash "Hello World" correctly	passed	0.001s
SHA-256 Hashing	should produce the same hash for the same input (deterministic)	passed	0.002s
SHA-256 Hashing	should always return a 64-character hex string	passed	0.002s
SHA-256 Hashing	should produce different hashes for different inputs	passed	0.001s
SHA-256 Hashing	should handle Arabic text correctly	passed	0.002s
SHA-256 Hashing	should handle emoji correctly	passed	0.002s

Figure 0.4 SHA-256 Hashing Tests

VS Codec (Variation Selector Encoding) Tests

v E:\Project App\Embding-Algo\tests\unit\stego\vs-codec.test.js			0.663s
VS Codec — Variation Selector Encoding > Constants	VS_BASE_START should be U+FE00	passed	0.01s
VS Codec — Variation Selector Encoding > Constants	VS_BASE_END should be U+FE0F	passed	0.001s
VS Codec — Variation Selector Encoding > Constants	VS_SUPPLEMENT_START should be U+E0100	passed	0.005s
VS Codec — Variation Selector Encoding > Constants	VS_SUPPLEMENT_END should be U+E01EF	passed	0.001s
VS Codec — Variation Selector Encoding > isBaseVariationSelector	should return true for VS1 (U+FE00)	passed	0.001s
VS Codec — Variation Selector Encoding > isBaseVariationSelector	should return true for VS16 (U+FE0F)	passed	0.001s
VS Codec — Variation Selector Encoding > isBaseVariationSelector	should return false for character outside range	passed	0.001s
VS Codec — Variation Selector Encoding > isSupplementaryVariationSelector	should return true for VS17 (U+E0100)	passed	0.001s
VS Codec — Variation Selector Encoding > isSupplementaryVariationSelector	should return true for VS256 (U+E01EF)	passed	0.001s
VS Codec — Variation Selector Encoding > isSupplementaryVariationSelector	should return false for character outside range	passed	0.001s
VS Codec — Variation Selector Encoding > toVariationSelector & fromVariationSelector	should map byte 0 to VS1 (U+FE00)	passed	0.001s
VS Codec — Variation Selector Encoding > toVariationSelector & fromVariationSelector	should map byte 15 to VS16 (U+FE0F)	passed	0.001s
VS Codec — Variation Selector Encoding > toVariationSelector & fromVariationSelector	should map byte 16 to VS17 (U+E0100)	passed	0.001s
VS Codec — Variation Selector Encoding > toVariationSelector & fromVariationSelector	should map byte 255 to VS256 (U+E01EF)	passed	0s

VS Codec — Variation Selector Encoding > toVariationSelector & fromVariationSelector	<i>should return null for out-of-range byte (256)</i>	passed	0s
VS Codec — Variation Selector Encoding > toVariationSelector & fromVariationSelector	<i>should return null for negative byte (-1)</i>	passed	0s
VS Codec — Variation Selector Encoding > toVariationSelector & fromVariationSelector	<i>should roundtrip all 256 byte values</i>	passed	0.071s
VS Codec — Variation Selector Encoding > padBinaryToByteAlignment	<i>should pad 5 bits to 8 bits</i>	passed	0.001s
VS Codec — Variation Selector Encoding > padBinaryToByteAlignment	<i>should not pad already byte-aligned string</i>	passed	0.003s
VS Codec — Variation Selector Encoding > padBinaryToByteAlignment	<i>should pad 1 bit to 8 bits</i>	passed	0s
VS Codec — Variation Selector Encoding > padBinaryToByteAlignment	<i>should handle empty string</i>	passed	0.001s
VS Codec — Variation Selector Encoding > xorKeyToVSString	<i>should convert 8-bit binary key to single VS character</i>	passed	0.005s
VS Codec — Variation Selector Encoding > xorKeyToVSString	<i>should convert 16-bit binary key to two VS characters</i>	passed	0.001s
VS Codec — Variation Selector Encoding > xorKeyToVSString	<i>should handle non-byte-aligned key with padding</i>	passed	0.001s
VS Codec — Variation Selector Encoding > extractVSFromText	<i>should extract VS bytes from stego text</i>	passed	0.011s
VS Codec — Variation Selector Encoding > extractVSFromText	<i>should return empty vsBytes for plain text</i>	passed	0s
VS Codec — Variation Selector Encoding > extractVSFromText	<i>should handle multiple VS characters</i>	passed	0.001s

VS Codec — Variation Selector Encoding > extractVSFromText	should handle Arabic cover text	passed	0s
VS Codec — Variation Selector Encoding > buildStegoObject	should prepend VS key to cover text	passed	0s
VS Codec — Variation Selector Encoding > buildStegoObject	should handle empty cover text	passed	0s

Figure 0.5 VS Codec (Variation Selector Encoding) Tests

Text Codec (Binary ↔ Text Conversion) Tests

v E:\Project App\Embdng-Algo\tests\unit\shared\text_codec.test.js			0.602s
Text Codec — Binary ↔ Text Conversion > stringToBinary	should convert "Hi" to correct binary	passed	0.008s
Text Codec — Binary ↔ Text Conversion > stringToBinary	should return empty string for empty input	passed	0.003s
Text Codec — Binary ↔ Text Conversion > stringToBinary	should return empty string for null/undefined	passed	0.001s
Text Codec — Binary ↔ Text Conversion > stringToBinary	should handle multi-byte UTF-8 characters	passed	0.003s
Text Codec — Binary ↔ Text Conversion > stringToBinary	should produce 8 bits per ASCII character	passed	0.001s
Text Codec — Binary ↔ Text Conversion > binaryToString	should convert binary back to "Hi"	passed	0.001s
Text Codec — Binary ↔ Text Conversion > binaryToString	should return empty string for empty input	passed	0.004s
Text Codec — Binary ↔ Text Conversion > binaryToString	should return empty string for null/undefined	passed	0.001s
Text Codec — Binary ↔ Text Conversion > Roundtrip: stringToBinary ↔ binaryToString	should roundtrip ASCII text	passed	0.003s
Text Codec — Binary ↔ Text Conversion > Roundtrip: stringToBinary ↔ binaryToString	should roundtrip Arabic text	passed	0.001s

Text Codec — Binary ↔ Text Conversion > Roundtrip: stringToBinary ↔ binaryToString	should roundtrip emoji	passed	0.001s
Text Codec — Binary ↔ Text Conversion > Roundtrip: stringToBinary ↔ binaryToString	should roundtrip mixed content	passed	0.001s
Text Codec — Binary ↔ Text Conversion > bytesToBinary	should convert single byte to 8-bit string	passed	0.001s
Text Codec — Binary ↔ Text Conversion > bytesToBinary	should convert multiple bytes	passed	0.001s
Text Codec — Binary ↔ Text Conversion > bytesToBinary	should handle empty array	passed	0s
Text Codec — Binary ↔ Text Conversion > binaryToBytes	should convert 8-bit string to single byte	passed	0.005s
Text Codec — Binary ↔ Text Conversion > binaryToBytes	should convert 16-bit string to two bytes	passed	0.003s
Text Codec — Binary ↔ Text Conversion > binaryToBytes	should ignore trailing bits that don't form a complete byte	passed	0.001s
Text Codec — Binary ↔ Text Conversion > binaryToBytes	should handle empty string	passed	0.001s
Text Codec — Binary ↔ Text Conversion > Roundtrip: bytesToBinary ↔ binaryToBytes	should roundtrip arbitrary bytes	passed	0.002s
Text Codec — Binary ↔ Text Conversion > Roundtrip: bytesToBinary ↔ binaryToBytes	should roundtrip large byte array	passed	0.005s

Figure 0.6 Text Codec (Binary ↔ Text Conversion) Tests

Payload Codec (Build & Parse) Tests

v E:\Project App\Embding-Algo\tests\unit\stego\payload-codec.test.js			0.564s
Payload Codec — Build & Parse > buildPayload	should encode secret message only (no hint) → raw bytes	passed	0.016s
Payload Codec — Build & Parse > buildPayload	should encode secret message with hint → delimiter format	passed	0.012s
Payload Codec — Build & Parse > buildPayload	should return empty array for empty inputs	passed	0.001s
Payload Codec — Build & Parse > buildPayload	should return empty array for null-ish message and whitespace hint	passed	0.008s

Payload Codec — Build & Parse > buildPayload	should treat whitespace-only hint as no hint	passed	0.001s
Payload Codec — Build & Parse > buildPayload	should handle Arabic text	passed	0.002s
Payload Codec — Build & Parse > buildPayload	should handle emoji	passed	0.001s
Payload Codec — Build & Parse > parsePayload	should parse raw payload (no hint)	passed	0.001s
Payload Codec — Build & Parse > parsePayload	should parse delimited payload (with hint)	passed	0.002s
Payload Codec — Build & Parse > parsePayload	should parse empty payload	passed	0.001s
Payload Codec — Build & Parse > Roundtrip: buildPayload → parsePayload	should roundtrip message only	passed	0.001s
Payload Codec — Build & Parse > Roundtrip: buildPayload → parsePayload	should roundtrip message + hint	passed	0.001s
Payload Codec — Build & Parse > Roundtrip: buildPayload → parsePayload	should roundtrip Arabic message + hint	passed	0.001s
Payload Codec — Build & Parse > Roundtrip: buildPayload → parsePayload	should roundtrip long message	passed	0.001s

Figure 0.7 Payload Codec (Build & Parse) Tests

Brotli Compression Tests

v E:\Project App\Embding-Algo\tests\unit\compression\brotli-service.test.js			0.603s
Brotli Compression — Logic Tests (Mocked WASM) > Compression Roundtrip Logic	should compress and decompress "Hello World" correctly	passed	0.026s
Brotli Compression — Logic Tests (Mocked WASM) > Compression Roundtrip Logic	should produce smaller output for repetitive text	passed	0.003s
Brotli Compression — Logic Tests (Mocked WASM) > Compression Roundtrip Logic	should handle empty input	passed	0.007s
Brotli Compression — Logic Tests (Mocked WASM) > Compression Roundtrip Logic	should handle single byte	passed	0.006s

Brotli Compression — Logic Tests (Mocked WASM) > Compression Roundtrip Logic	<i>should handle Arabic text</i>	passed	0.003s
Brotli Compression — Logic Tests (Mocked WASM) > Compression Roundtrip Logic	<i>should handle binary data (all byte values)</i>	passed	0.006s
Brotli Compression — Logic Tests (Mocked WASM) > Compression Ratio Validation	<i>should achieve good ratio on highly repetitive text</i>	passed	0.003s
Brotli Compression — Logic Tests (Mocked WASM) > Compression Ratio Validation	<i>should achieve reasonable ratio on natural text</i>	passed	0.002s
Brotli Compression — Logic Tests (Mocked WASM) > Compression Ratio Validation	<i>may not compress random data well</i>	passed	0.004s
Brotli Compression — Logic Tests (Mocked WASM) > Compression Decision Logic (as in stego-composer)	<i>should use compression for repetitive data</i>	passed	0.002s
Brotli Compression — Logic Tests (Mocked WASM) > Compression Decision Logic (as in stego-composer)	<i>should skip compression for very short data</i>	passed	0.001s
Brotli Compression — Logic Tests (Mocked WASM) > Compression Decision Logic (as in stego-composer)	<i>should apply 0xFE marker for compressed payload</i>	passed	0.002s

Figure 0.8 Brotli Compression Tests

AES-CTR Encryption Tests

v E:\Project App\Embding-Algo\tests\unit\crypto\aes-ctr.test.js			2.234s
AES-256-CTR Encryption Engine > deriveAesCtrParams	<i>should derive keyBytes (32 bytes) and counterBytes (16 bytes)</i>	passed	0.155s
AES-256-CTR Encryption Engine > deriveAesCtrParams	<i>should produce deterministic output for same inputs</i>	passed	0.196s
AES-256-CTR Encryption Engine > deriveAesCtrParams	<i>should produce different keys for different passwords</i>	passed	0.156s
AES-256-CTR Encryption Engine > deriveAesCtrParams	<i>should produce different keys for different cover texts</i>	passed	0.147s

AES-256-CTR Encryption Engine > deriveAesCtrParams	<i>should handle empty cover text gracefully</i>	passed	0.074s
AES-256-CTR Encryption Engine > encryptPayloadCtr & decryptPayloadCtr	<i>should roundtrip encrypt→decrypt correctly</i>	passed	0.149s
AES-256-CTR Encryption Engine > encryptPayloadCtr & decryptPayloadCtr	<i>should return plainBytes unchanged when key is empty (bypass)</i>	passed	0.001s
AES-256-CTR Encryption Engine > encryptPayloadCtr & decryptPayloadCtr	<i>should have zero-overhead (ciphertext same length as plaintext)</i>	passed	0.077s
AES-256-CTR Encryption Engine > encryptPayloadCtr & decryptPayloadCtr	<i>should produce different ciphertext than plaintext</i>	passed	0.071s
AES-256-CTR Encryption Engine > encryptPayloadCtr & decryptPayloadCtr	<i>should fail to decrypt with wrong key</i>	passed	0.169s
AES-256-CTR Encryption Engine > encryptPayloadCtr & decryptPayloadCtr	<i>should fail to decrypt with wrong cover text</i>	passed	0.15s
AES-256-CTR Encryption Engine > encryptPayloadCtr & decryptPayloadCtr	<i>should handle single-byte payload</i>	passed	0.141s
AES-256-CTR Encryption Engine > encryptPayloadCtr & decryptPayloadCtr	<i>should handle large payload (1KB)</i>	passed	0.143s
AES-256-CTR Encryption Engine > encryptPayloadCtr & decryptPayloadCtr	<i>should handle Arabic text in key and cover</i>	passed	0.145s

Figure 0.9 AES-CTR Encryption Tests

Full Stego Pipeline Integration Tests

v E:\Project App\Embding-Algo\tests\integration\stego-pipeline.test.js			2.188s
Integration — Full Stego Pipeline (Embed → Extract)	should embed and extract a simple English message	passed	0.164s
Integration — Full Stego Pipeline (Embed → Extract)	should embed and extract a message with hint	passed	0.147s
Integration — Full Stego Pipeline (Embed → Extract)	should embed and extract Arabic text	passed	0.148s
Integration — Full Stego Pipeline (Embed → Extract)	should preserve cover text exactly after embedding	passed	0.145s
Integration — Full Stego Pipeline (Embed → Extract)	should fail to extract with wrong stego key	passed	0.168s
Integration — Full Stego Pipeline (Embed → Extract)	should fail to extract with wrong encryption key	passed	0.153s
Integration — Full Stego Pipeline (Embed → Extract)	should work with auto-generated stego key (empty key)	passed	0.152s
Integration — Full Stego Pipeline (Embed → Extract)	stego text should look identical to cover text when VS are stripped	passed	0.068s
Integration — Full Stego Pipeline (Embed → Extract)	should handle single character message	passed	0.142s
Integration — Full Stego Pipeline (Embed → Extract)	should throw when message exceeds cover capacity	passed	0.077s

Figure 0.10 Full Stego Pipeline Integration Tests

ESLint Security Static Analysis

The ESLint static analysis was executed using the ESLint-plugin-security and ESLint-plugin-sonarjs plugins to scan the entire codebase for potential security vulnerabilities. The scan covered 48 JavaScript files and detected 139 warnings with 0 errors, confirming that the codebase contains no critical security flaws.

ESLint Report 139 problems (0 errors, 139 warnings) - Generated on Fri Jun 05 2026 19:26:00 GMT+0300 (التوقيت العربي الرسمي)	
[+] E:\Project App\Embding-Algo\eslint.config.js	0 problems
[+] E:\Project App\Embding-Algo\jest.config.js	0 problems
[+] E:\Project App\Embding-Algo\js\core\compression\aviflavif-bridge.js	1 problem (0 errors, 1 warning)
[+] E:\Project App\Embding-Algo\js\core\compression\aviflavif-service.js	0 problems
[+] E:\Project App\Embding-Algo\js\core\compression\brotli-service.js	0 problems
[+] E:\Project App\Embding-Algo\js\core\compression\wasm-bridge.js	0 problems
[+] E:\Project App\Embding-Algo\js\core\crypto\aes-ctr.js	0 problems
[+] E:\Project App\Embding-Algo\js\core\crypto\sha256.js	0 problems
[+] E:\Project App\Embding-Algo\js\core\stego\payload-codec.js	0 problems
[+] E:\Project App\Embding-Algo\js\core\stego\png-generator.js	5 problems (0 errors, 5 warnings)
[+] E:\Project App\Embding-Algo\js\core\stego\stego-composer.js	0 problems
[+] E:\Project App\Embding-Algo\js\core\stego\vs-codec.js	0 problems
[+] E:\Project App\Embding-Algo\js\core\stego\xor-mask.js	4 problems (0 errors, 4 warnings)
[+] E:\Project App\Embding-Algo\js\docs\article-shell.js	2 problems (0 errors, 2 warnings)
[+] E:\Project App\Embding-Algo\js\docs\docs.js	21 problems (0 errors, 21 warnings)
[+] E:\Project App\Embding-Algo\js\docs\nav-data.js	0 problems
[+] E:\Project App\Embding-Algo\js\features\F_chat_parser.js	11 problems (0 errors, 11 warnings)
[+] E:\Project App\Embding-Algo\js\features\F_chat_scanner.js	6 problems (0 errors, 6 warnings)
[+] E:\Project App\Embding-Algo\js\features\F_stego_analysis.js	2 problems (0 errors, 2 warnings)
[+] E:\Project App\Embding-Algo\js\features\F_stego_capacity.js	0 problems
[+] E:\Project App\Embding-Algo\js\features\F_stego_hint.js	5 problems (0 errors, 5 warnings)
[+] E:\Project App\Embding-Algo\js\features\embed-controller.js	1 problem (0 errors, 1 warning)

[+] E:\Project App\Embding-Algo\js\features\extract-controller.js	0 problems
[+] E:\Project App\Embding-Algo\js\features\forensics-dashboard.js	28 problems (0 errors, 28 warnings)
[+] E:\Project App\Embding-Algo\js\features\page-loader.js	0 problems
[+] E:\Project App\Embding-Algo\js\shared\template-loader.js	0 problems
[+] E:\Project App\Embding-Algo\js\shared\text_codec.js	1 problem (0 errors, 1 warning)
[+] E:\Project App\Embding-Algo\js\shared\theme-manager.js	15 problems (0 errors, 15 warnings)
[+] E:\Project App\Embding-Algo\js\shared\ui-helpers.js	8 problems (0 errors, 8 warnings)
[+] E:\Project App\Embding-Algo\js\utils.js	0 problems
[+] E:\Project App\Embding-Algo\stegnolines_ui_library\js\docs\article-shell.js	2 problems (0 errors, 2 warnings)
[+] E:\Project App\Embding-Algo\stegnolines_ui_library\js\docs\docs.js	0 problems
[+] E:\Project App\Embding-Algo\stegnolines_ui_library\js\docs\nav-data.js	0 problems
[+] E:\Project App\Embding-Algo\stegnolines_ui_library\js\features\F_chat_parser.js	10 problems (0 errors, 10 warnings)
[+] E:\Project App\Embding-Algo\stegnolines_ui_library\js\features\F_chat_scanner.js	5 problems (0 errors, 5 warnings)
[+] E:\Project App\Embding-Algo\stegnolines_ui_library\js\features\F_stego_analysis.js	1 problem (0 errors, 1 warning)
[+] E:\Project App\Embding-Algo\stegnolines_ui_library\js\features\F_stego_capacity.js	0 problems
[+] E:\Project App\Embding-Algo\stegnolines_ui_library\js\features\F_stego_hint.js	1 problem (0 errors, 1 warning)
[+] E:\Project App\Embding-Algo\stegnolines_ui_library\js\shared\text_codec.js	1 problem (0 errors, 1 warning)
[+] E:\Project App\Embding-Algo\stegnolines_ui_library\js\stage3_hide\embedding.js	0 problems
[+] E:\Project App\Embding-Algo\stegnolines_ui_library\js\stage3_hide\extraction.js	0 problems
[+] E:\Project App\Embding-Algo\stegnolines_ui_library\js\stage3_hide\step1_payload.js	0 problems
[+] E:\Project App\Embding-Algo\stegnolines_ui_library\js\stage3_hide\step2_png.js	5 problems (0 errors, 5 warnings)
[+] E:\Project App\Embding-Algo\stegnolines_ui_library\js\stage3_hide\step3_xor.js	4 problems (0 errors, 4 warnings)
[+] E:\Project App\Embding-Algo\stegnolines_ui_library\js\stage3_hide\step4_vs_codec.js	0 problems
[+] E:\Project App\Embding-Algo\stegnolines_ui_library\js\stage3_hide\step5_stego_object.js	0 problems
[+] E:\Project App\Embding-Algo\stegnolines_ui_library\js\utils.js	0 problems

Figure 0.11 ESLint Security Static Code Analysis

10. Terms and Acronyms

Acronym	term
AES	Advanced Encryption Standard
AES-CTR	Advanced Encryption Standard - Counter Mode
ASCII	American Standard Code for Information Interchange
CIA	Confidentiality, Integrity, Availability
CT	Cover Text
CSS / CSS3	Cascading Style Sheets
CVE	Common Vulnerabilities and Exposures
DOM	Document Object Model
FPE	Format-Preserving Encryption
HMAC	Hash-based Message Authentication Code
HTML / HTML5	Hypertext Markup Language
IDE	Integrated Development Environment
IP	Invisible Plus
IS	Invisible Separator
ISS	Inhabit Symmetric Swapping
IV	Initialization Vector
JS	JavaScript
LRE	Left-To-Right Embedding
LRM	Left-to-Right Mark
LRO	Left-to-Right Override
LZW	Lempel–Ziv–Welch
NIST	National Institute of Standards and Technology
NSA	National Security Agency
OCR	Optical Character Recognition
iOS	iPhone Operating System
OTP	One-Time Pad
PBKDF2	Password-Based Key Derivation Function 2
PDF	Pop Directional Formatting (<i>also used as Portable Document Format</i>)
PRNG	Pseudo-Random Number Generator
Regex	Regular Expressions

Acronym	term
SHA / SHA-256	Secure Hash Algorithm 256-bit
ST	Secret Text
UI	User Interface
URL	Uniform Resource Locator
UTF-8	Unicode Transformation Format - 8-bit
VS	Variation Selectors
W3C	World Wide Web Consortium
WASM	WebAssembly
XOR	Exclusive OR
XSS	Cross-Site Scripting
ZWC	Zero Width Characters
ZWJ	Zero Width Joiner
ZWNJ	Zero Width Non-Joiner

Table 10.1 terms